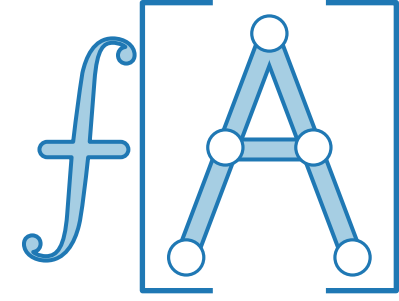
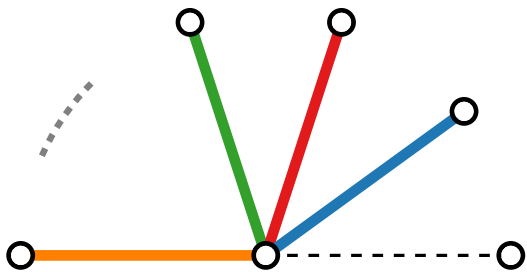
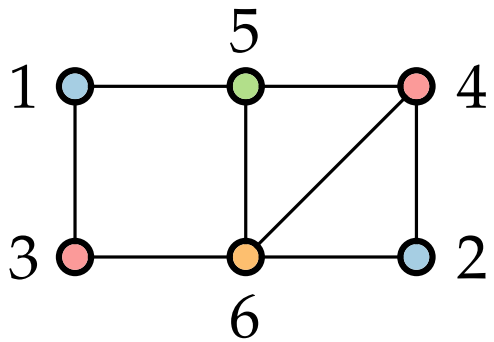




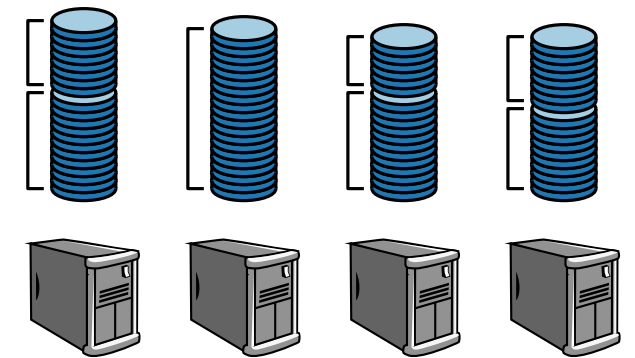
Fortgeschrittene Algorithmen

21. Vorlesung

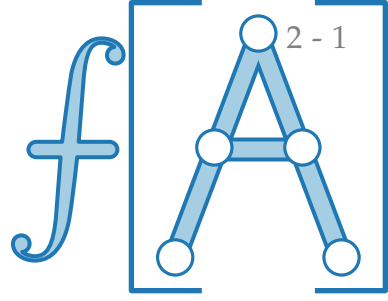
Approximationsalgorithmen



Teil I: Definitionen

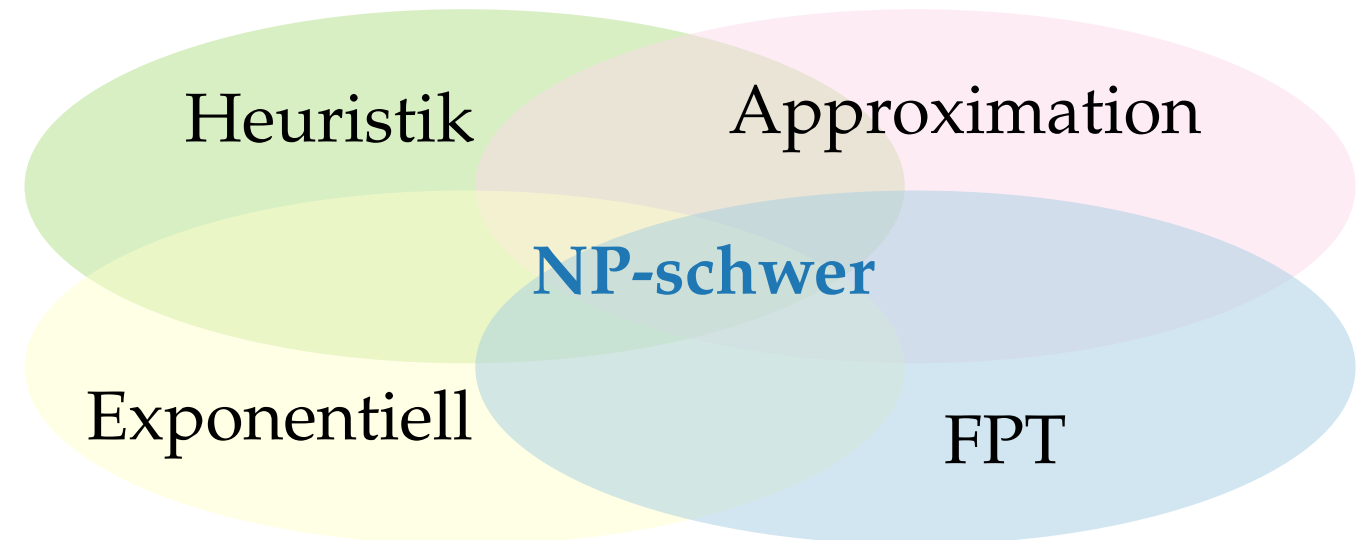


NP-schwere Probleme lösen

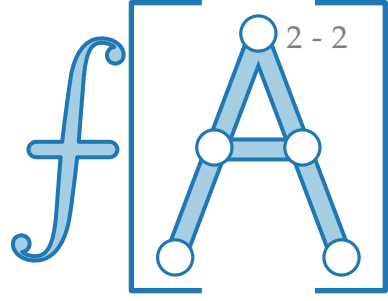


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

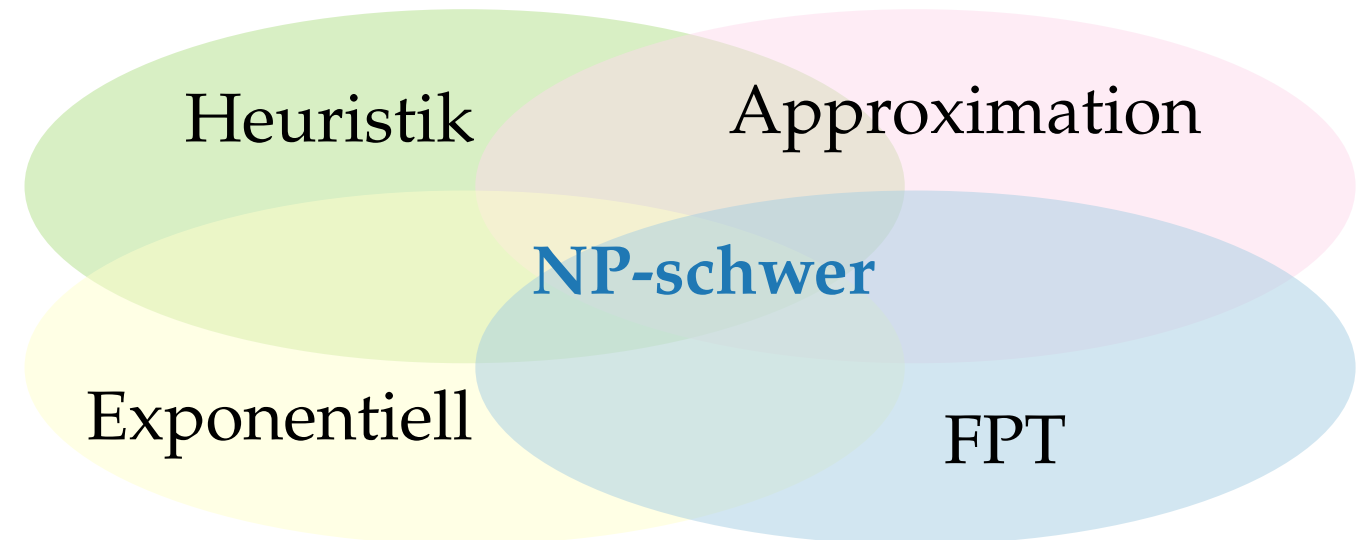


NP-schwere Probleme lösen

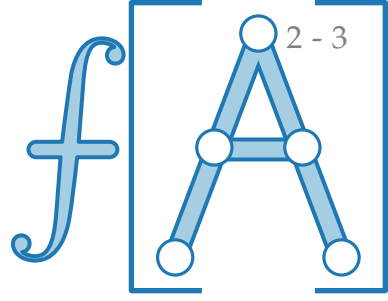


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

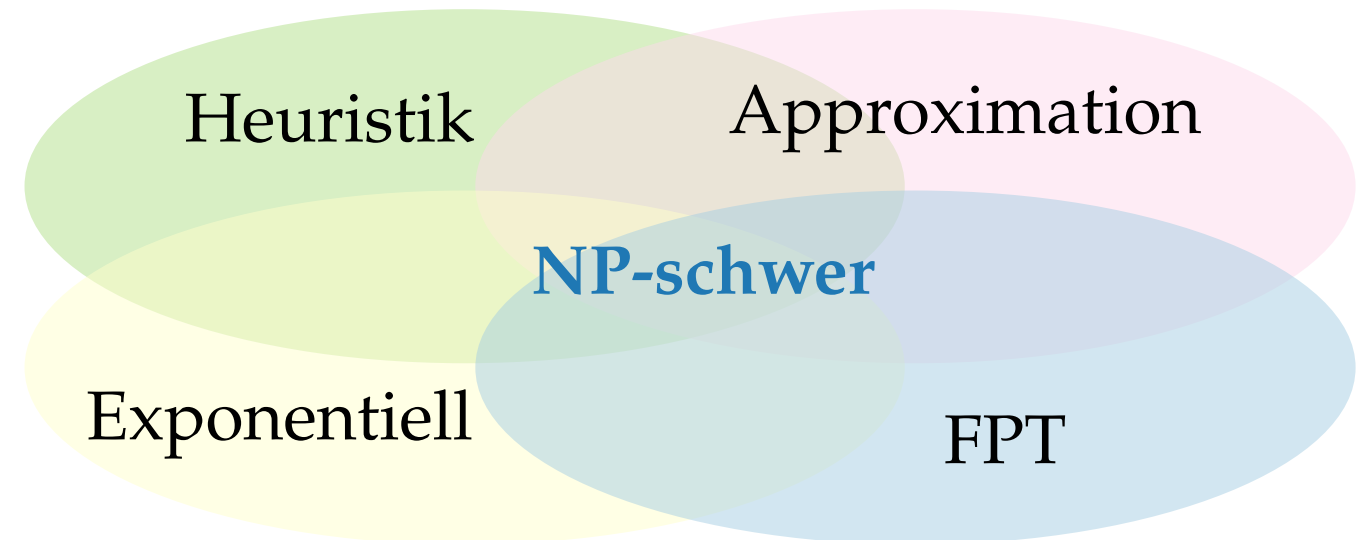


NP-schwere Probleme lösen

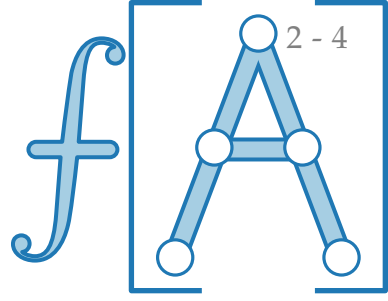


Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen

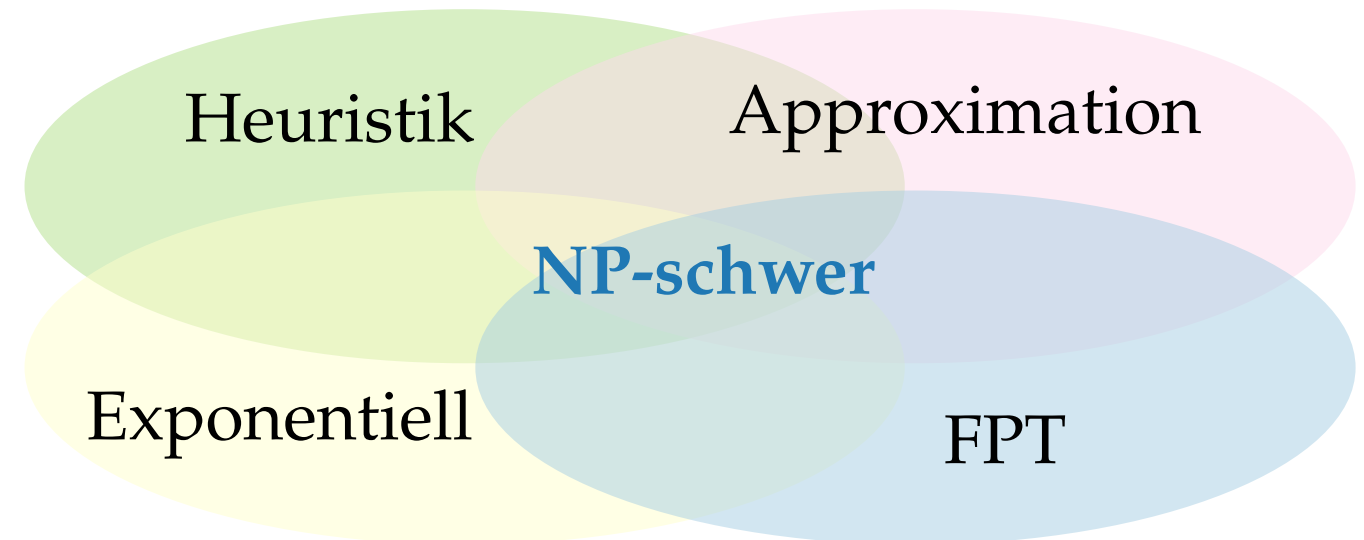


NP-schwere Probleme lösen



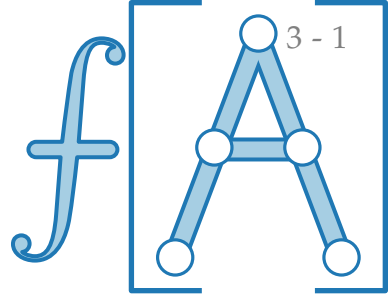
Was können wir tun?

- Optimalität für Geschwindigkeit opfern
 - Heuristiken (Simulated Annealing, Tabu-Search)
 - Approximationsalgorithmen heute
- Optimale Lösungen
 - Exakte Exponentialzeitalgorithmen
 - feinkörnige Analyse
 - parametrisierte Algorithmen



Approximationsalgorithmen

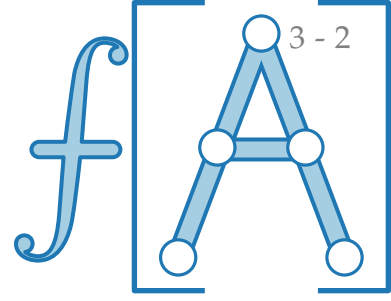
Problem.



Approximationsalgorithmen

Problem.

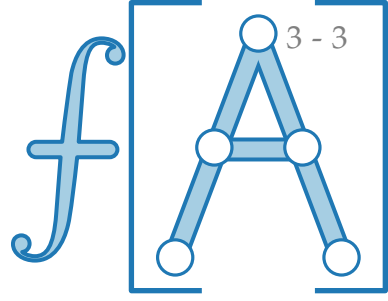
- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).



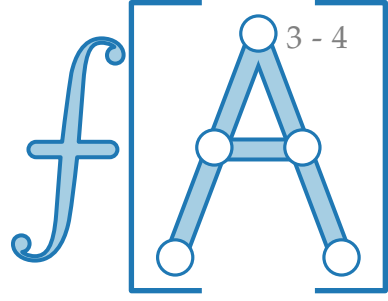
Approximationsalgorithmen

Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.



Approximationsalgorithmen



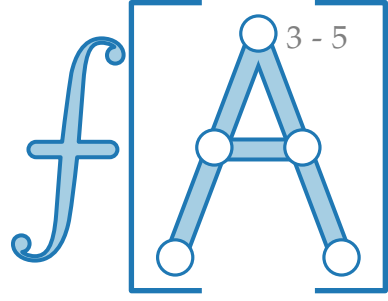
Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

Ziel.

- Entwickle **Approximationsalgorithmen**, die

Approximationsalgorithmen



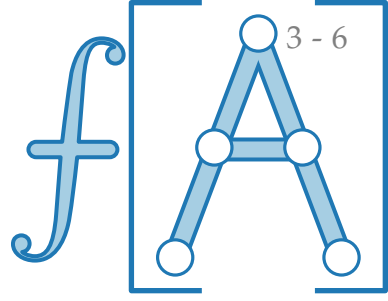
Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen

Approximationsalgorithmen



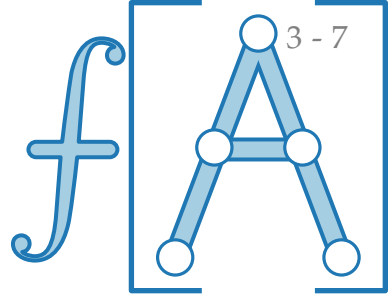
Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden

Approximationsalgorithmen



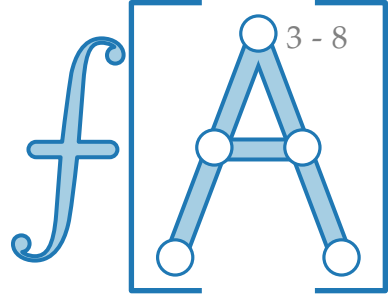
Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden
- Studiere Techniken für Design und Analyse von Approximationsalgorithmen

Approximationsalgorithmen



Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

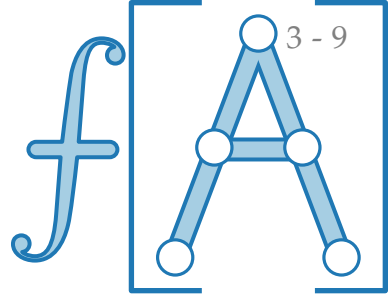
Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden
- Studiere Techniken für Design und Analyse von Approximationsalgorithmen

Überblick.

- Approximationsalgorithmen, die Lösungen berechnen, die

Approximationsalgorithmen



Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

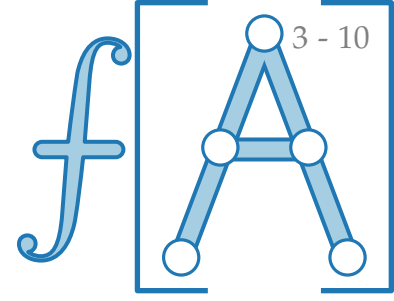
Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden
- Studiere Techniken für Design und Analyse von Approximationsalgorithmen

Überblick.

- Approximationsalgorithmen, die Lösungen berechnen, die
 - additive Gütegarantie haben

Approximationsalgorithmen



Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

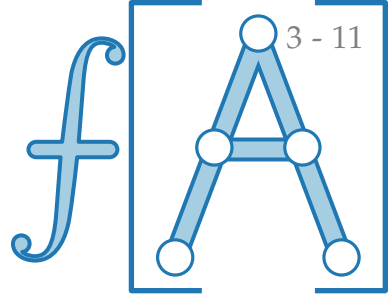
Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden
- Studiere Techniken für Design und Analyse von Approximationsalgorithmen

Überblick.

- Approximationsalgorithmen, die Lösungen berechnen, die
 - additive Gütegarantie haben
 - relative Gütegarantie haben

Approximationsalgorithmen



Problem.

- Für NP-schwere Optimierungsprobleme können wir nicht effizient jede Instanz optimal lösen (außer $P = NP$).
- Heuristiken geben keine Gütegarantie.

Ziel.

- Entwickle **Approximationsalgorithmen**, die
 - in Polynomialzeit laufen
 - Lösungen mit Gütegarantie finden
- Studiere Techniken für Design und Analyse von Approximationsalgorithmen

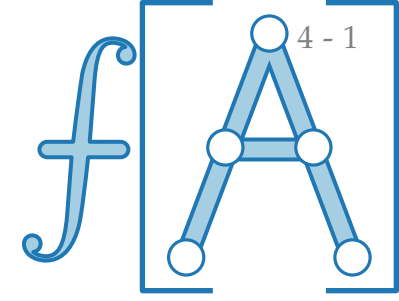
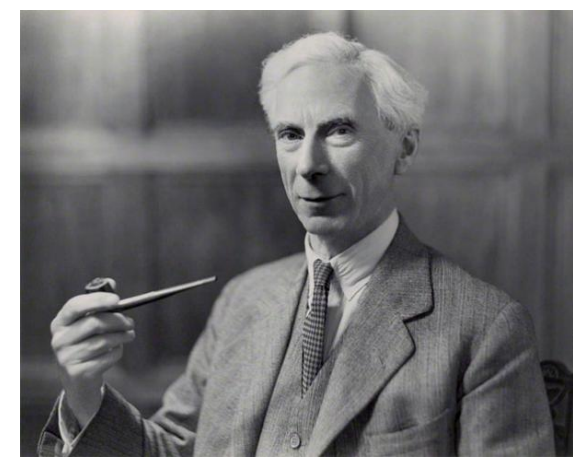
Überblick.

- Approximationsalgorithmen, die Lösungen berechnen, die
 - additive Gütegarantie haben
 - relative Gütegarantie haben
 - „beliebig gut“ sind.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

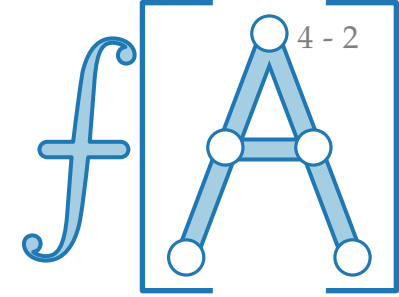
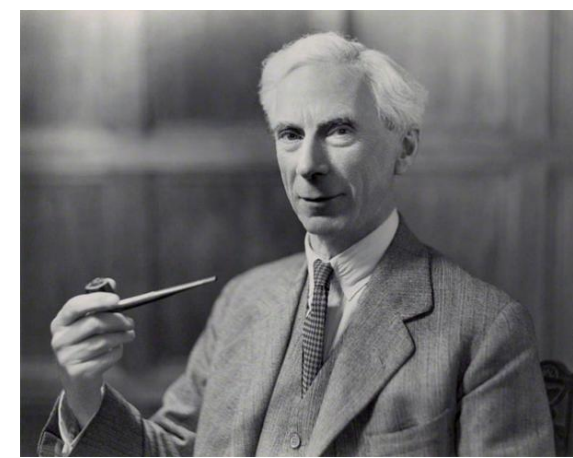
Bertrand Russell (1872–1970)



Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“ Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

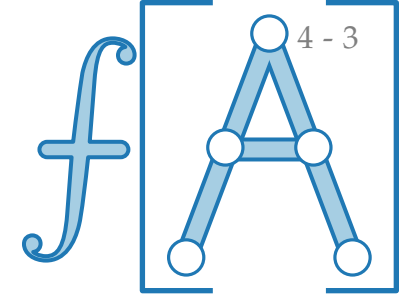
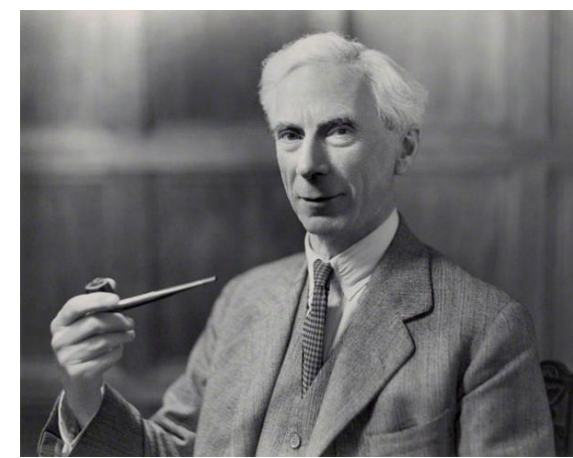


Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.



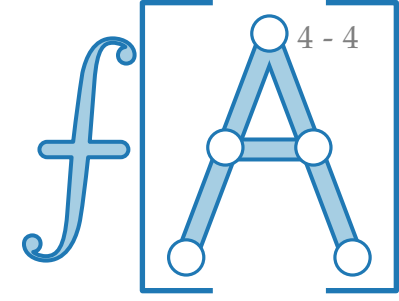
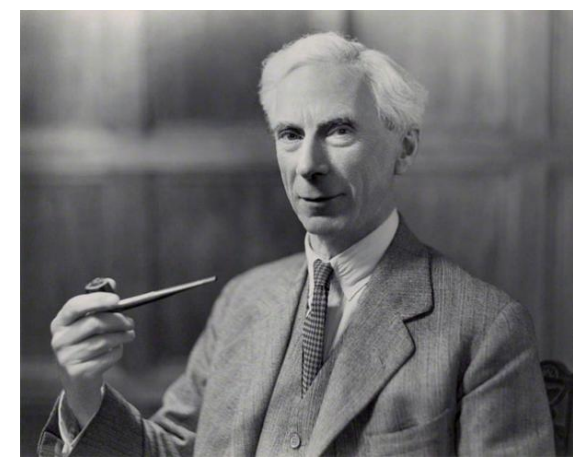
z.B. Ablaufplanung

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“
Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.



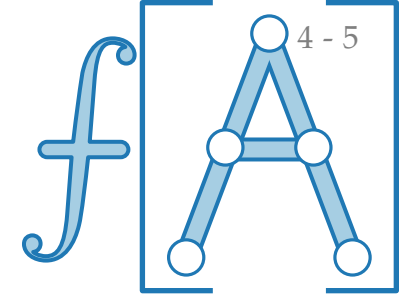
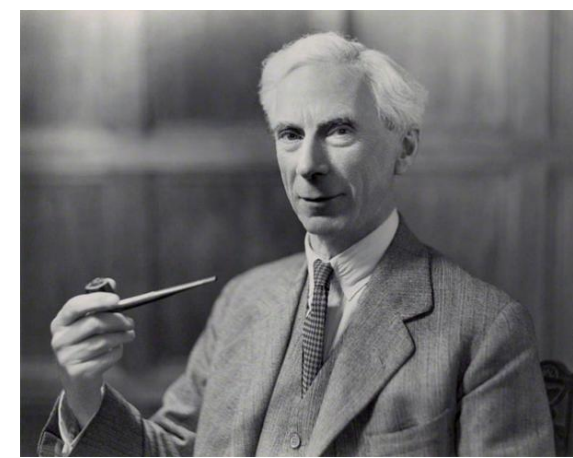
z.B. Ablaufplanung

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“
Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.



z.B. Ablaufplanung

$$\zeta = \ell$$

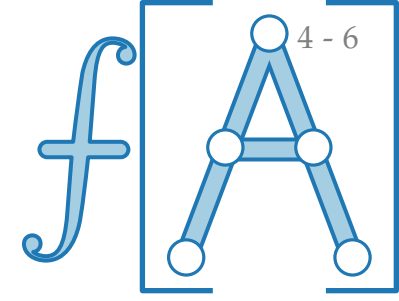
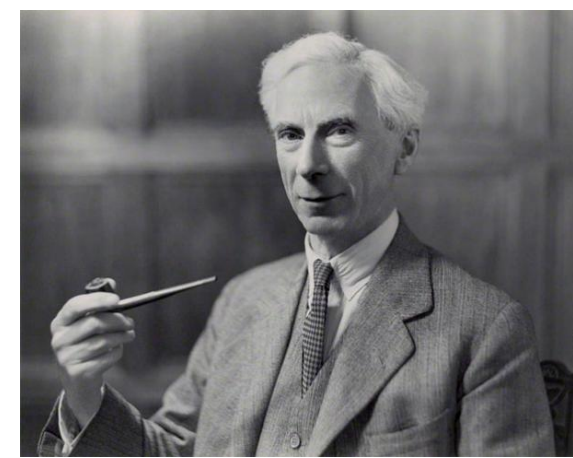
Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“
Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .



z.B. Ablaufplanung

$$\zeta = \ell$$

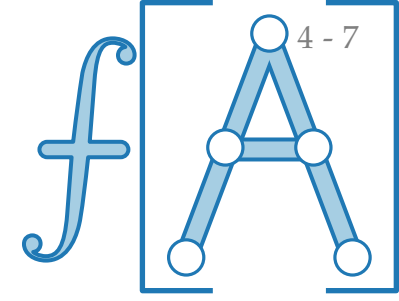
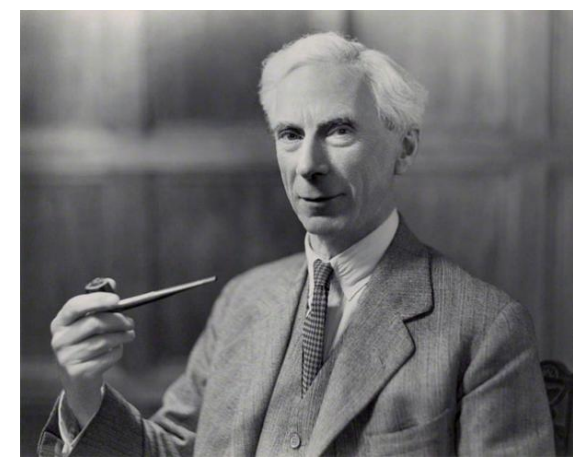
Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“
Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .



z.B. Ablaufplanung

$$\zeta = \ell$$

$$\gamma = 1/3$$

Approximationsalgorithmen

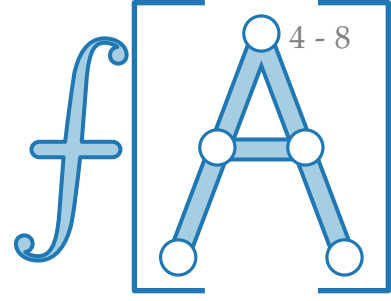
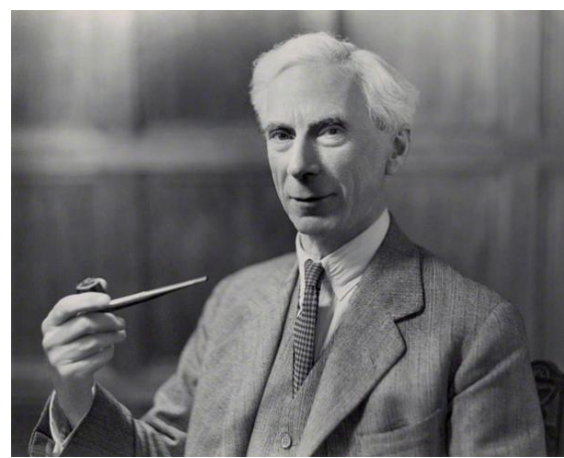
„All exact science is dominated by the idea of approximation.“
Bertrand Russell (1872–1970)

Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn



z.B. Ablaufplanung

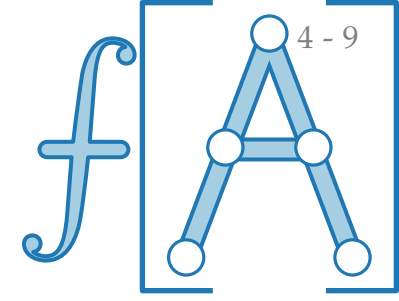
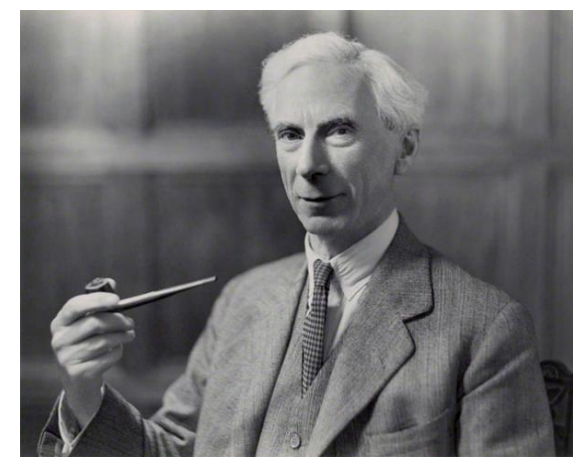
$$\zeta = \ell$$

$$\gamma = 1/3$$

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

z.B. Ablaufplanung

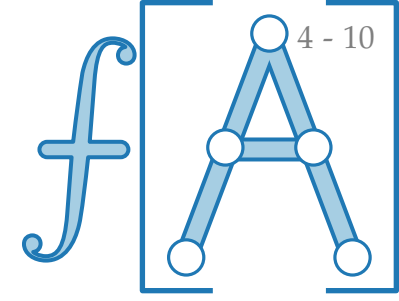
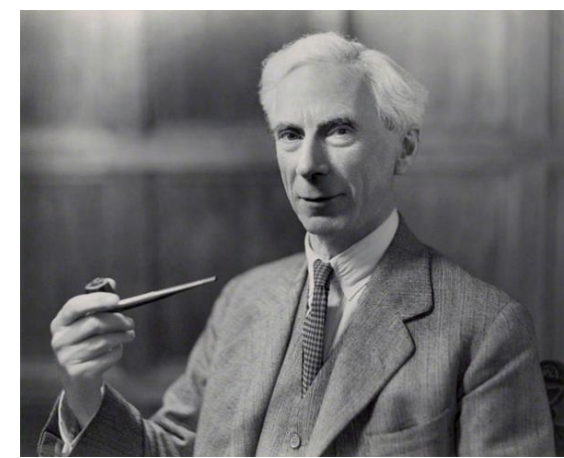
$$\zeta = \ell$$

$$\gamma = 1/3$$

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

z.B. Ablaufplanung

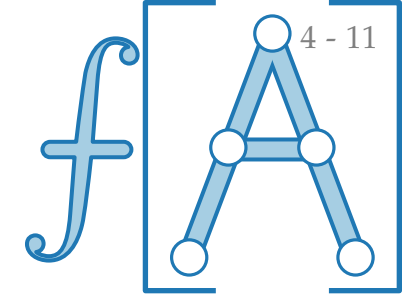
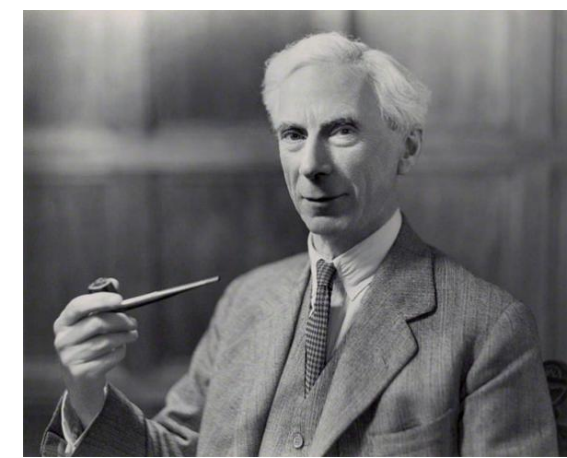
$$\zeta = \ell$$

$$\gamma = 1/3$$

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

z.B. Ablaufplanung

$$\zeta = \ell$$

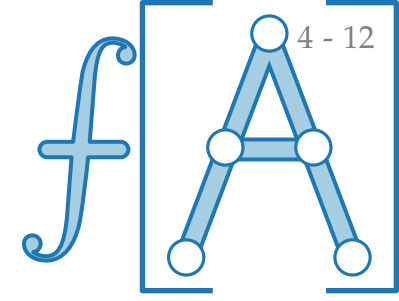
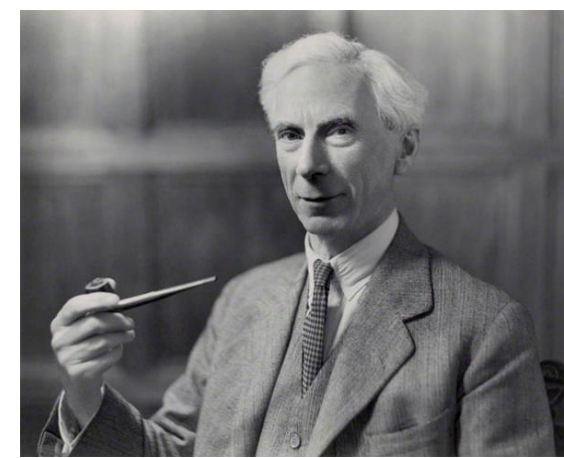
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

z.B. Ablaufplanung

$$\zeta = \ell$$

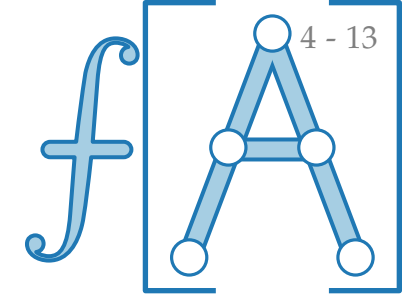
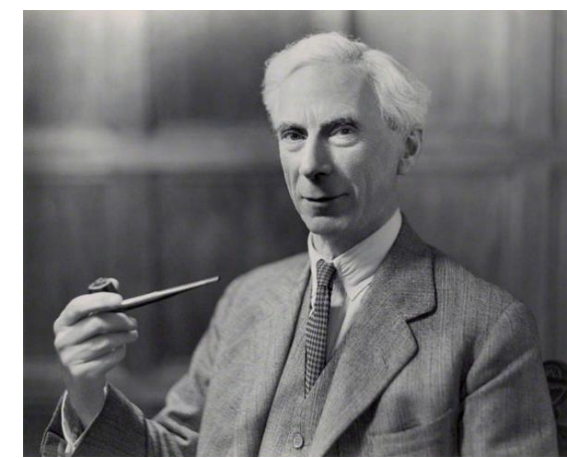
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

$\zeta(\text{optimale Lösung})$

z.B. Ablaufplanung

$$\zeta = \ell$$

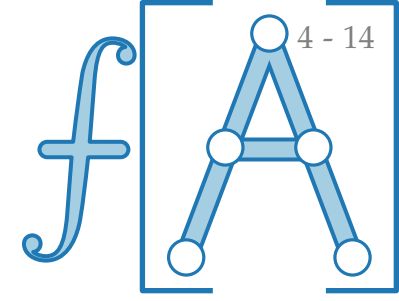
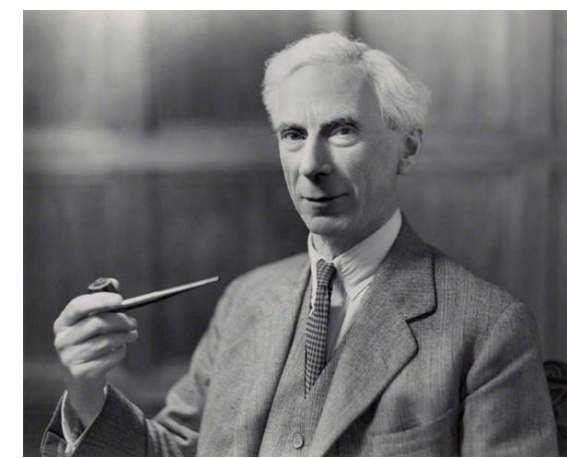
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

$\zeta(\text{optimale Lösung})$

- die Laufzeit von $\mathcal{A}$ polynomiell in $|I|$ ist.

z.B. Ablaufplanung

$$\zeta = \ell$$

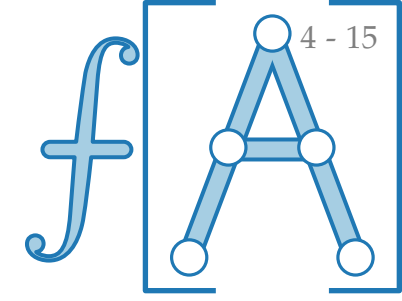
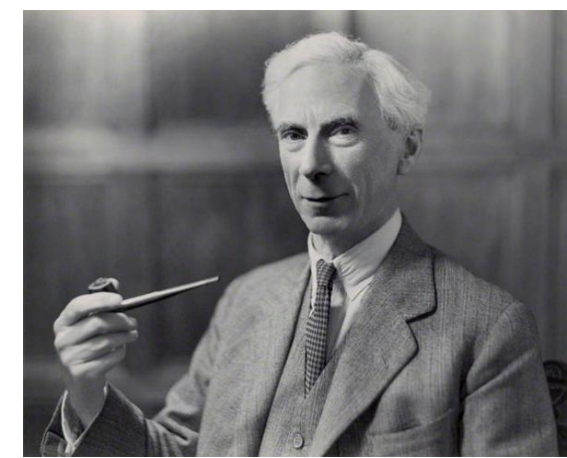
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

$\zeta(\text{optimale Lösung})$

- die Laufzeit von $\mathcal{A}$ polynomiell in $|I|$ ist.

z.B. Ablaufplanung

$$\zeta = \ell$$

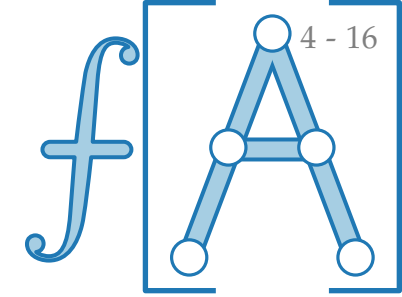
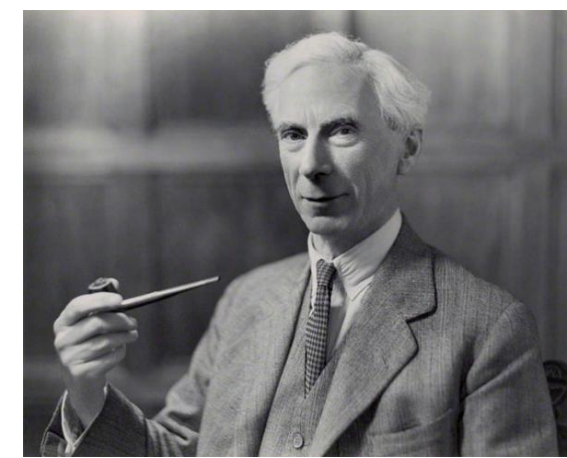
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

$\zeta(\text{optimale Lösung})$

Größe der Instanz I

- die Laufzeit von $\mathcal{A}$ polynomiell in $|I|$ ist.

z.B. Ablaufplanung

$$\zeta = \ell$$

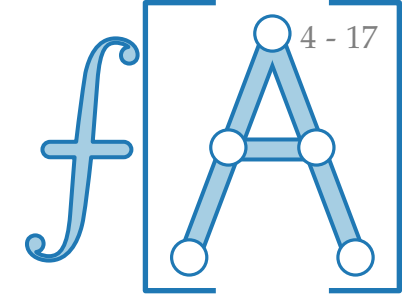
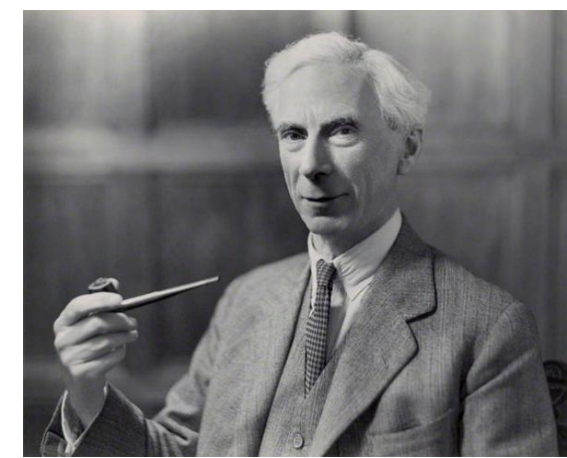
$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

Approximationsalgorithmen

„All exact science is dominated by the idea of approximation.“

Bertrand Russell (1872–1970)



Sei Π ein **Maximierungsproblem**.

Sei ζ die **Zielfunktion** von Π : Lösung $\mapsto \mathbb{Q}_{\geq 0}$.

Sei γ eine Zahl ≤ 1 .

Ein Algorithmus $\mathcal{A}$ heißt **γ -Approximation**, wenn

- $\mathcal{A}$ für jede Instanz I von Π eine Lösung $\mathcal{A}(I)$ berechnet, so dass

$$\frac{\zeta(\mathcal{A}(I))}{\text{OPT}(I)} \geq \gamma$$

$\zeta(\text{optimale Lösung})$

Größe der Instanz I

- die Laufzeit von $\mathcal{A}$ polynomiell in $|I|$ ist.

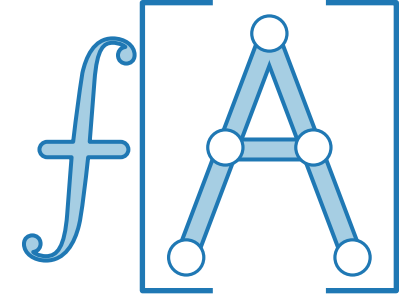
z.B. Ablaufplanung

$$\zeta = \ell$$

$$\gamma = 1/3$$

1/3-Approx. liefert Menge von Aktivitäten, deren Gesamtlänge mindestens 1/3 der maximal möglichen Länge ist.

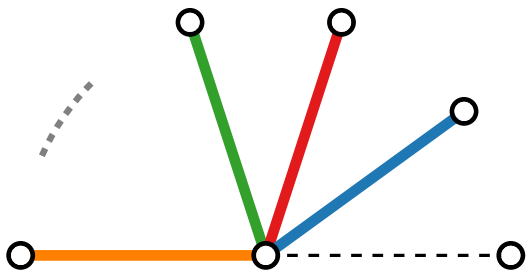
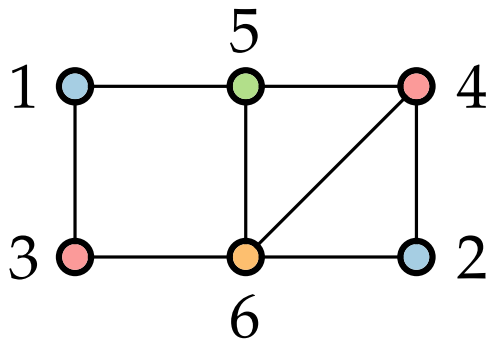
$$\mathcal{O}(n \log n)$$



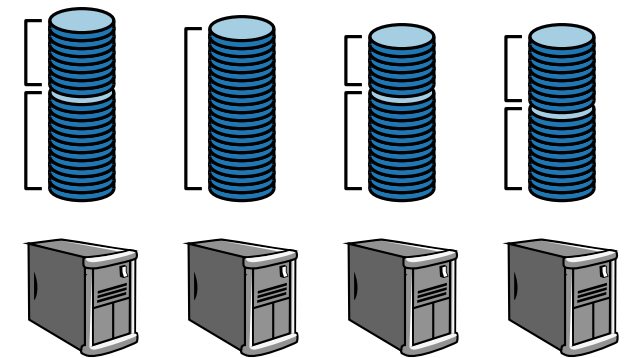
Fortgeschrittene Algorithmen

21. Vorlesung

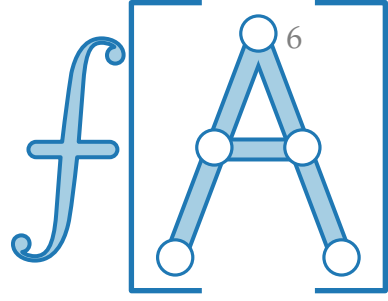
Approximationsalgorithmen



Teil II: Ablaufplanung

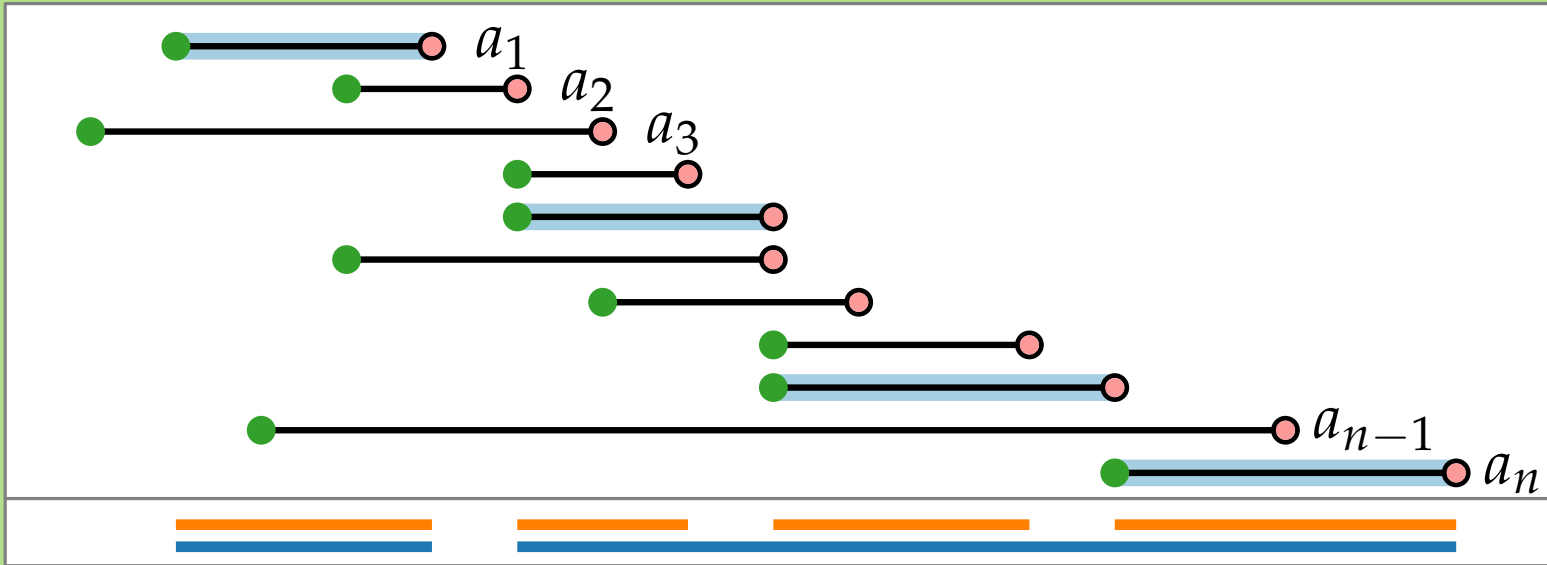


Ablaufplanung Reloaded



Problem. ABLAUFPLANUNG'

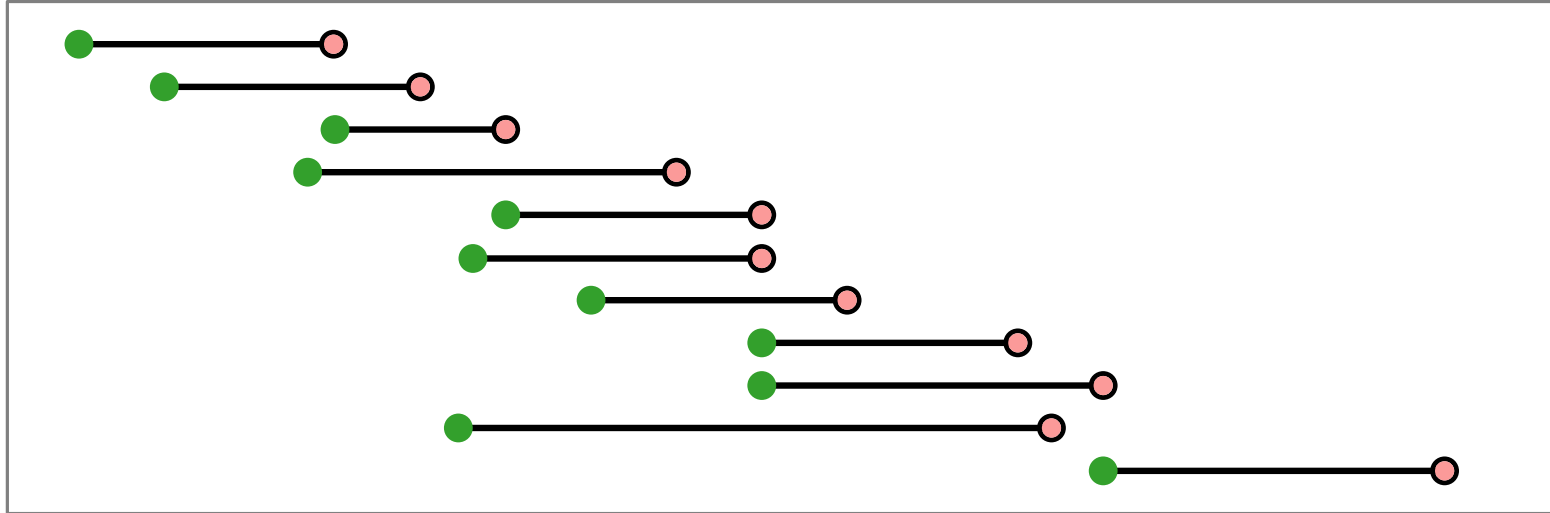
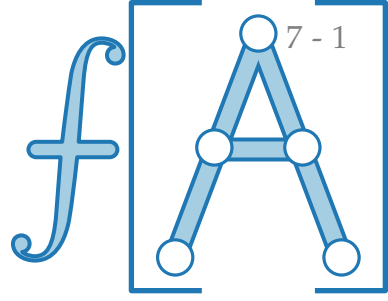
Geg. Menge $A = \{a_1, \dots, a_n\}$ von **halboffenen Intervallen**, wobei für $i = 1, \dots, n$ gilt $a_i = [s_i, e_i)$.



Ges. eine Menge $A' \subseteq A$ paarweise disjunkter Intervalle, deren **Gesamtlänge $\ell(A')$ maximal** ist.

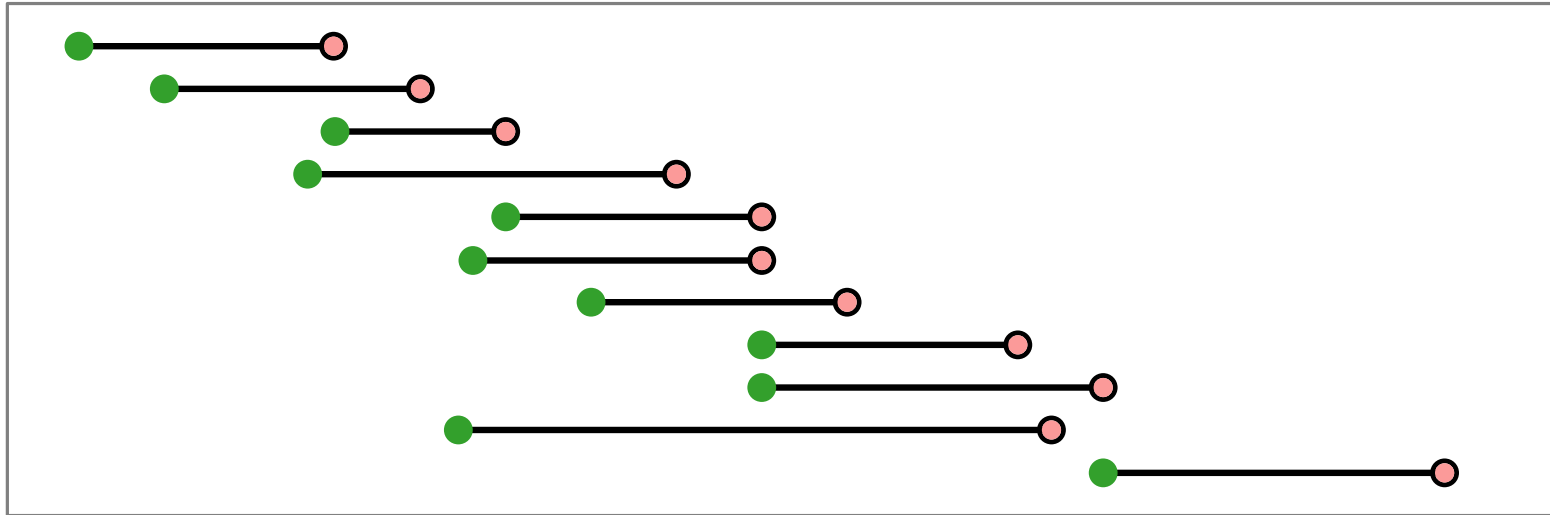
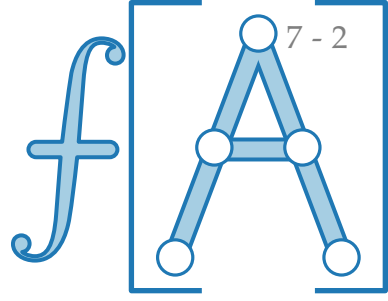
Greedy. Nimm längste Aktivität, streiche dazu inkompatible Aktivitäten und iteriere.

Wie gut/schlecht ist der Greedy Algorithmus?



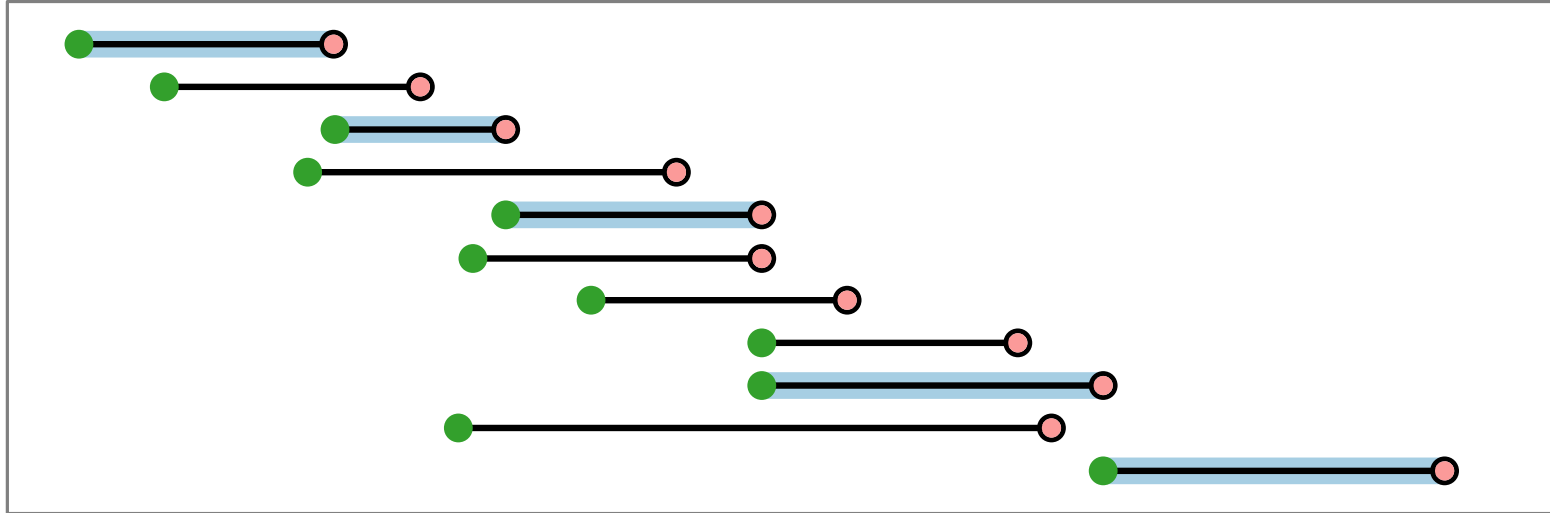
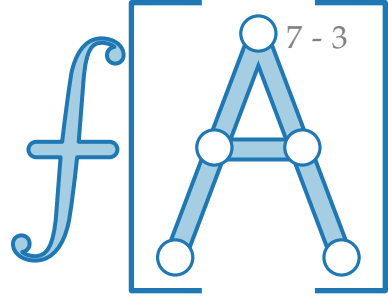
Wie gut/schlecht ist der Greedy Algorithmus?

Betrachte eine optimale Lösung $L \subseteq A$.

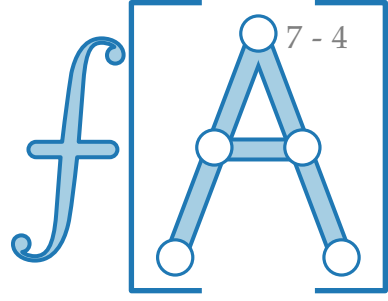


Wie gut/schlecht ist der Greedy Algorithmus?

Betrachte eine optimale Lösung $L \subseteq A$.

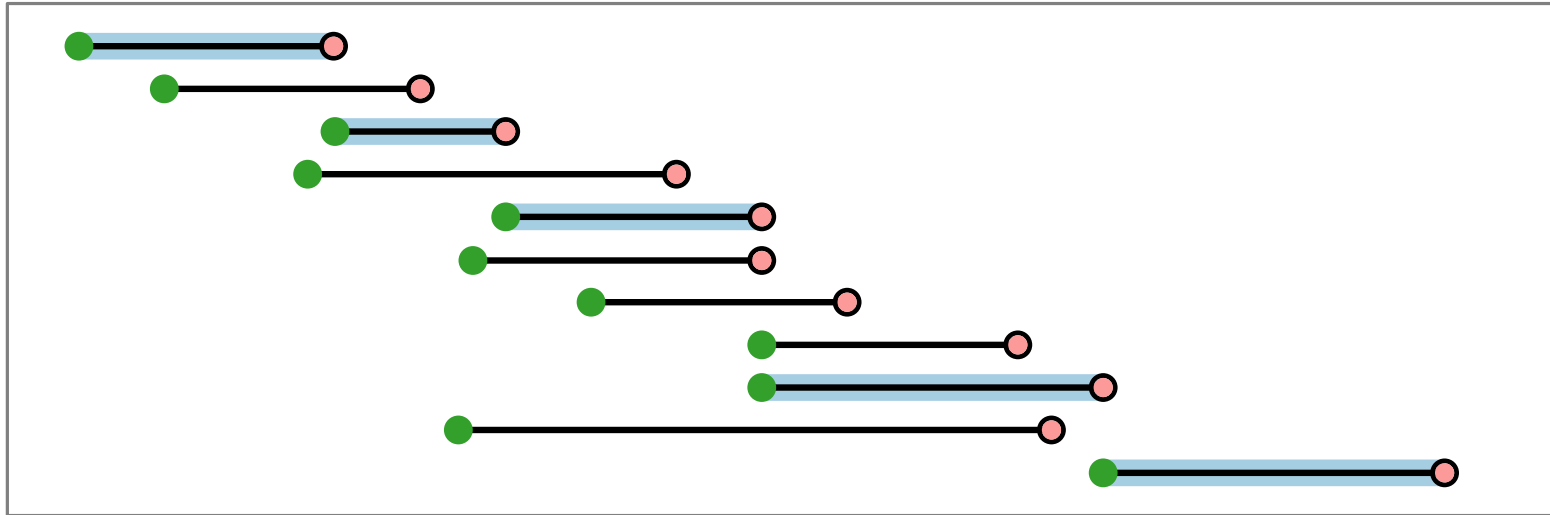


Wie gut/schlecht ist der Greedy Algorithmus?

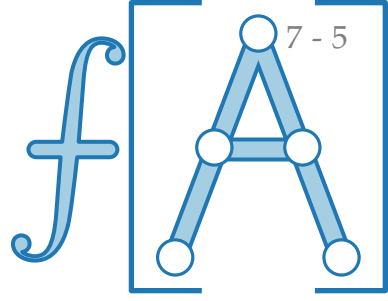


Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

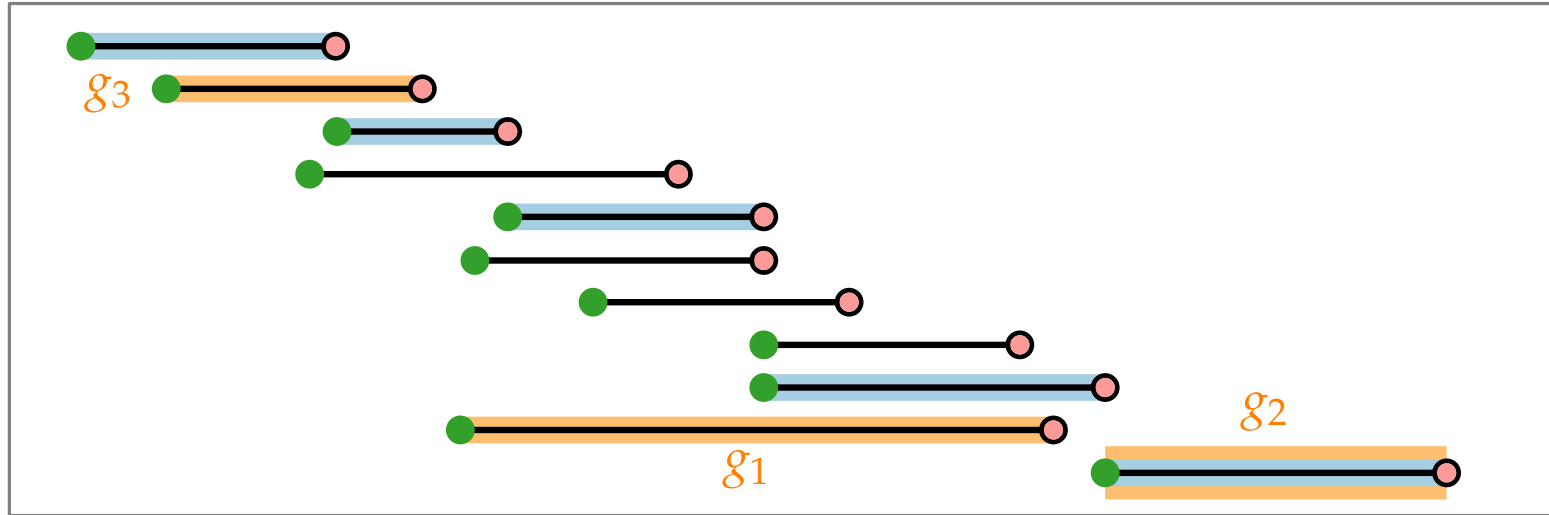


Wie gut/schlecht ist der Greedy Algorithmus?

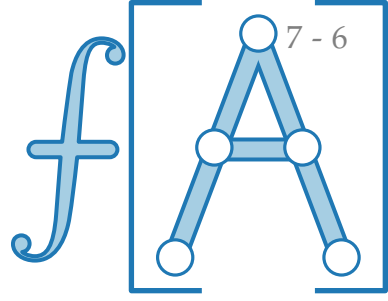


Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)



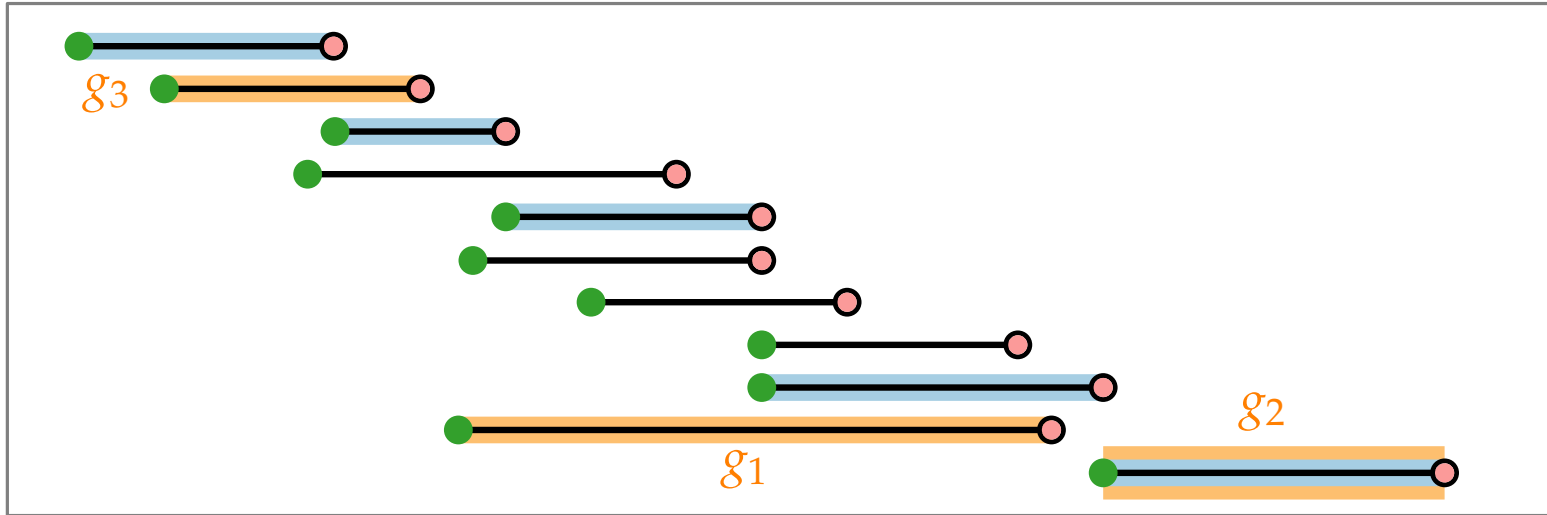
Wie gut/schlecht ist der Greedy Algorithmus?



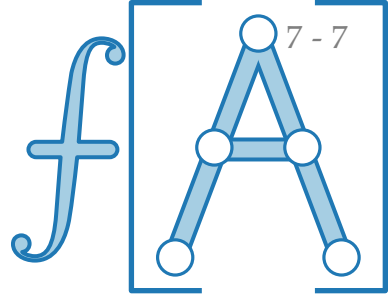
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\}$



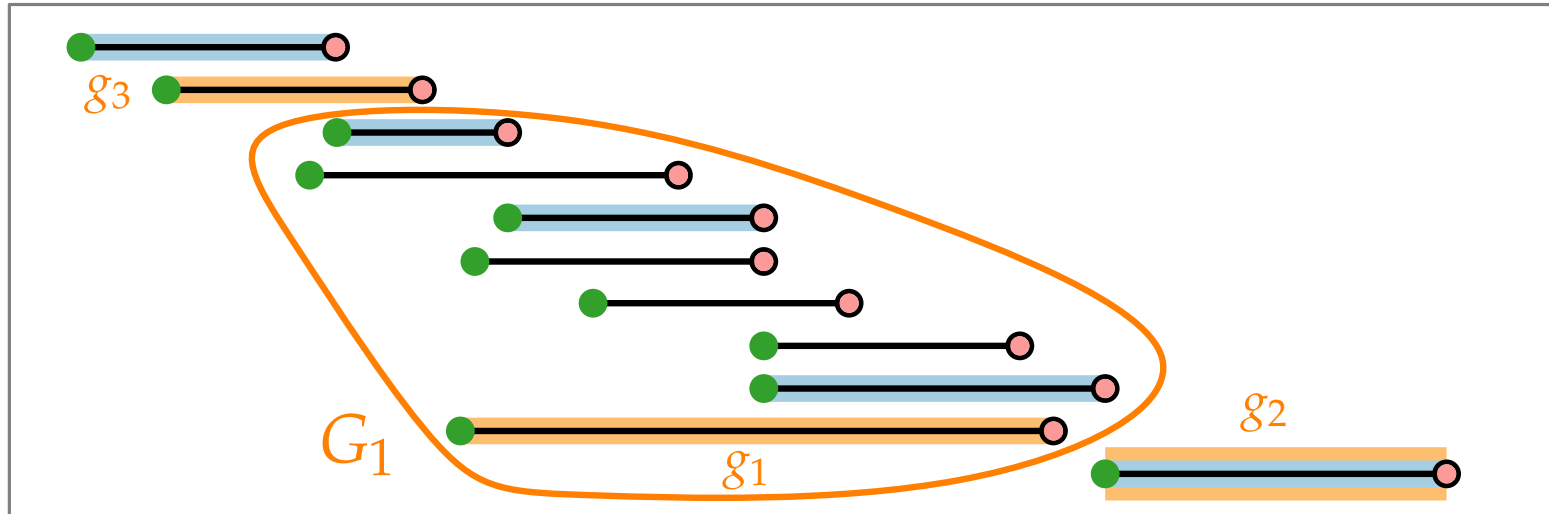
Wie gut/schlecht ist der Greedy Algorithmus?



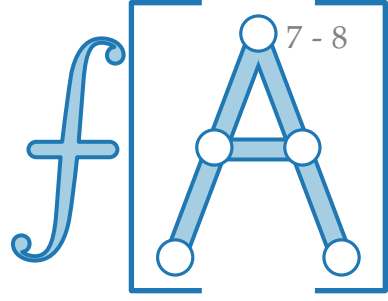
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\}$



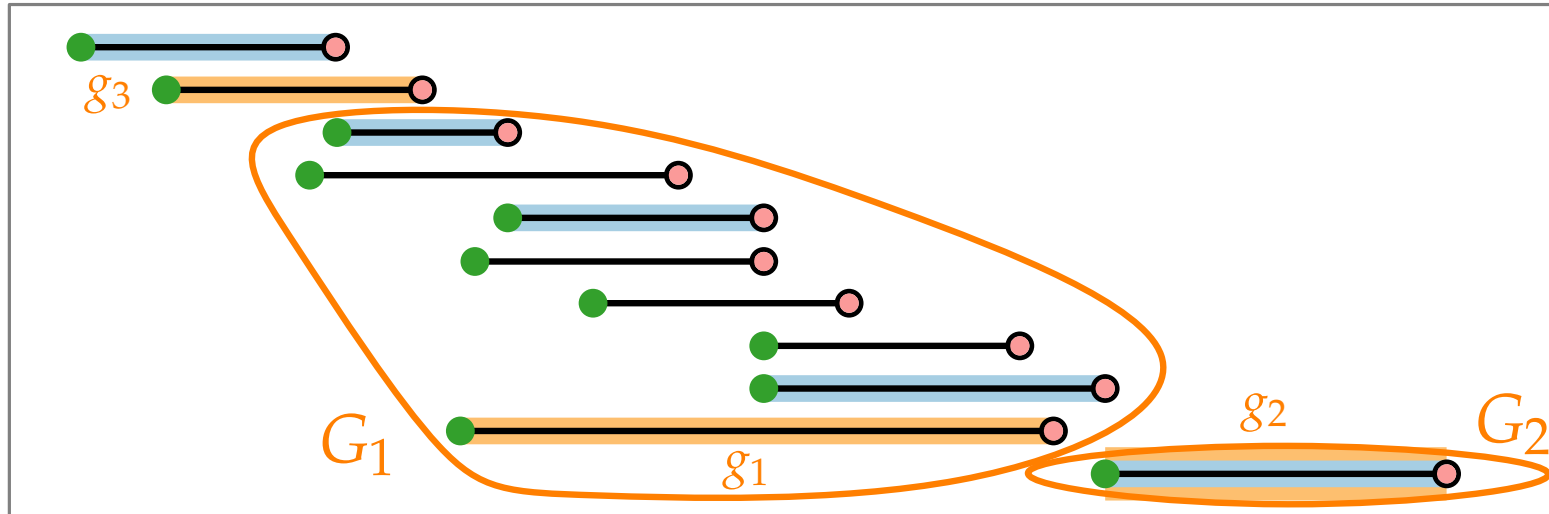
Wie gut/schlecht ist der Greedy Algorithmus?



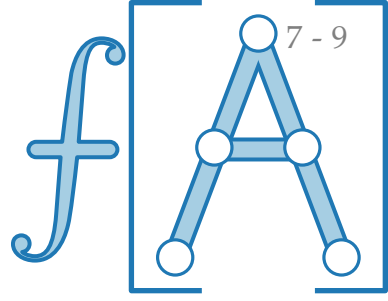
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\}$



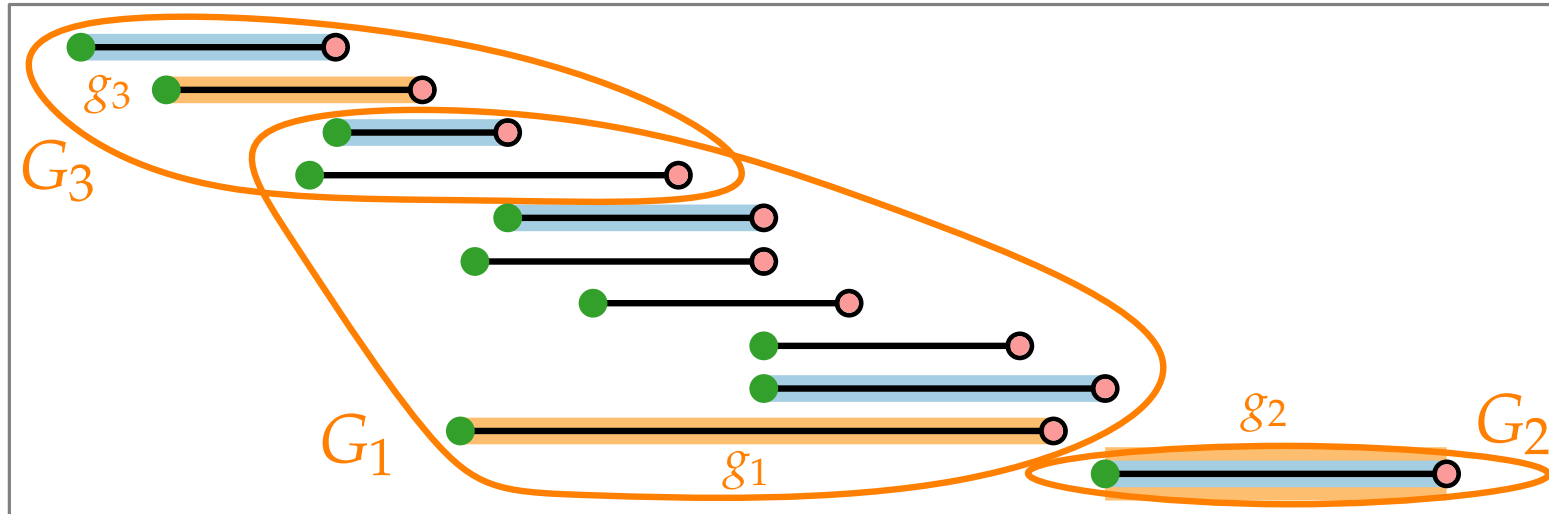
Wie gut/schlecht ist der Greedy Algorithmus?



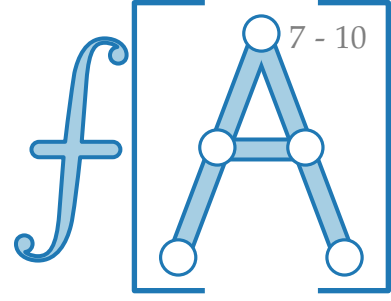
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\}$



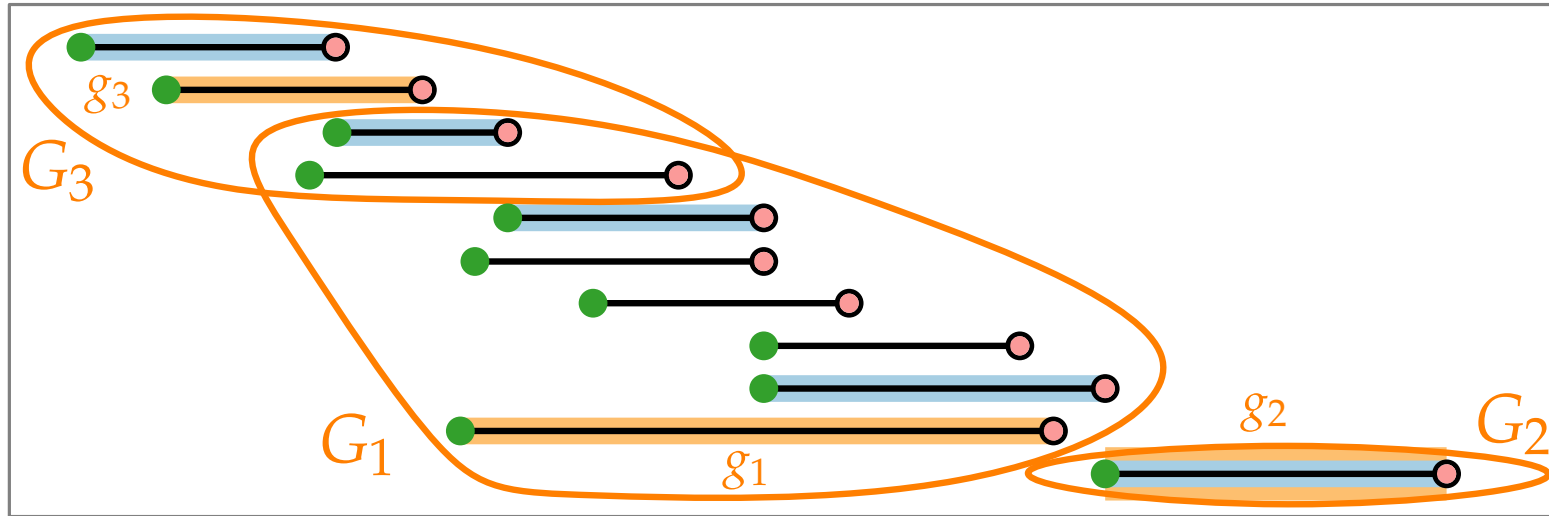
Wie gut/schlecht ist der Greedy Algorithmus?



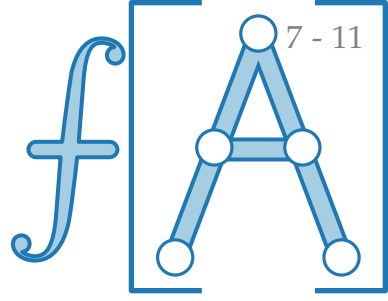
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



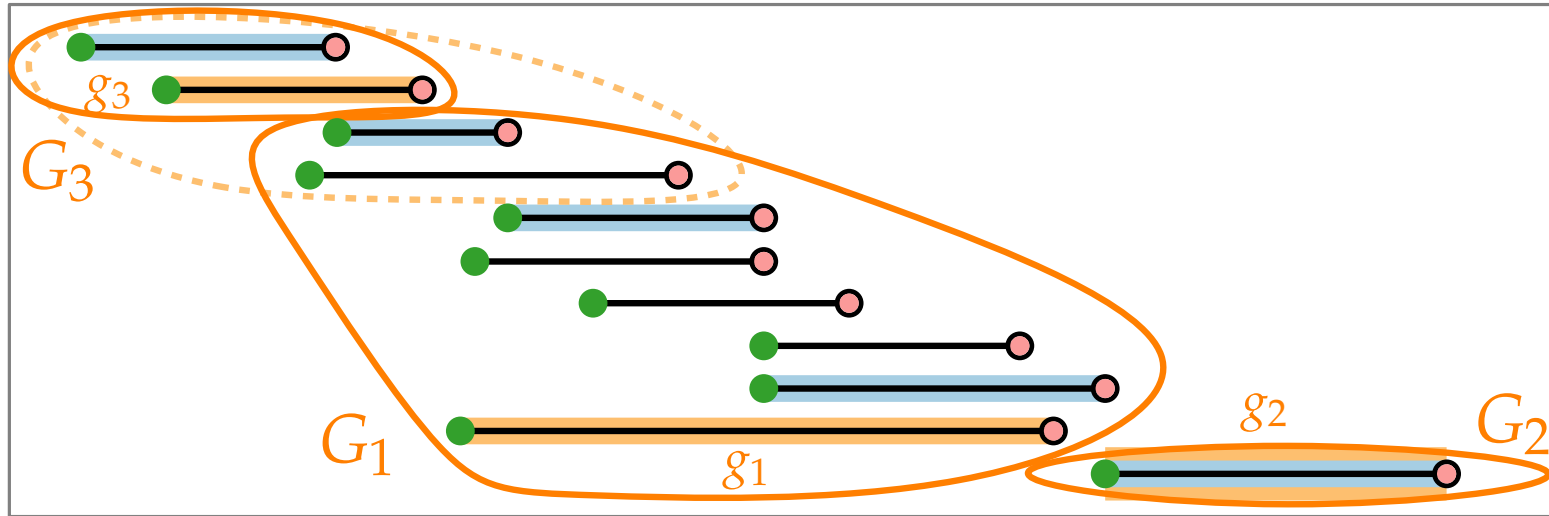
Wie gut/schlecht ist der Greedy Algorithmus?



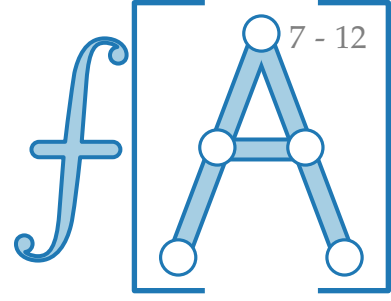
Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



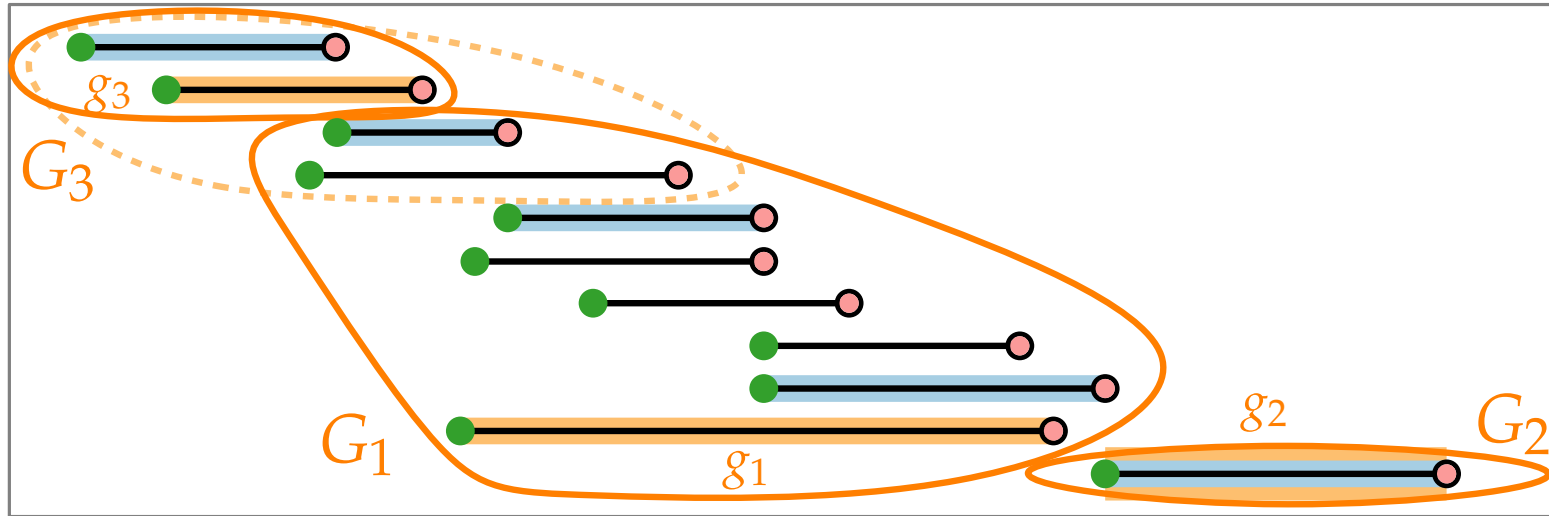
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

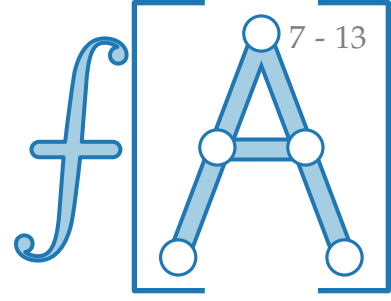
Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



und $L_i = L \cap G_i$.

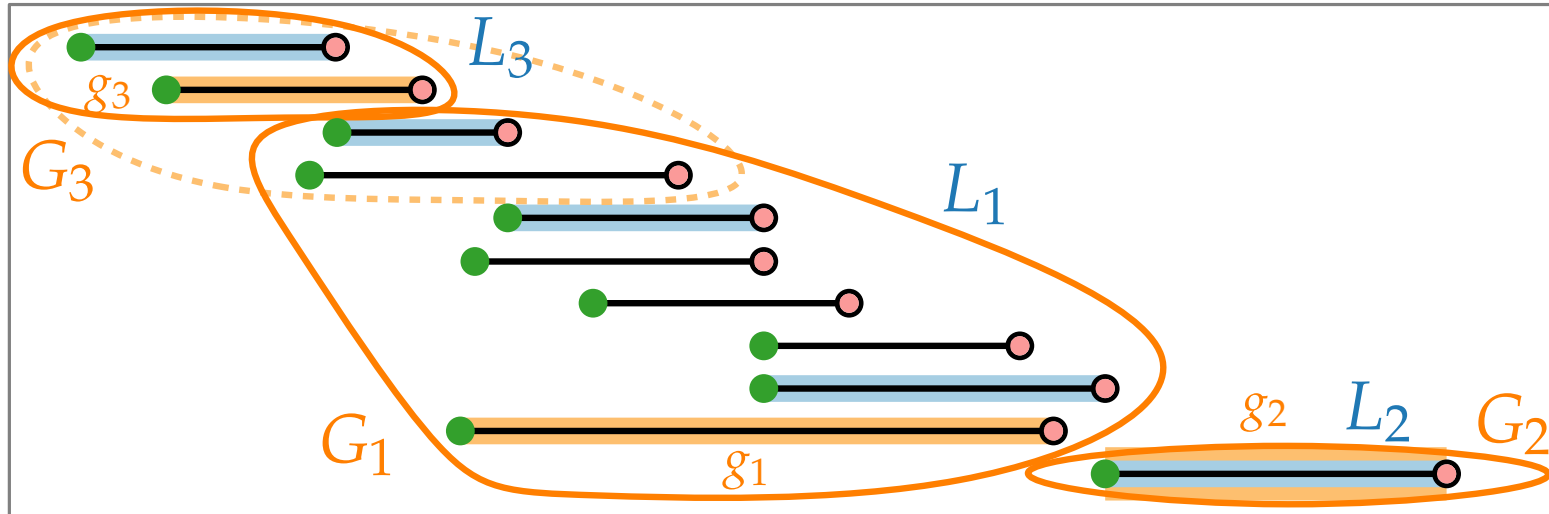
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

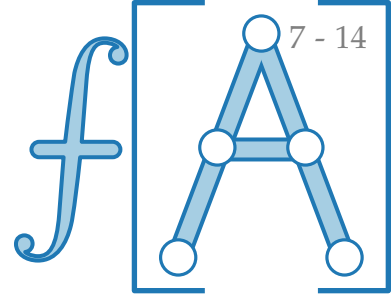
Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



und $L_i = L \cap G_i$.

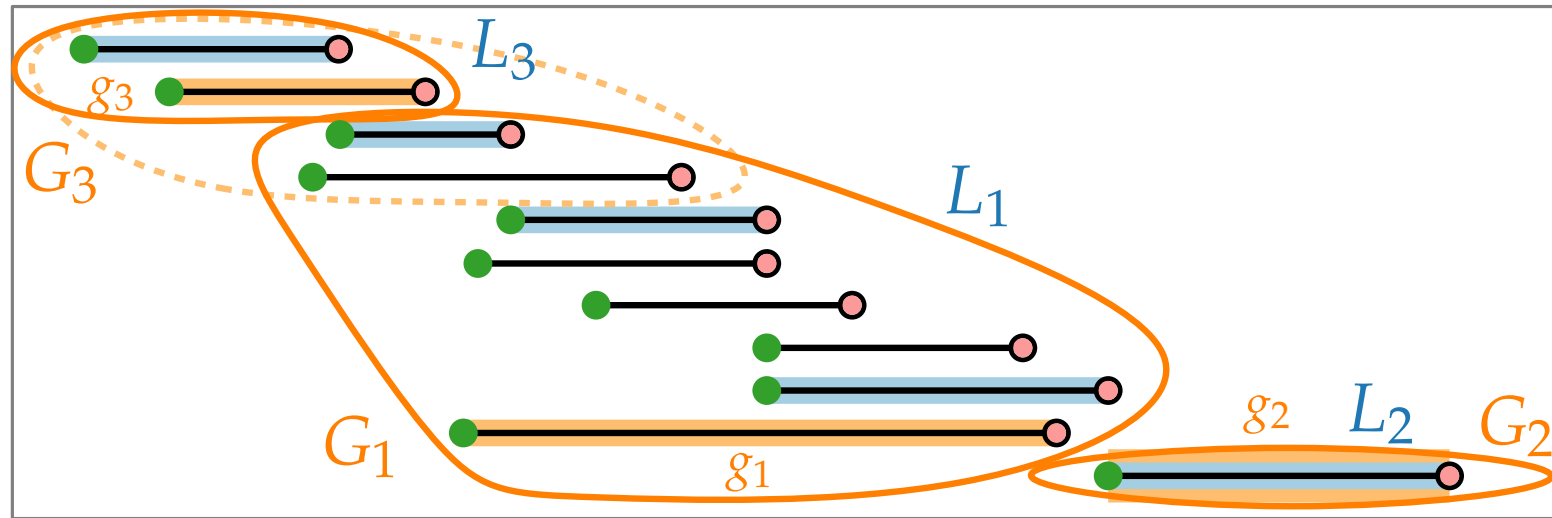
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

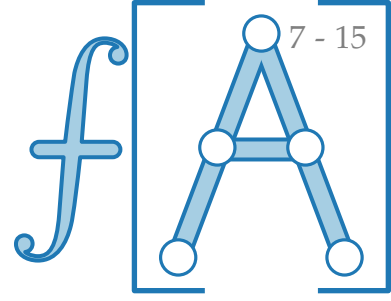
Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$

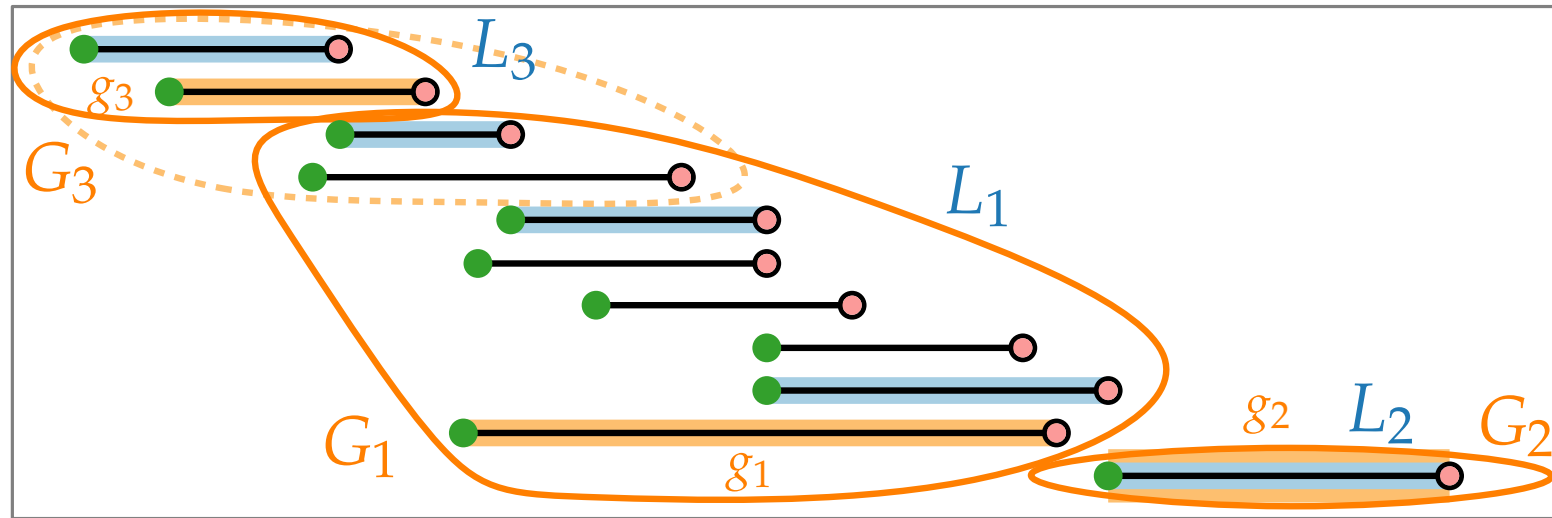
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$

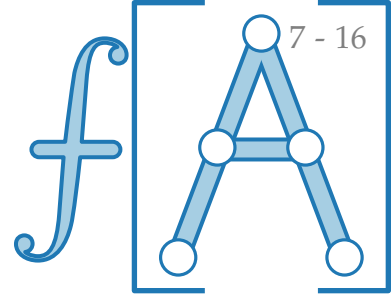


und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$

„ $\subseteq$ “:

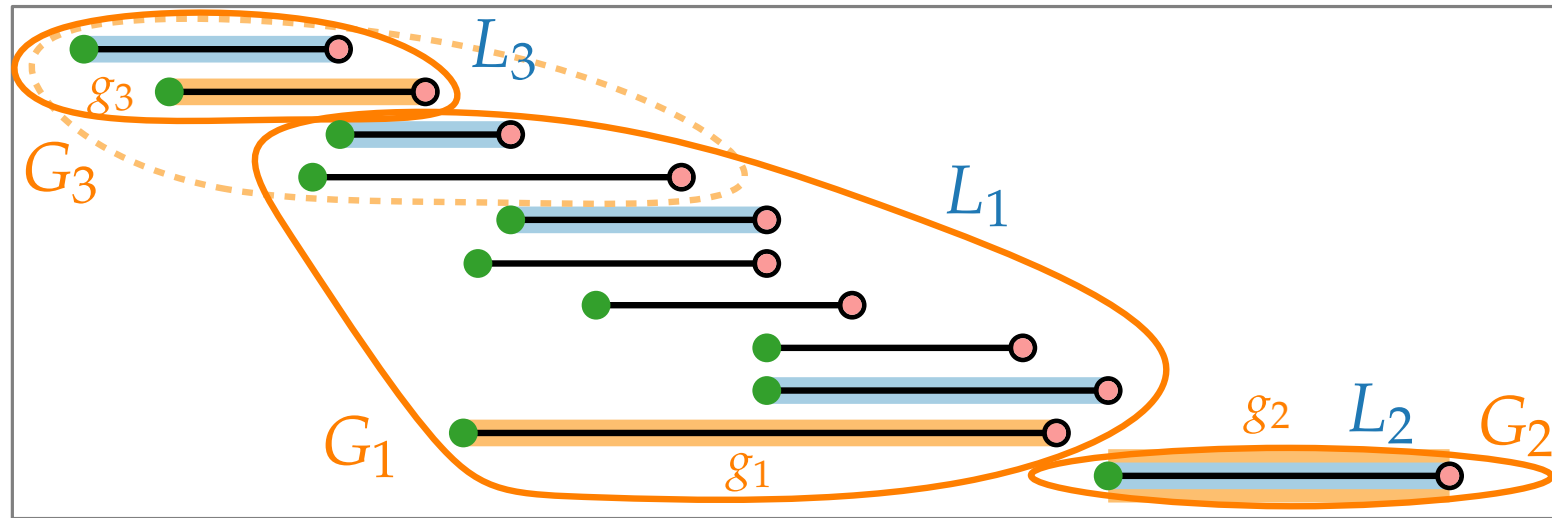
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$

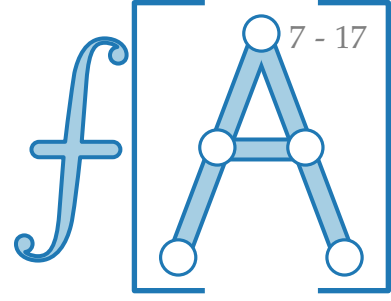


und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$

„ $\subseteq$ “: GA wählt so lange Intervalle aus, bis es keine mehr gibt.

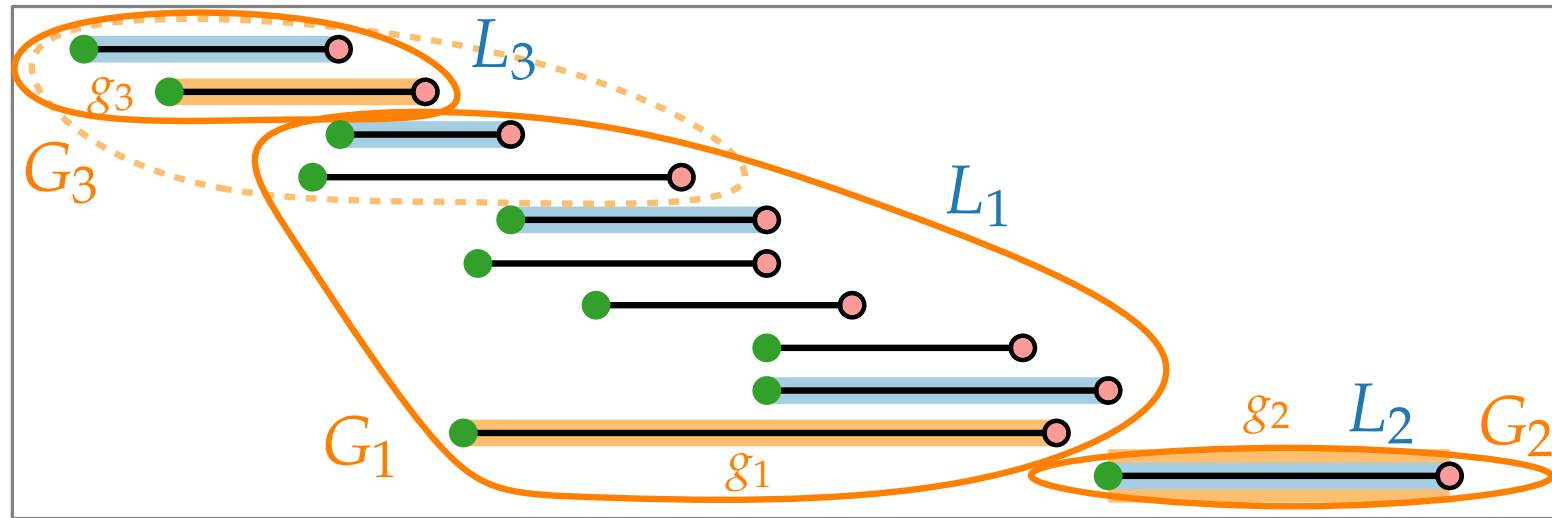
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



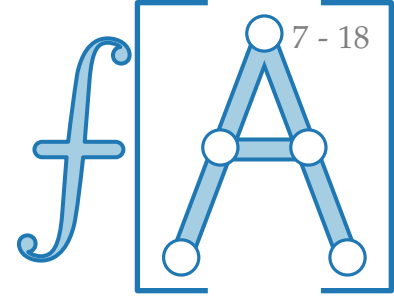
und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$

„ $\subseteq$ “: GA wählt so lange Intervalle aus, bis es keine mehr gibt.

„ $\supseteq$ “:

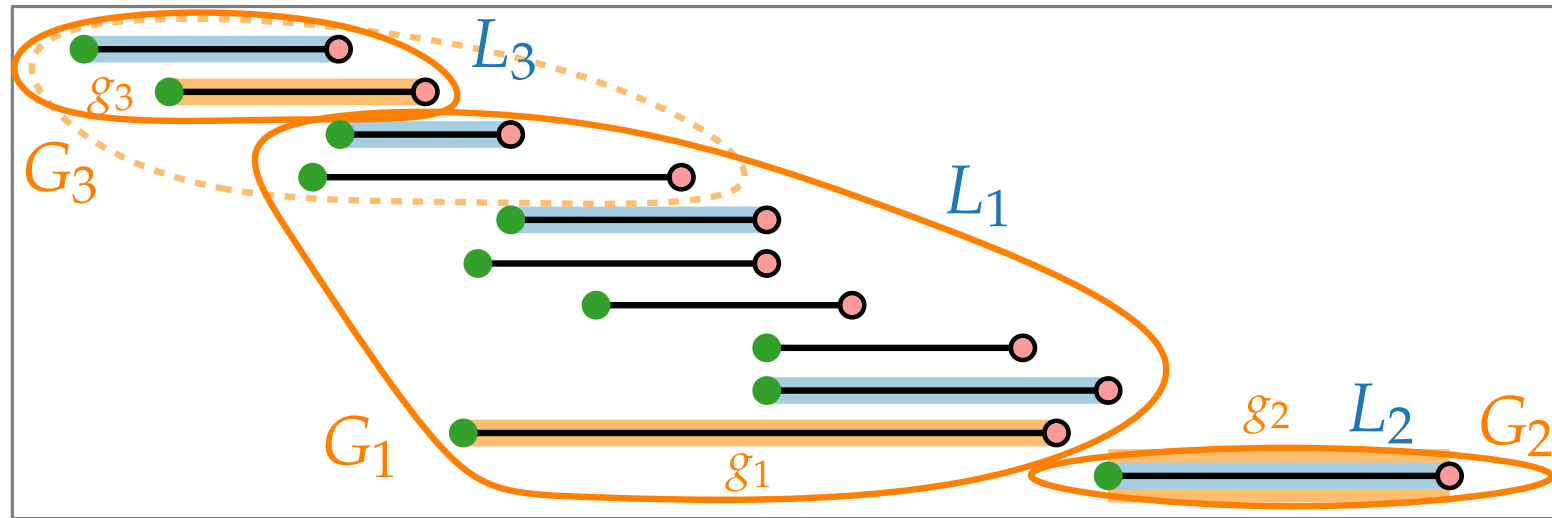
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



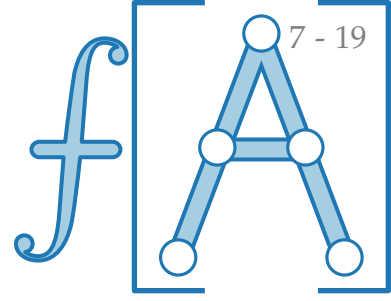
und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$

„ $\subseteq$ “: GA wählt so lange Intervalle aus, bis es keine mehr gibt.

„ $\supseteq$ “: klar, da $G_1 \subseteq A$, $G_2 \subseteq A$, $\dots$, $G_k \subseteq A$

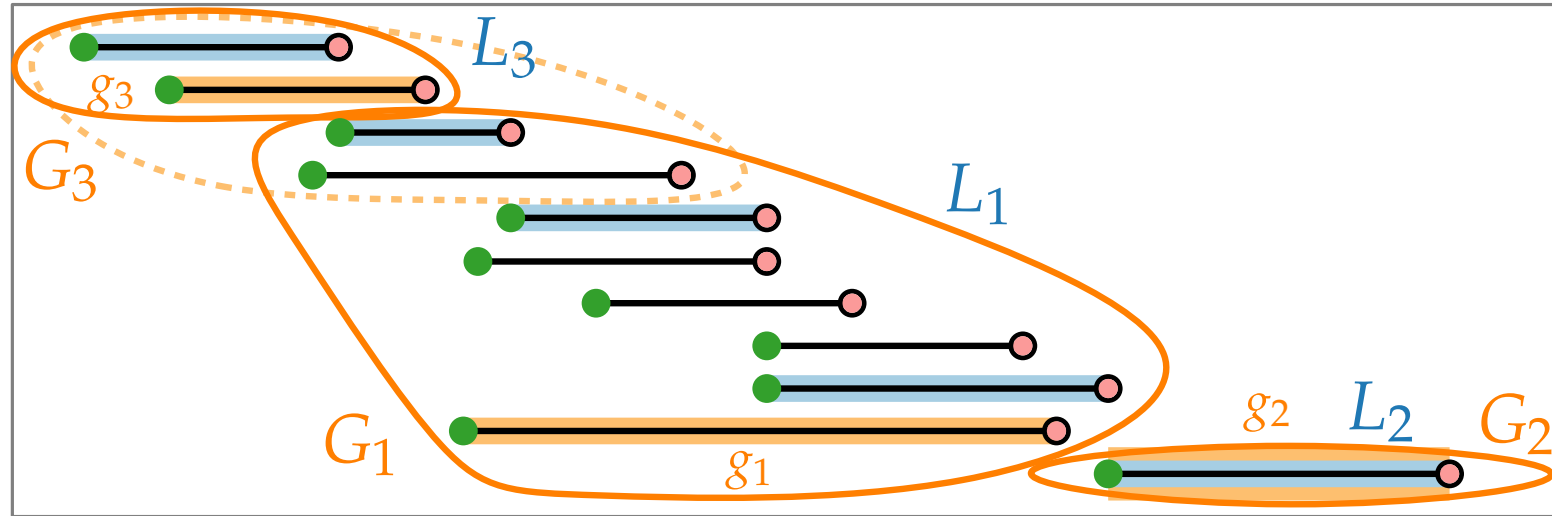
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



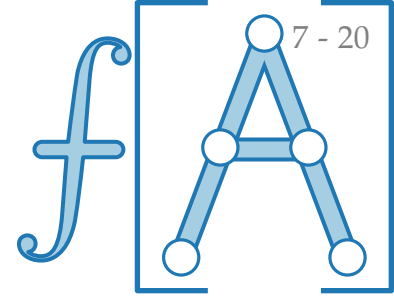
und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$ und $L = L_1 \dot{\cup} L_2 \dot{\cup} \dots \dot{\cup} L_k$.

„ $\subseteq$ “: GA wählt so lange Intervalle aus, bis es keine mehr gibt.

„ $\supseteq$ “: klar, da $G_1 \subseteq A$, $G_2 \subseteq A$, $\dots$, $G_k \subseteq A$

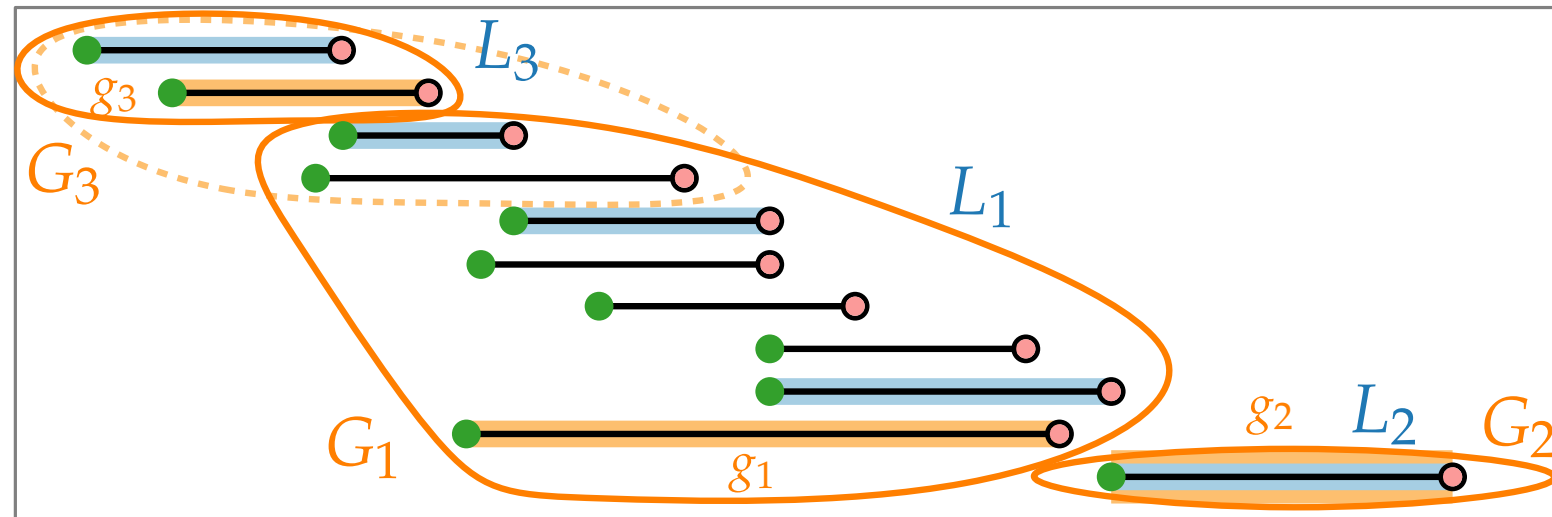
Wie gut/schlecht ist der Greedy Algorithmus?



Betrachte eine optimale Lösung $L \subseteq A$.

Sei $G = \{g_1, \dots, g_k\} \subseteq A$ die Greedy-Lösung (in dieser Reihenfolge gewählt)

Für $i = 1, \dots, k$ sei $G_i = \{a \in A \mid a \cap g_i \neq \emptyset\} \setminus (G_1 \cup \dots \cup G_{i-1})$



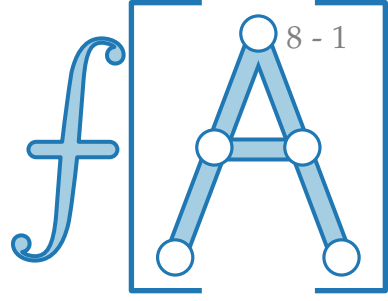
und $L_i = L \cap G_i$.

Dann gilt $A = G_1 \dot{\cup} G_2 \dot{\cup} \dots \dot{\cup} G_k$ und $L = L_1 \dot{\cup} L_2 \dot{\cup} \dots \dot{\cup} L_k$.

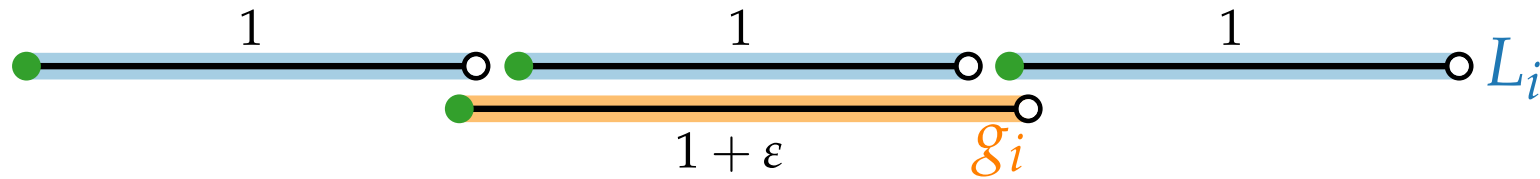
„ $\subseteq$ “: GA wählt so lange Intervalle aus, bis es keine mehr gibt.

„ $\supseteq$ “: klar, da $G_1 \subseteq A$, $G_2 \subseteq A$, $\dots$, $G_k \subseteq A$

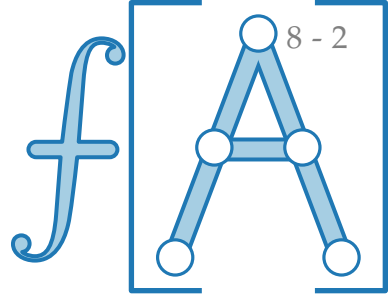
Wie gut/schlecht ist der 2. GA?



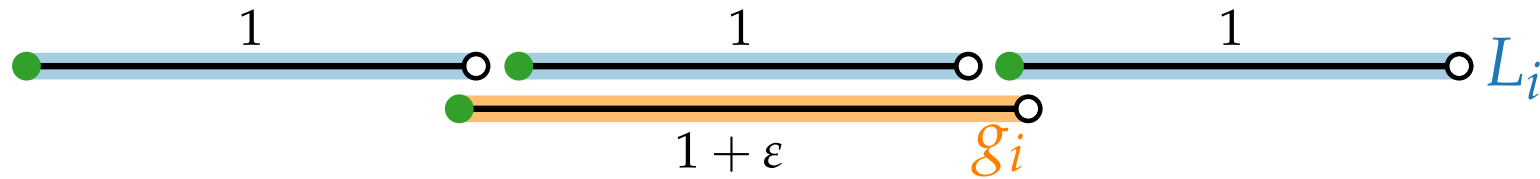
Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



Wie gut/schlecht ist der 2. GA?

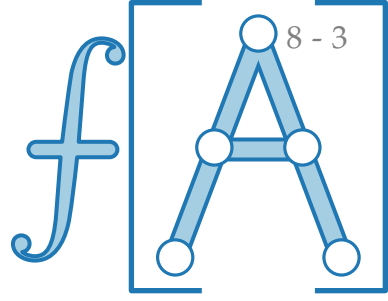


Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.

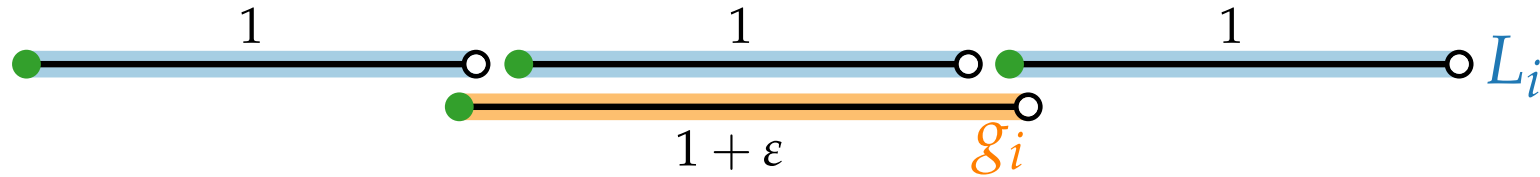


Beweis.

Wie gut/schlecht ist der 2. GA?

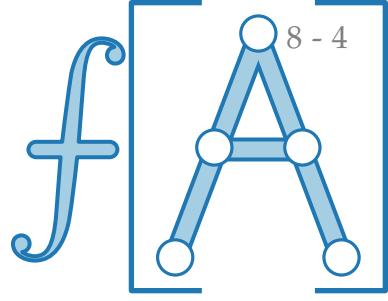


Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.

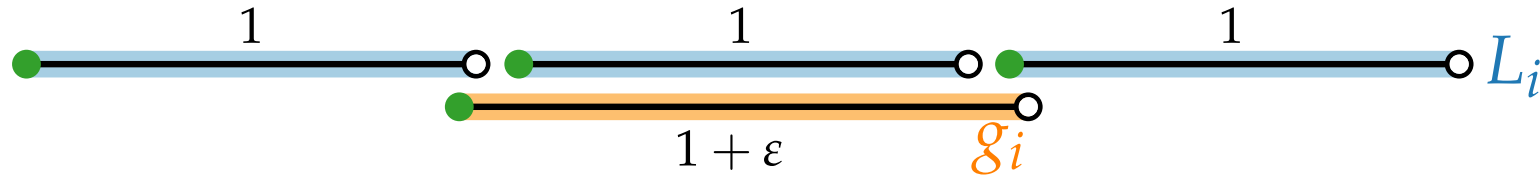


Beweis. (a) g_i ist nach Wahl ein längstes Intervall in G_i

Wie gut/schlecht ist der 2. GA?



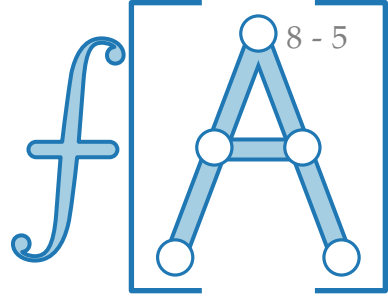
Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



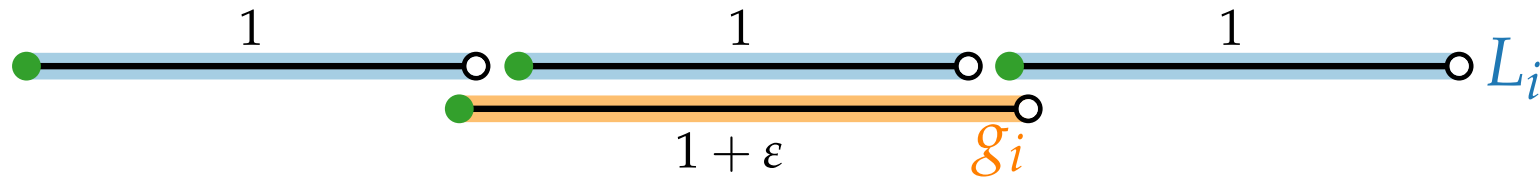
Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i

Wie gut/schlecht ist der 2. GA?



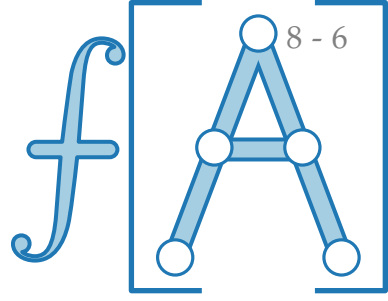
Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



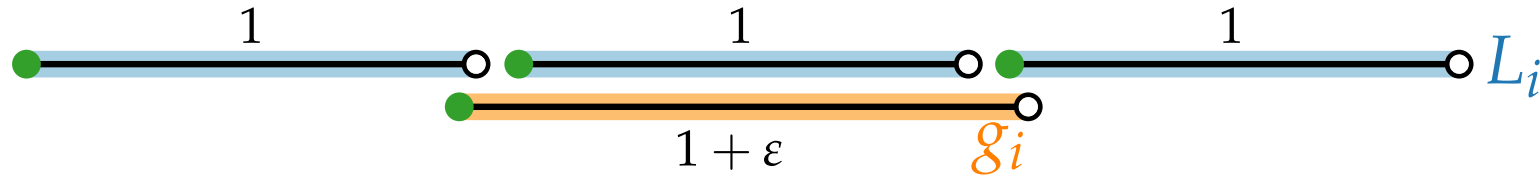
Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

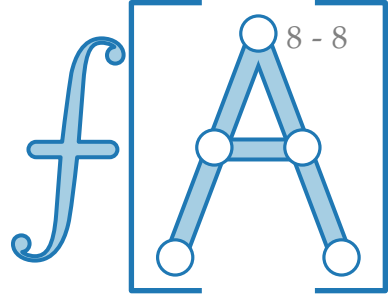


The diagram shows a horizontal line representing a path. It is divided into several segments, each labeled with the number '1'. A specific segment is highlighted in orange and labeled $1 + \epsilon$ and g_i . The path starts at a green dot on the left and ends at a white circle on the right, which is labeled L_i . There are also green dots and white circles at the boundaries of the segments.

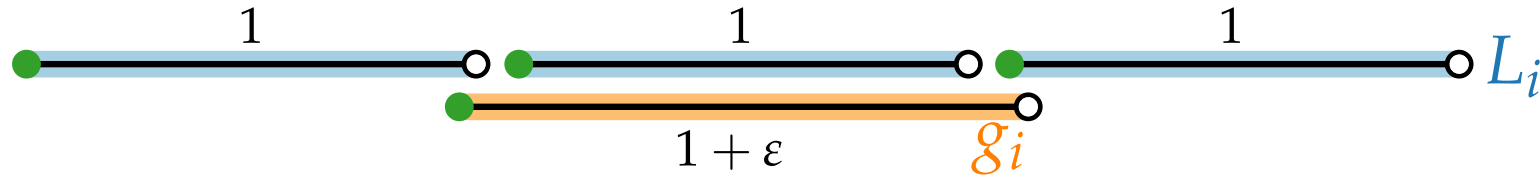
- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

$$\sum_{i=1}^k \ell(L_i)$$

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



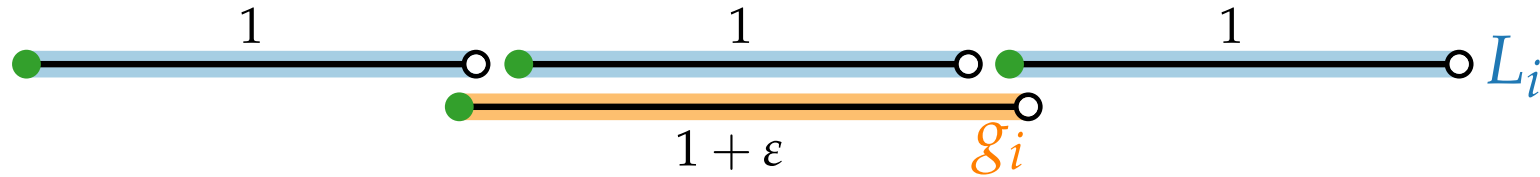
Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt



$$\sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i)$$

Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.

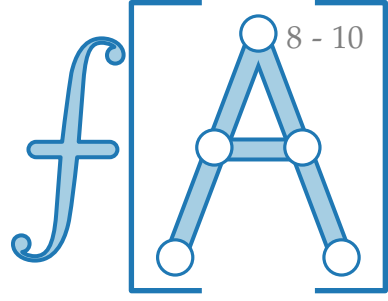


Beweis.

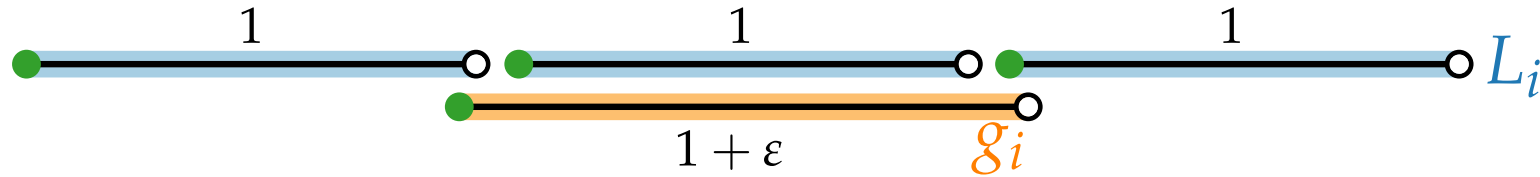
- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

$$\sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i) = 3\ell(G)$$

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



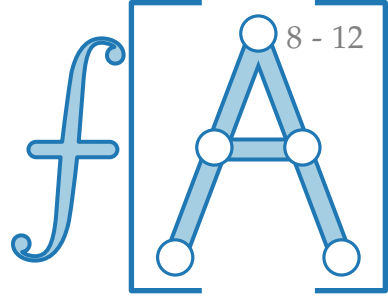
Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

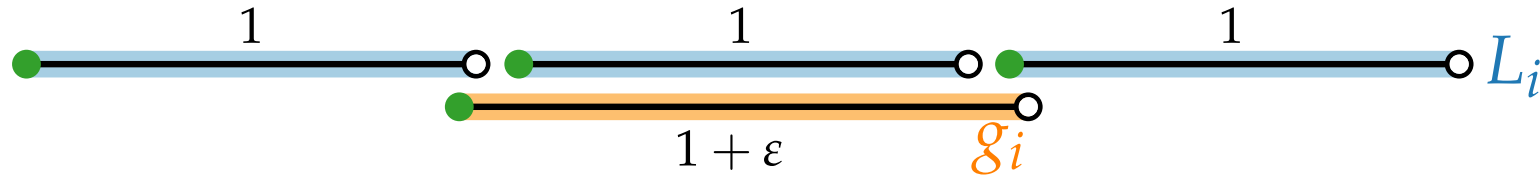


$$\ell(L) = \sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i) = 3\ell(G)$$

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



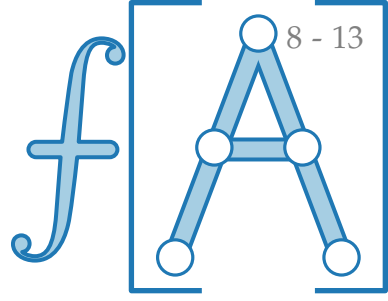
Beweis.

- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

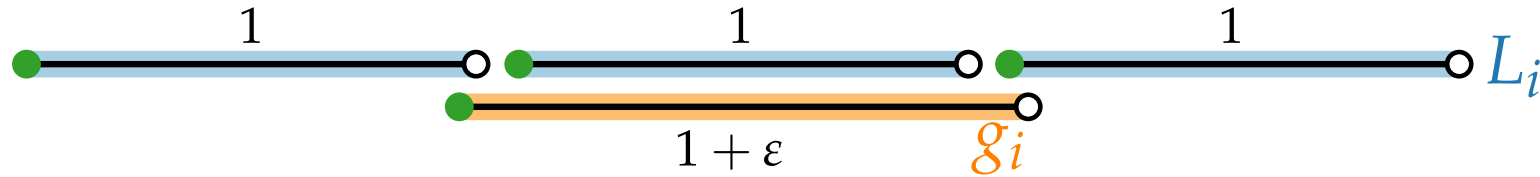
$$\rightarrow \text{OPT} = \ell(L) = \sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i) = 3\ell(G)$$

$$\rightarrow \ell(G) > \text{OPT}/3$$

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



Beweis.

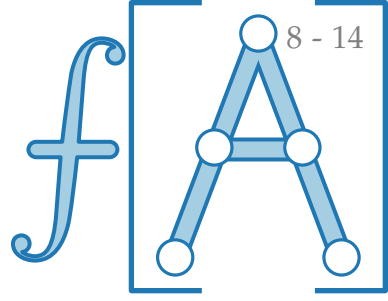
- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

$$\rightarrow \text{OPT} = \ell(L) = \sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i) = 3\ell(G)$$

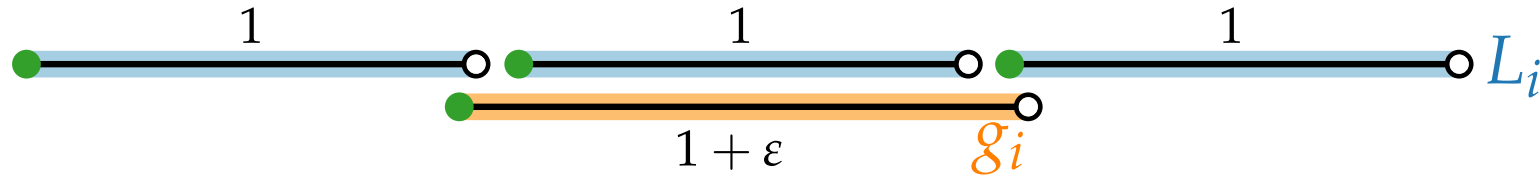
$$\rightarrow \ell(G) > \text{OPT}/3$$

→ 2. GA liefert **immer** mindestens $1/3$ der maximalen Gesamtlänge.

Wie gut/schlecht ist der 2. GA?



Behauptung. Für $i = 1, \dots, k$ gilt $\ell(L_i) < 3\ell(g_i)$.



Beweis.

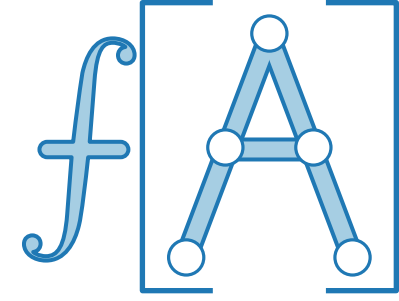
- (a) g_i ist nach Wahl ein längstes Intervall in G_i
- (b) jedes $a \in L_i$ schneidet g_i
- (c) Intervalle in L_i sind paarweise disjunkt

$$\rightarrow \text{OPT} = \ell(L) = \sum_{i=1}^k \ell(L_i) < 3 \sum_{i=1}^k \ell(g_i) = 3\ell(G)$$

$$\rightarrow \ell(G) > \text{OPT}/3$$

$\rightarrow$ 2. GA liefert **immer** mindestens $1/3$ der maximalen Gesamtlänge.

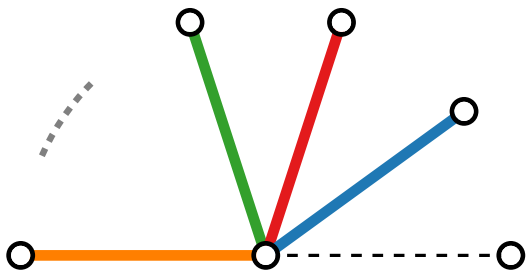
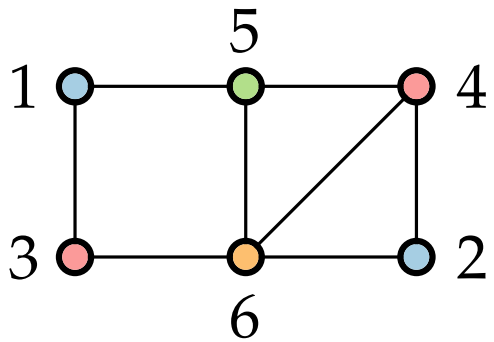
Also ist der 2. GA ein **Faktor-(1/3)-Approximationsalgorithmus**.



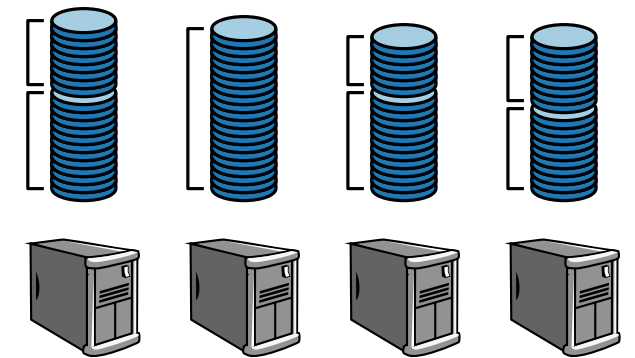
Fortgeschrittene Algorithmen

21. Vorlesung

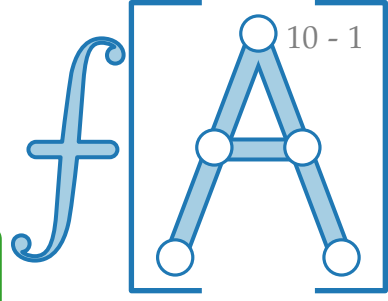
Approximationsalgorithmen



Teil III: Schwere von TSP



Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

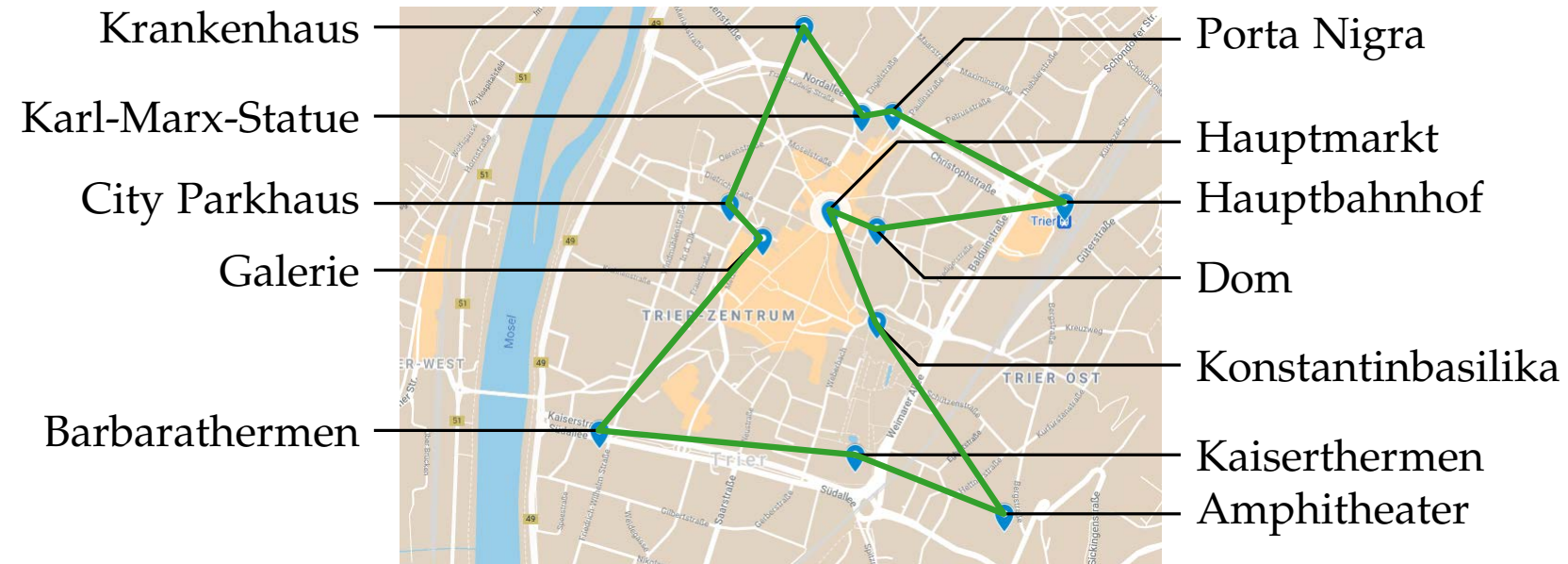
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972

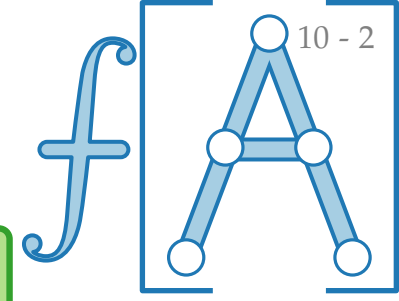


TSP ist NP-schwer.

Es ist sogar NP-schwer,
eine Lösung zu finden, die
„nur“ 1.000.000 mal so lang
ist wie die kürzeste.



Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

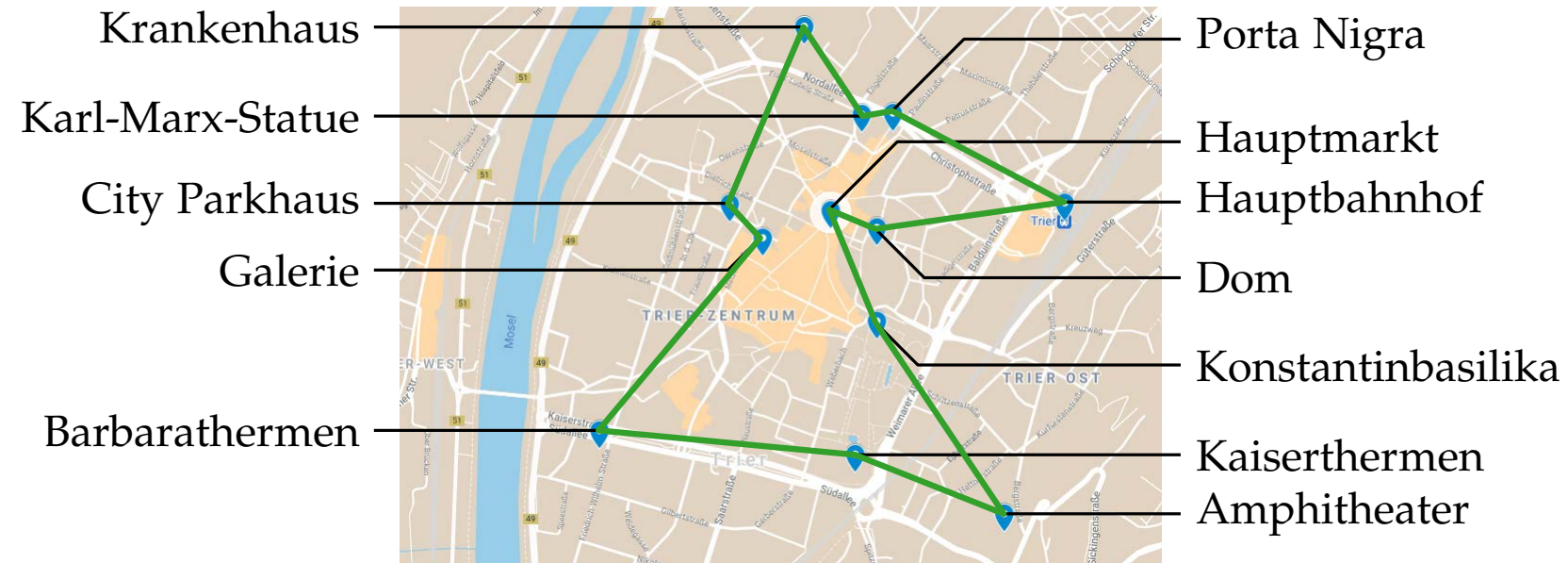
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972



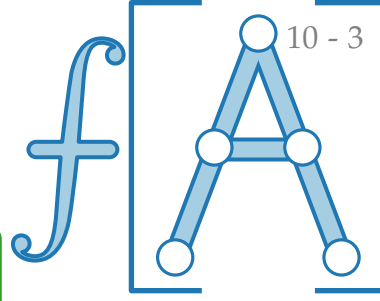
TSP ist NP-schwer.

Es ist sogar NP-schwer,
eine Lösung zu finden, die
„nur“ 1.000.000 mal so lang
ist wie die kürzeste.



BELLMANHELDKARP:
 $\mathcal{O}^*(2^n)$ Laufzeit,
 $\mathcal{O}(n)$ Speicher

Problem des Handlungsreisenden (TSP)



Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

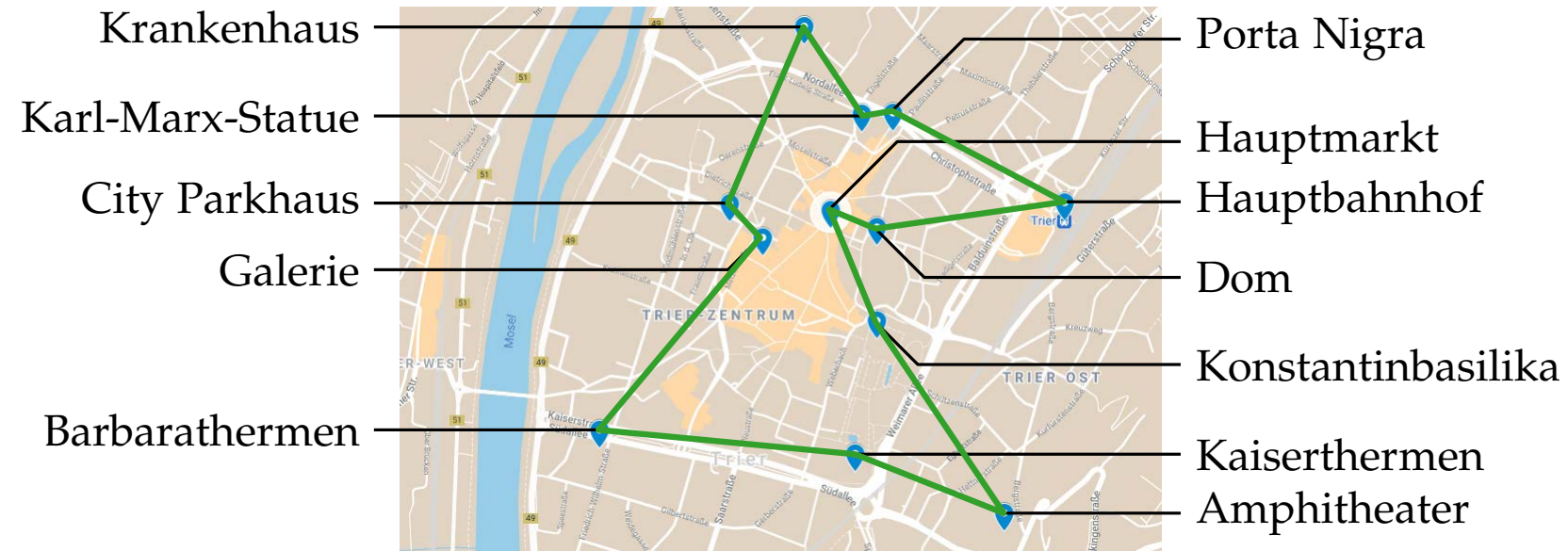
Ges. Tour des Handlungsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Richard M. Karp, 1972



TSP ist NP-schwer.

Es ist sogar NP-schwer, eine Lösung zu finden, die „nur“ 1.000.000 mal so lang ist wie die kürzeste.

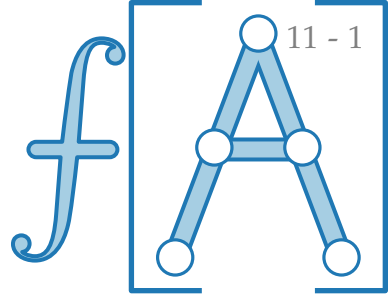


BELLMANHELDKARP:
 $\mathcal{O}^*(2^n)$ Laufzeit,
 $\mathcal{O}(n)$ Speicher

[Nederlof, STOC'20]: Bipartites TSP randomisiert in
 $\mathcal{O}(1.9999^n)$ Zeit mit konstanter Fehlerwahrscheinlichkeit,
falls $n \times n$ -Matrixmultiplikation in $n^{2+o(1)}$ lösbar

NP-Schwere

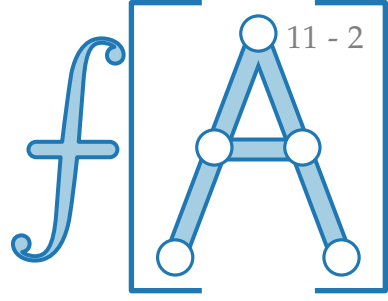
Satz. TSP ist NP-schwer.



NP-Schwere

Satz. TSP ist NP-schwer.

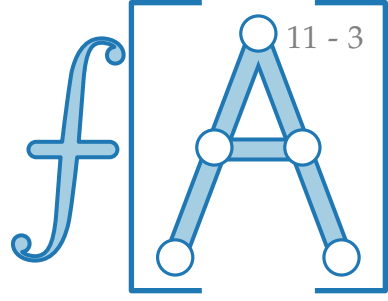
Beweis.



NP-Schwere

Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

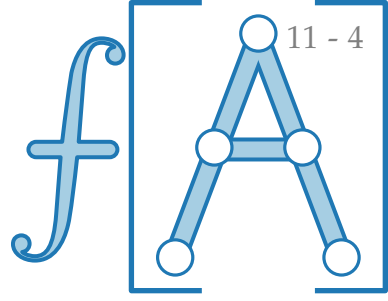


NP-Schwere

Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP effizient lösen könnten,
dann auch Hamiltonkreis.

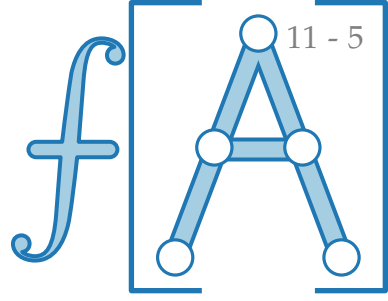


NP-Schwere

Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP effizient lösen könnten,
dann auch Hamiltonkreis.

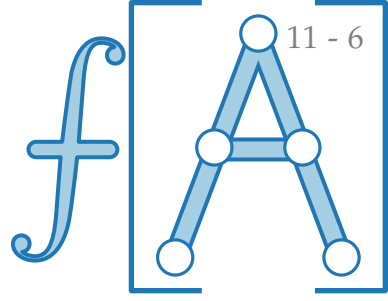


NP-Schwere

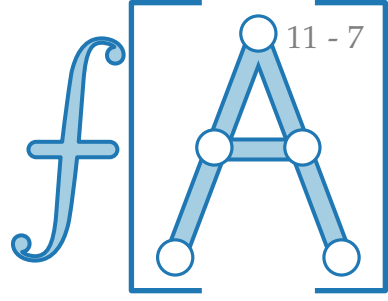
Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP effizient lösen könnten,
dann auch Hamiltonkreis. in Polynomialzeit!



NP-Schwere



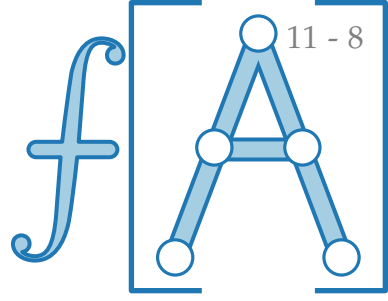
Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP effizient lösen könnten,
dann auch Hamiltonkreis. in Polynomialzeit!

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

NP-Schwere



Satz. TSP ist NP-schwer.

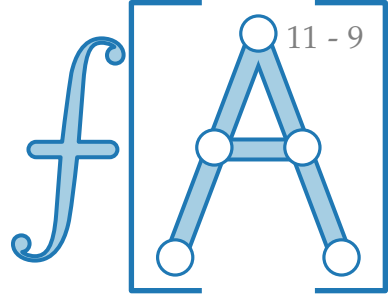
Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:

NP-Schwere



Satz. TSP ist NP-schwer.

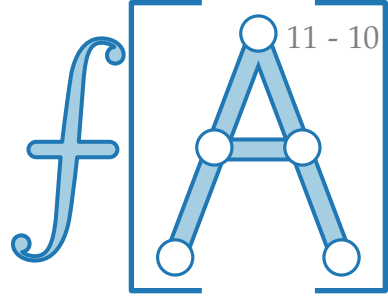
Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

NP-Schwere



Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

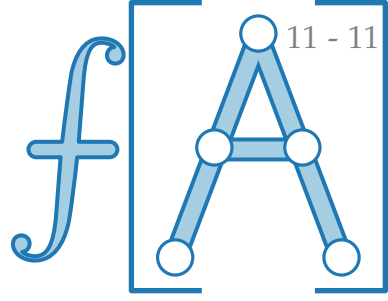
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} ? & \text{falls } (u, v) \in E, \\ ? & \text{sonst.} \end{cases}$$

NP-Schwere



Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

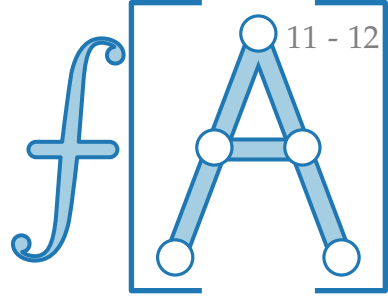
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

NP-Schwere



Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

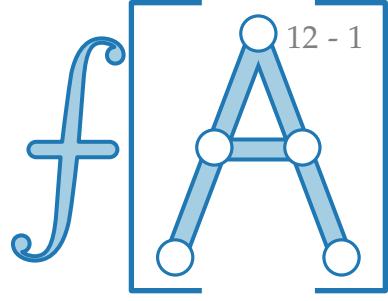
Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.

NPO-Schwere



Satz. TSP ist NP-schwer.

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

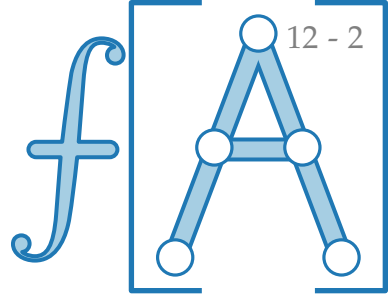
Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.

NPO-Schwere



Satz. TSP ist NP-schwer **zu approximieren!**

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. **in Polynomialzeit!**

Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.

NPO-Schwere

D.h. für kein $\gamma \geq 1$ gibt's eine γ -Approximation für TSP (außer P=NP).

Satz. TSP ist NP-schwer **zu approximieren!**

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP **effizient** lösen könnten,
dann auch Hamiltonkreis. *in Polynomialzeit!*

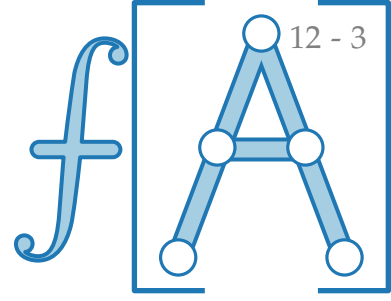
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.



NPO-Schwere

D.h. für kein $\gamma \geq 1$ gibt's eine γ -Approximation für TSP (außer P=NP).

Satz. TSP ist NP-schwer **zu approximieren!**

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP ~~effizient lösen~~ ^{mit Güte γ approximieren} könnten,
dann auch Hamiltonkreis. in Polynomialzeit!

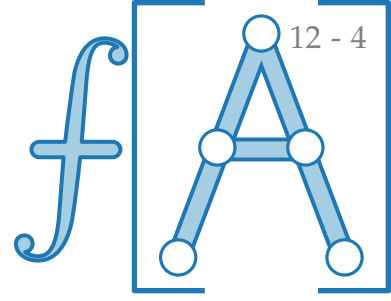
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.



NPO-Schwere

D.h. für kein $\gamma \geq 1$ gibt's eine γ -Approximation für TSP (außer $P=NP$).

Satz. TSP ist NP-schwer **zu approximieren!**

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP ~~effizient lösen~~ ^{mit Güte γ approximieren} könnten,
dann auch Hamiltonkreis. in Polynomialzeit!

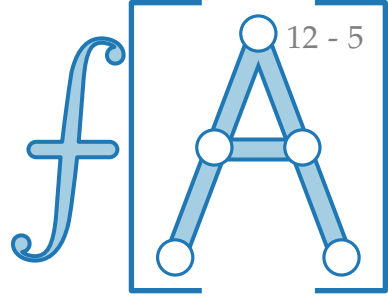
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| \cdot \gamma & \text{sonst.} \end{cases}$$

→ optimale TSP-Tour kostet $|V| \Leftrightarrow H$ ham.



NPO-Schwere

D.h. für kein $\gamma \geq 1$ gibt's eine γ -Approximation für TSP (außer P=NP).

Satz. TSP ist NP-schwer **zu approximieren!**

Beweis. Durch Reduktion vom Problem Hamiltonkreis.

Zu zeigen. wenn wir TSP ~~effizient lösen~~ ^{mit Güte γ approximieren} könnten,
dann auch Hamiltonkreis. in Polynomialzeit!

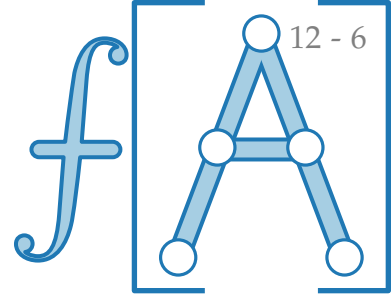
Geg. ungerichteter (i.A. *nicht* vollständiger)
Graph $H = (V, E)$.

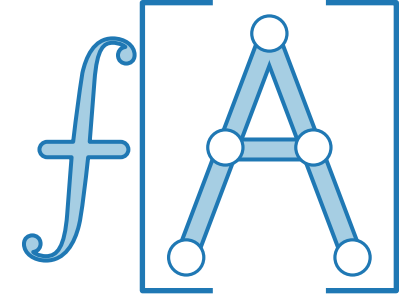
Definiere vollständigen Graphen G mit Kosten d , so dass:
 G hat **billige** TSP-Tour $\Leftrightarrow H$ hamiltonsch.

Nimm $G = \left(V, \binom{V}{2}\right)$ und für $u, v \in V$ setze

$$d(u, v) = \begin{cases} 1, & \text{falls } (u, v) \in E, \\ |V| \cdot \gamma & \text{sonst.} \end{cases}$$

$\Rightarrow$ ~~optimale~~ TSP-Tour ^{mit Güte γ} kostet $\leq \gamma \cdot |V| \Leftrightarrow H$ ham.

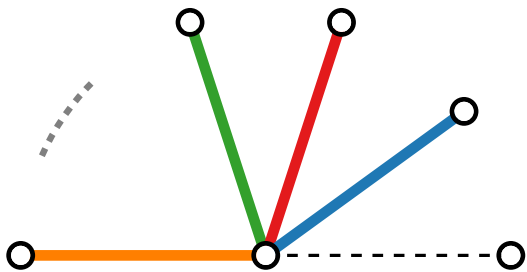
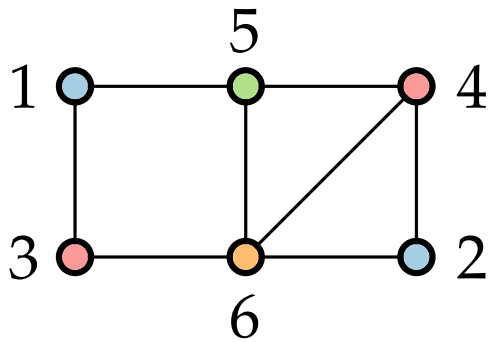




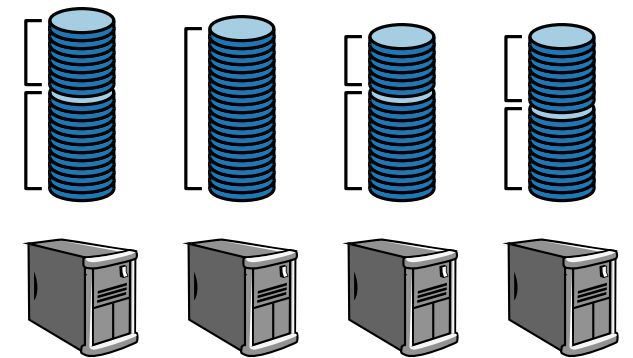
Fortgeschrittene Algorithmen

21. Vorlesung

Approximationsalgorithmen



Teil IV: Metrisches TSP

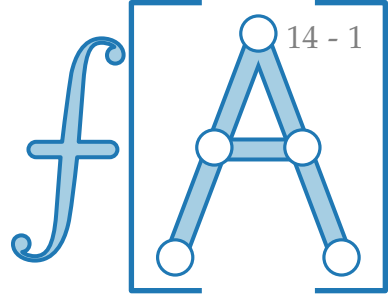


Was tun?

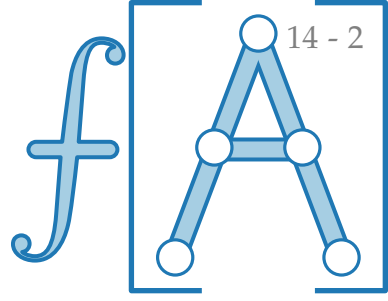
Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



Was tun? – Mach das Problem leichter!

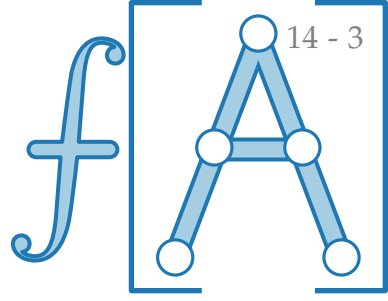


Problem. TRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$;
gerichteter, vollständiger Graph G mit Kantengewichten d

Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.

Was tun? – Mach das Problem leichter!

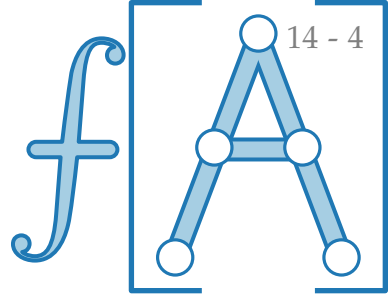


Problem. METRICTRAVELINGSALESMANPROBLEM

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$,
die die Dreiecksungleichung erfüllen.

Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte
genau einmal besucht und zum Startpunkt zurückkehrt.

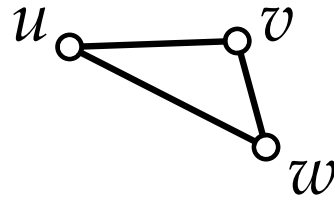
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM

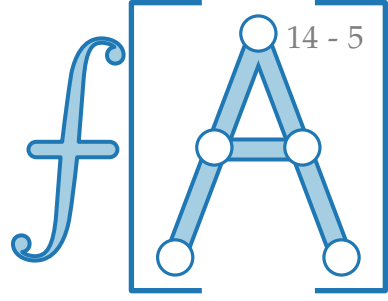
Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$,
die die Dreiecksungleichung erfüllen.

Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte
genau einmal besucht und zum Startpunkt zurückkehrt.



$$\text{d.h. } \forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w).$$

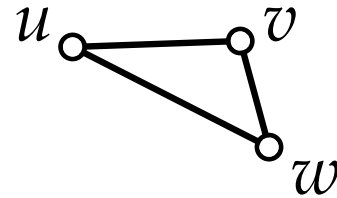
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

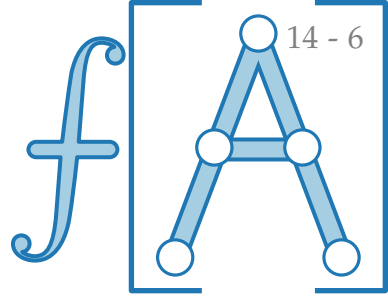
Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



$$\text{d.h. } \forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w).$$

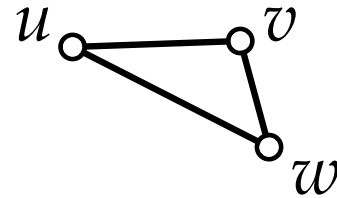
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

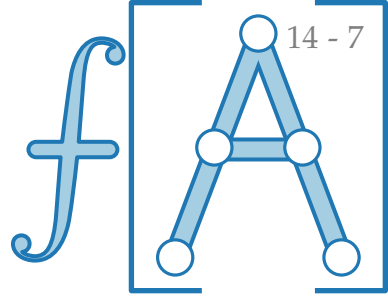
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

Satz. Es gibt eine 2-Approximation für Δ -TSP.

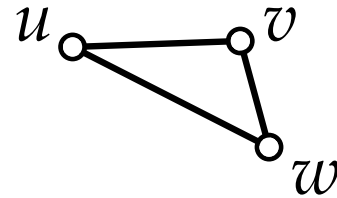
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

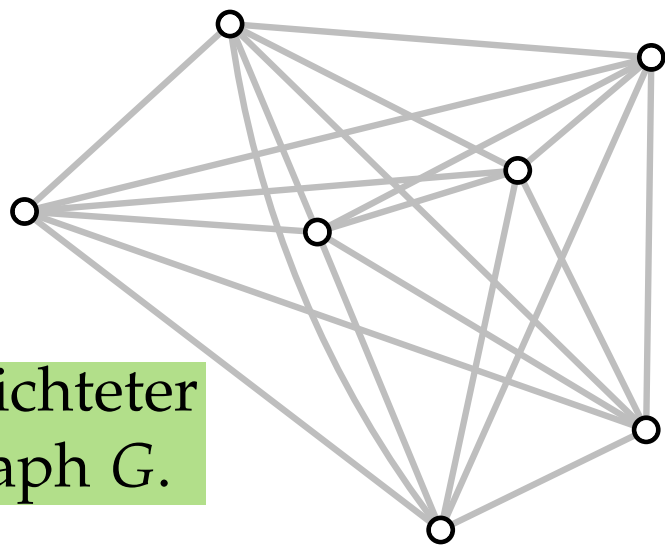
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



$$\text{d.h. } \forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w).$$

Satz. Es gibt eine 2-Approximation für Δ -TSP.

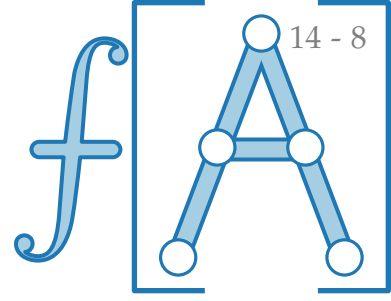
Beweis.



Geg. gewichteter
vollst. Graph G .



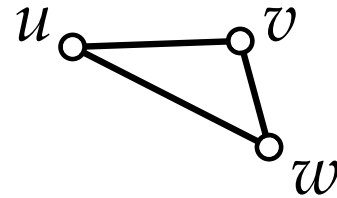
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

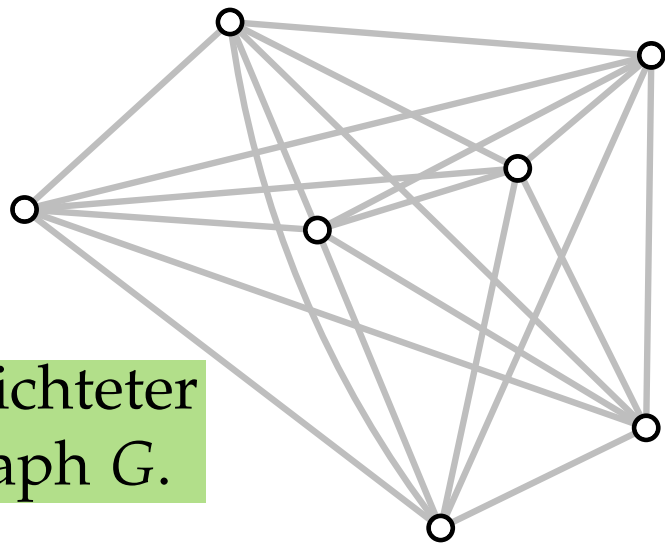
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



$$\text{d.h. } \forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w).$$

Satz. Es gibt eine 2-Approximation für Δ -TSP.

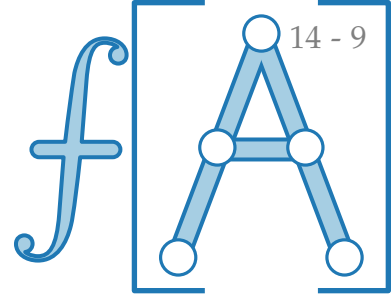
Beweis.



Geg. gewichteter
vollst. Graph G .

Berechne min. Spannbaum **MSB**.

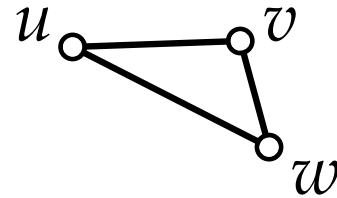
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

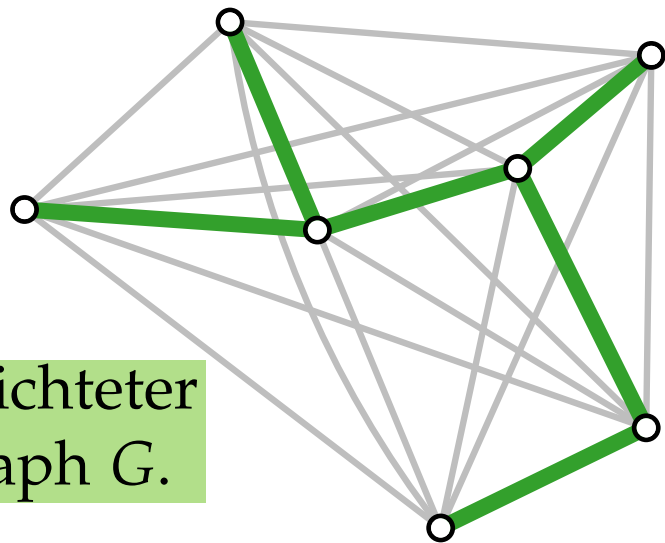
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

Satz. Es gibt eine 2-Approximation für Δ -TSP.

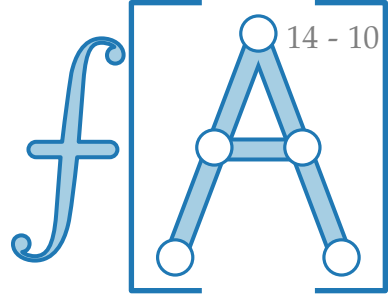
Beweis.



Geg. gewichteter
vollst. Graph G .

Berechne min. Spannbaum **MSB**.

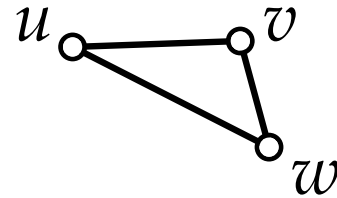
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

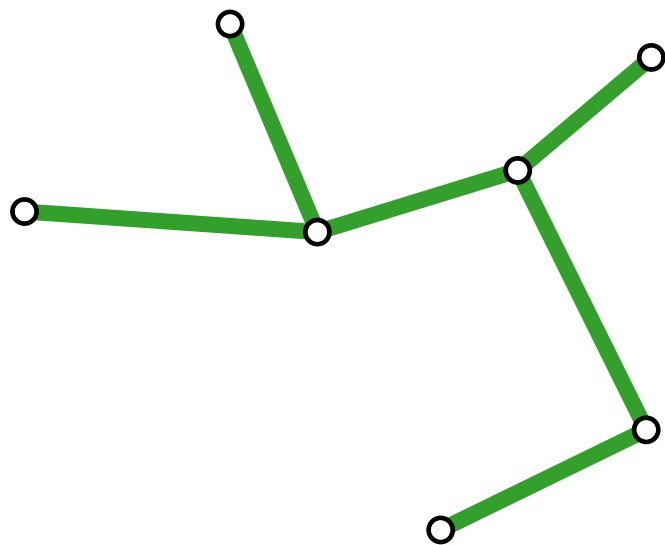
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

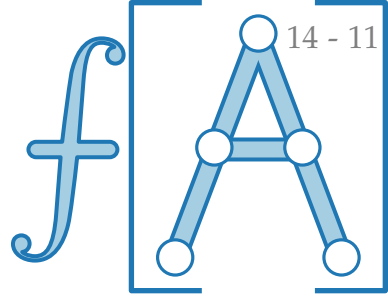
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.

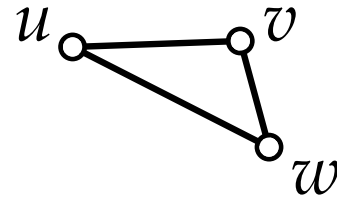
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

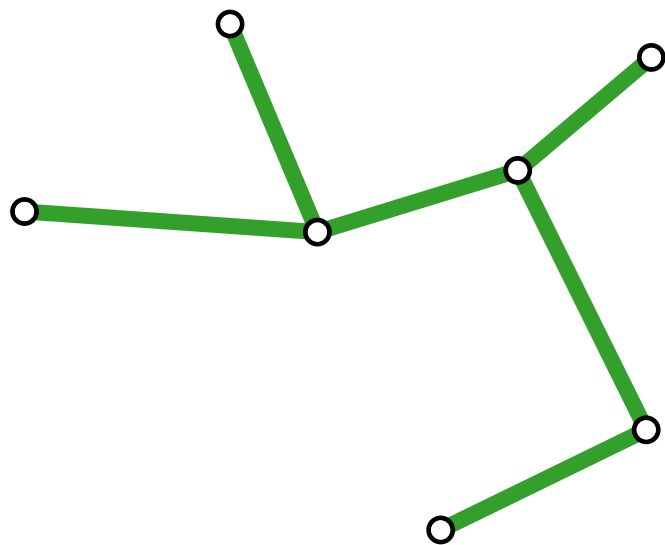
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

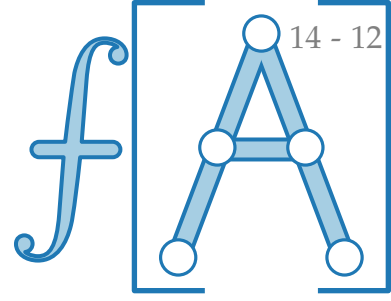
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!

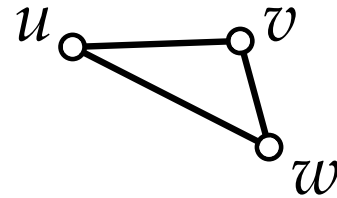
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

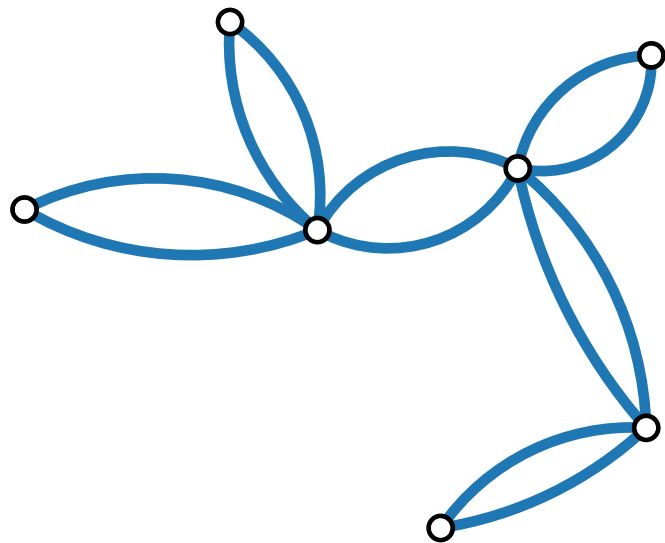
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

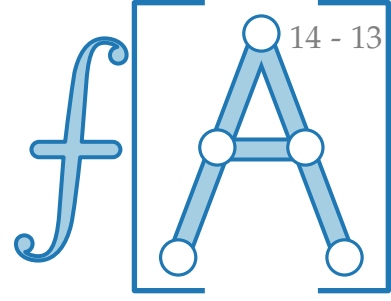
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!

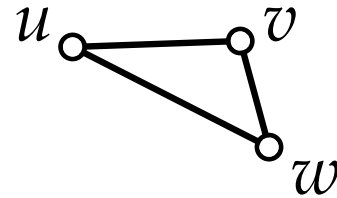
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

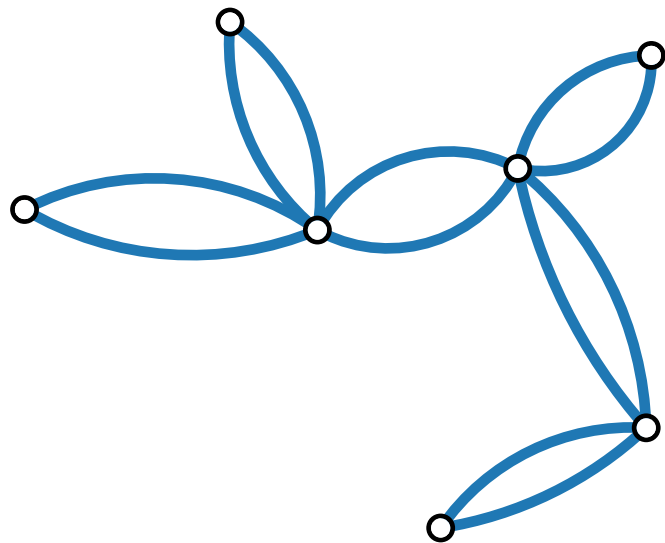
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

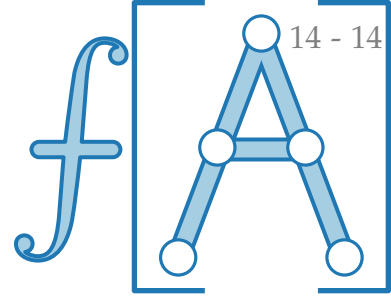
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

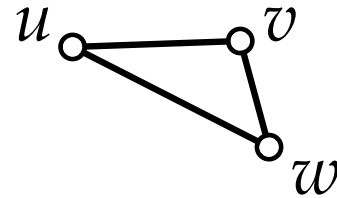
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

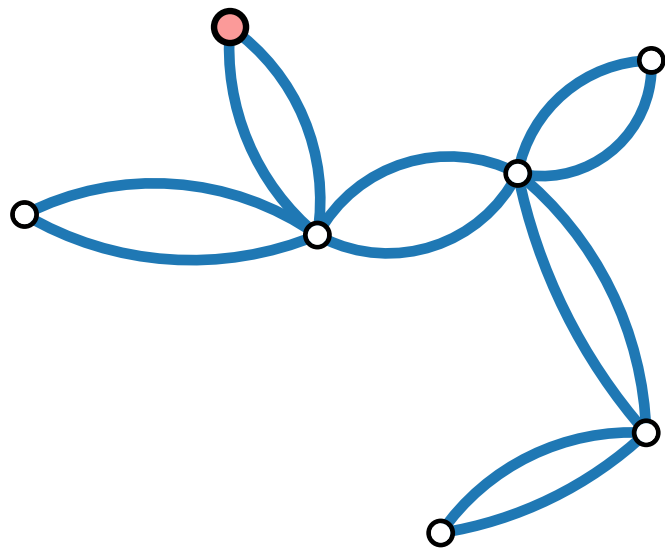
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

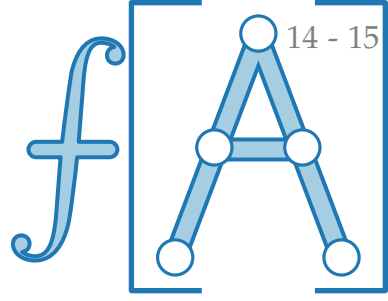
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

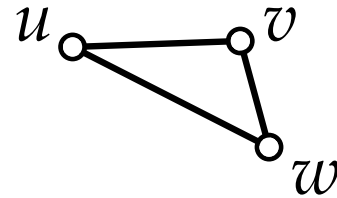
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

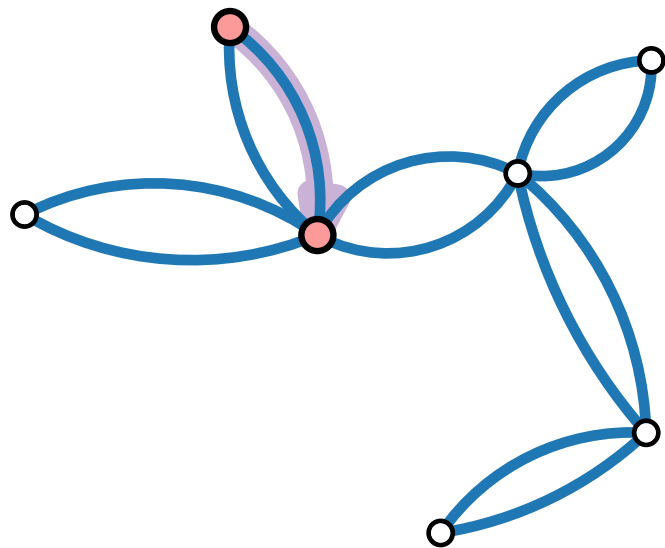
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

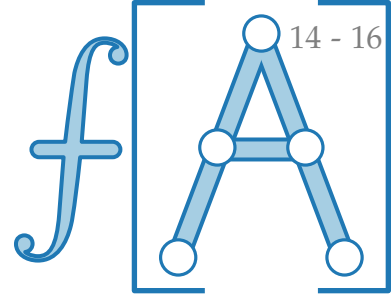
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

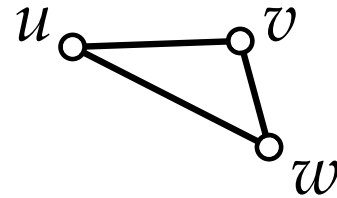
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

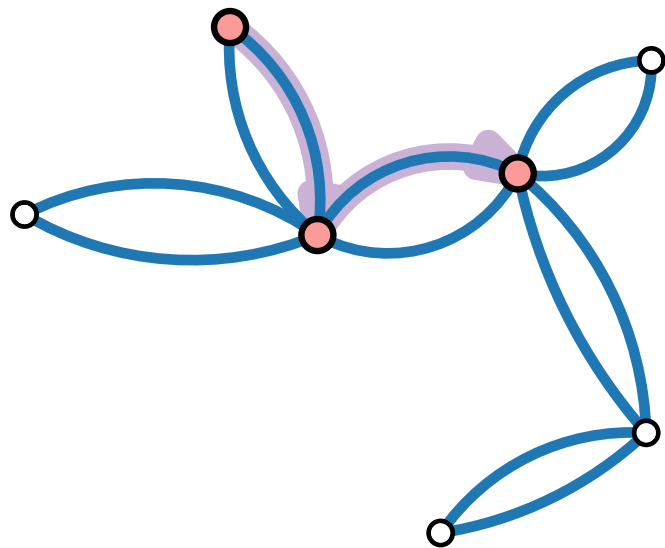
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



$$\text{d.h. } \forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w).$$

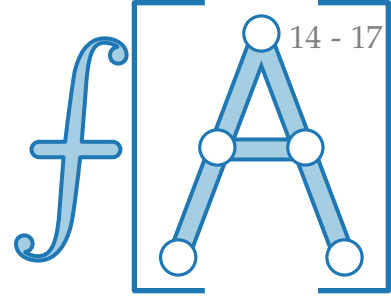
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

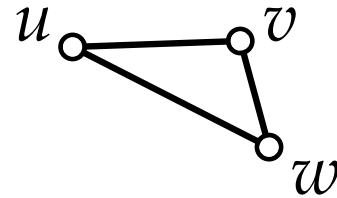
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

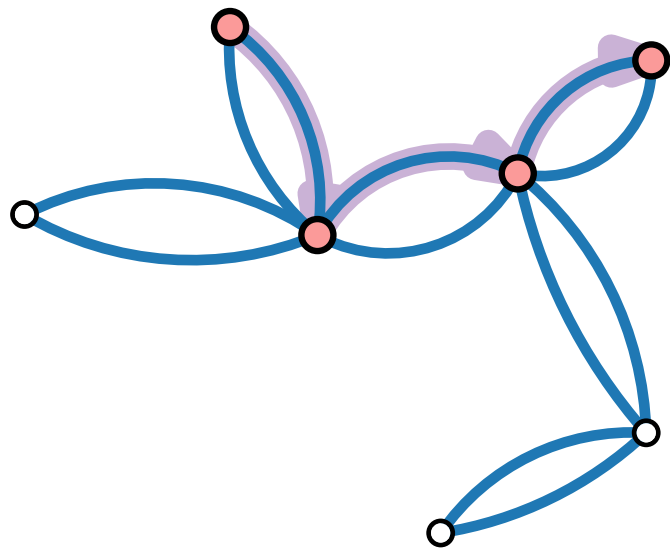
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

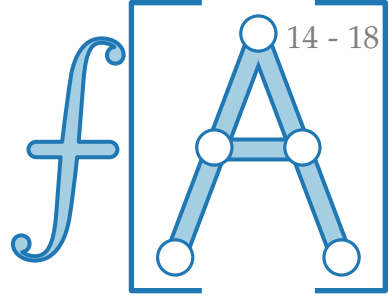
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

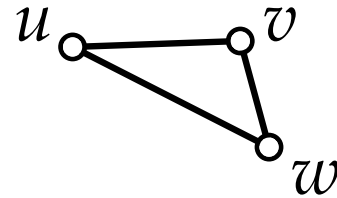
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

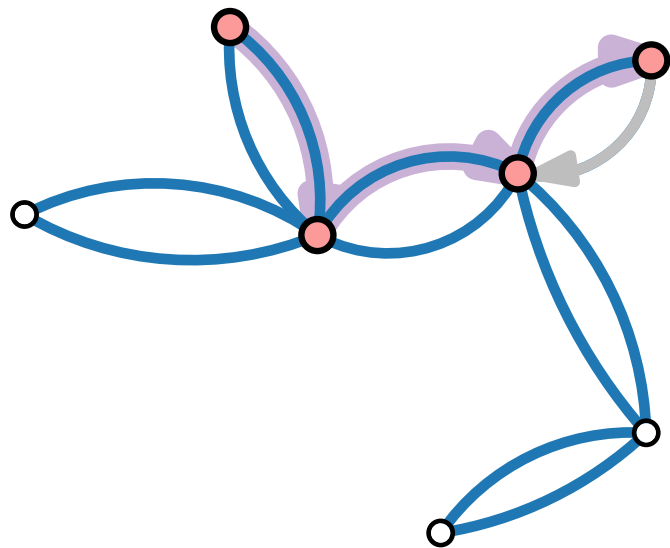
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

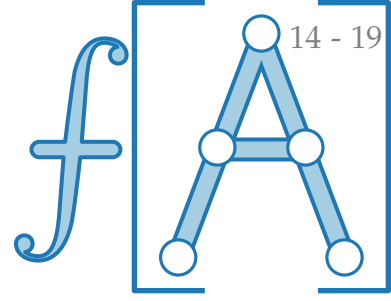
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.

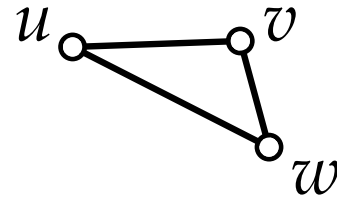
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

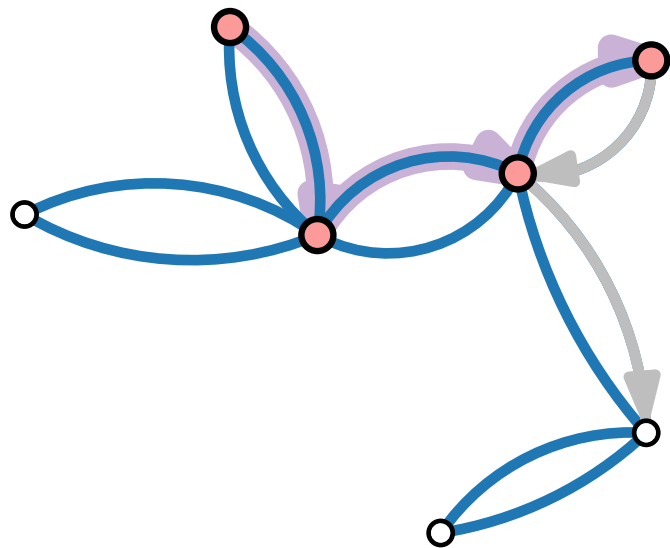
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

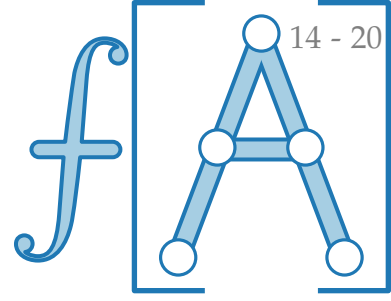
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.

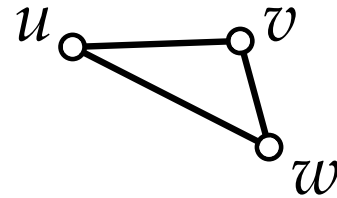
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

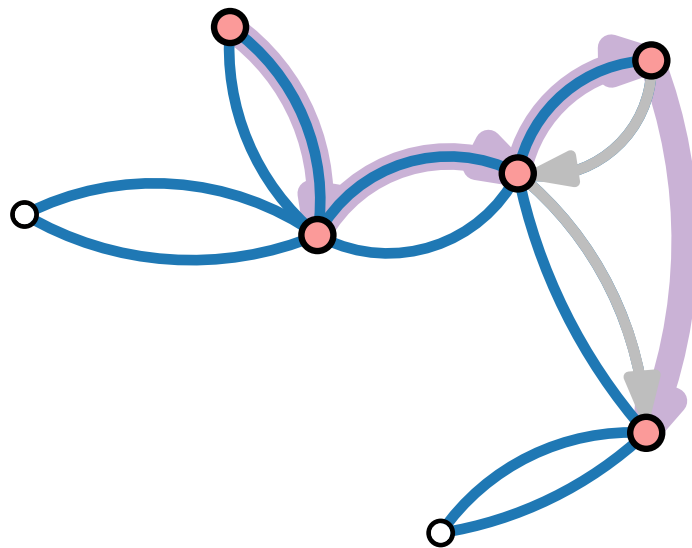
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

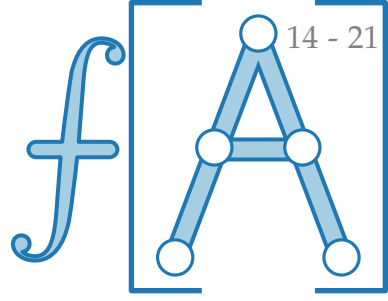
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

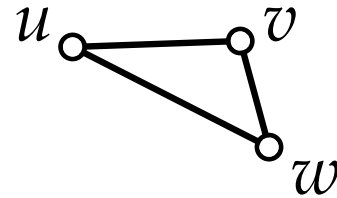
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

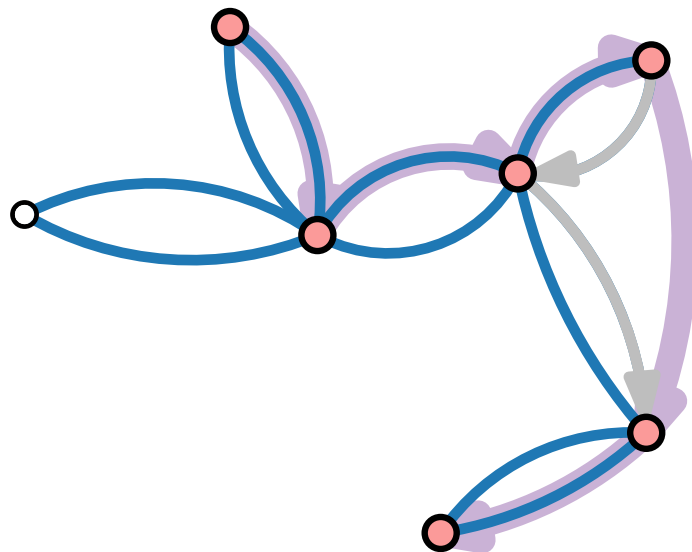
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

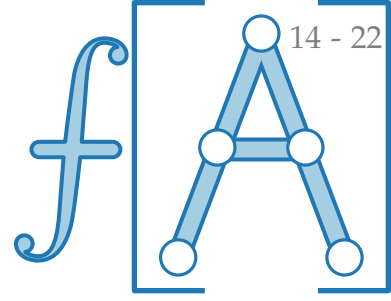
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

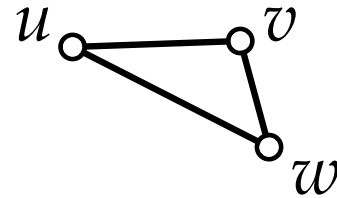
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

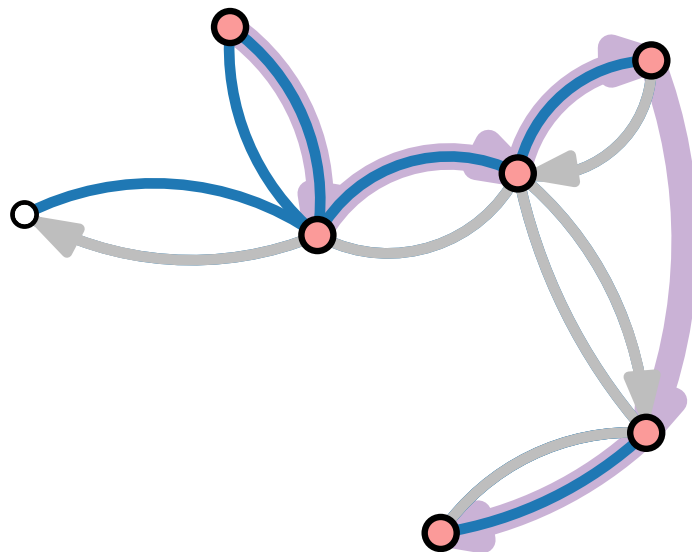
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

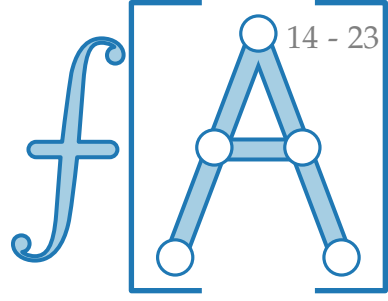
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

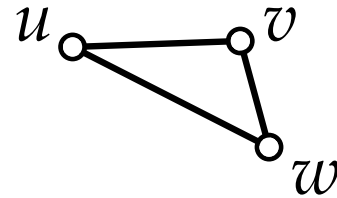
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

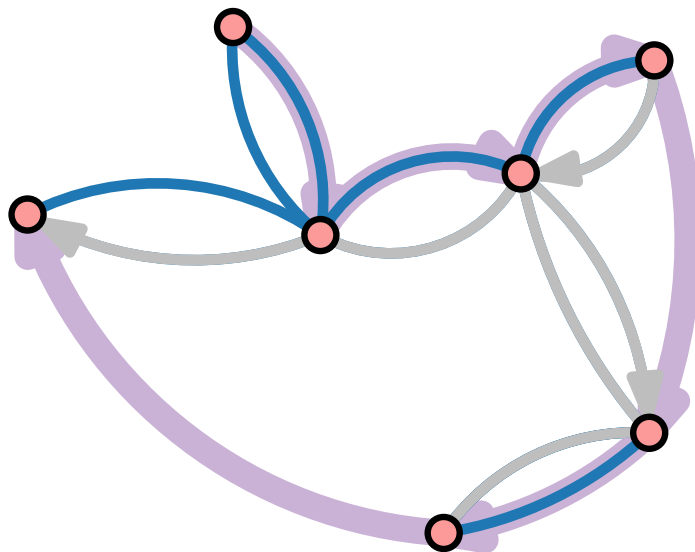
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

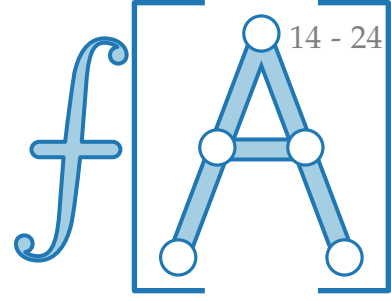
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

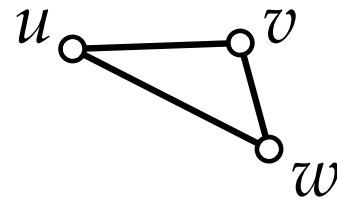
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

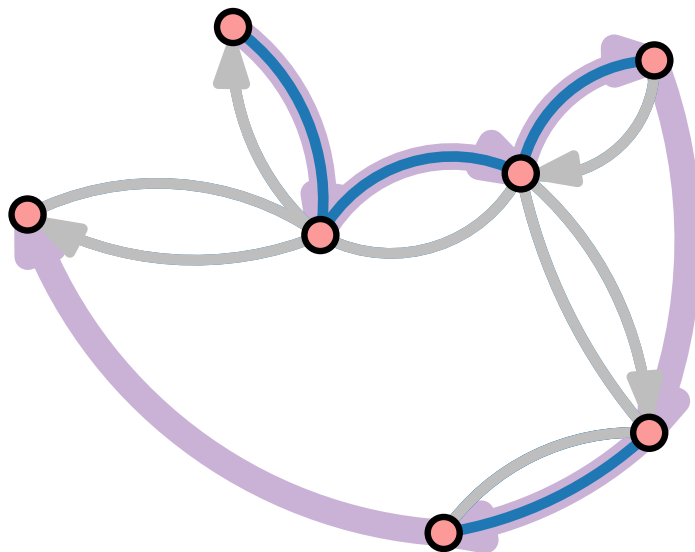
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

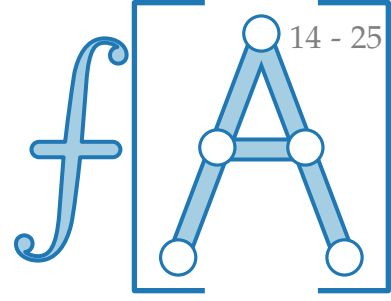
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

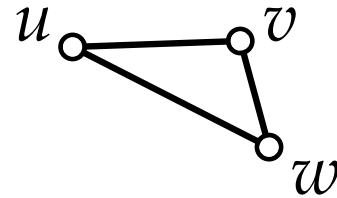
Was tun? – Mach das Problem leichter!



Problem. METRICTRAVELINGSALESMANPROBLEM (Δ -TSP)

Geg. Städte $\{v_1, v_2, \dots, v_n\}$ mit Distanzen $d(v_i, v_j) \in \mathbb{Q}_{\geq 0}$, die die Dreiecksungleichung erfüllen.

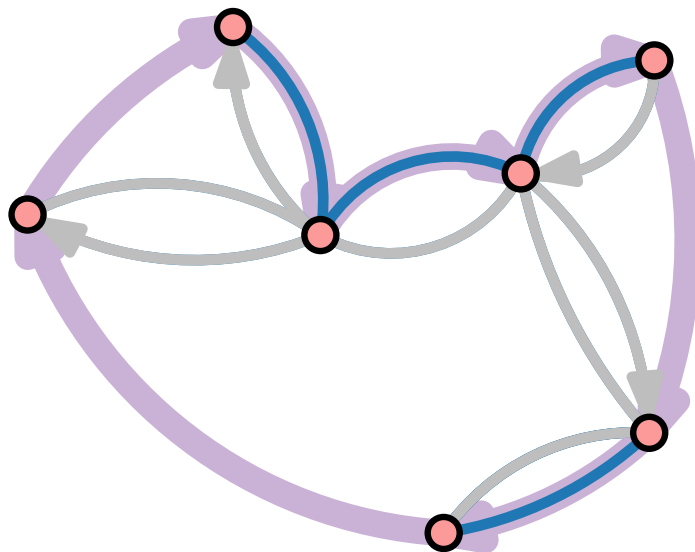
Ges. Tour des Handelsreisenden minimaler Länge, die alle Städte genau einmal besucht und zum Startpunkt zurückkehrt.



d.h. $\forall u, v, w \in V: d(u, w) \leq d(u, v) + d(v, w)$.

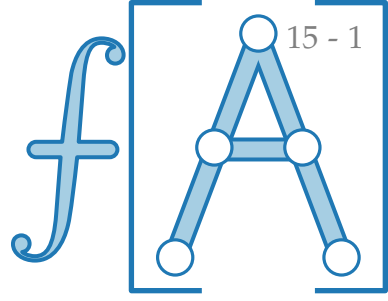
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



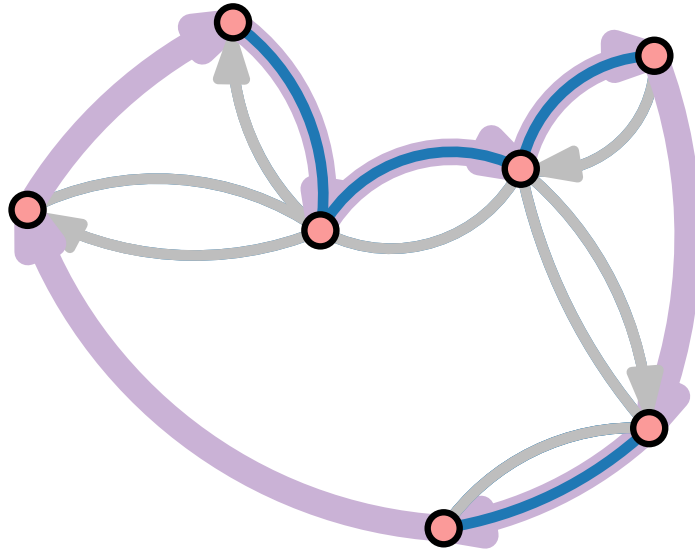
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse



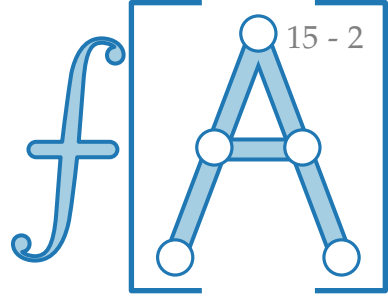
Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



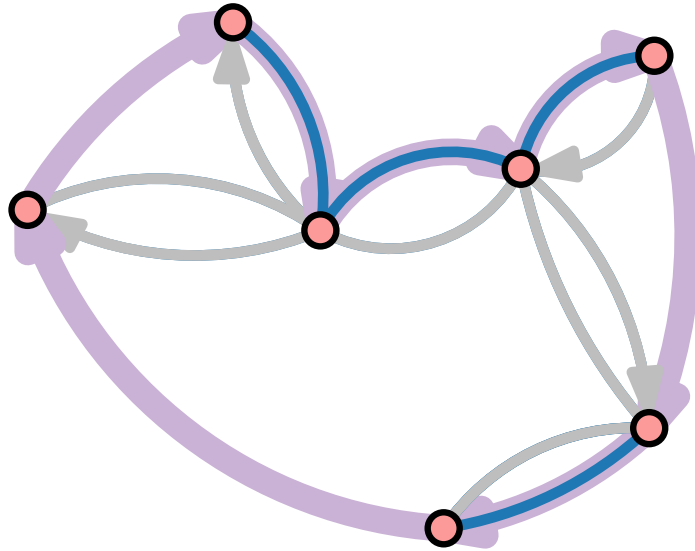
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

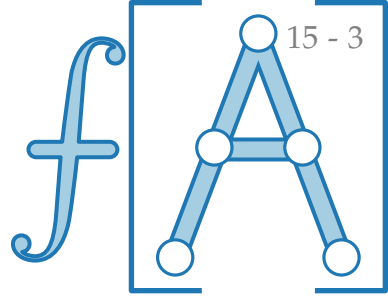
Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

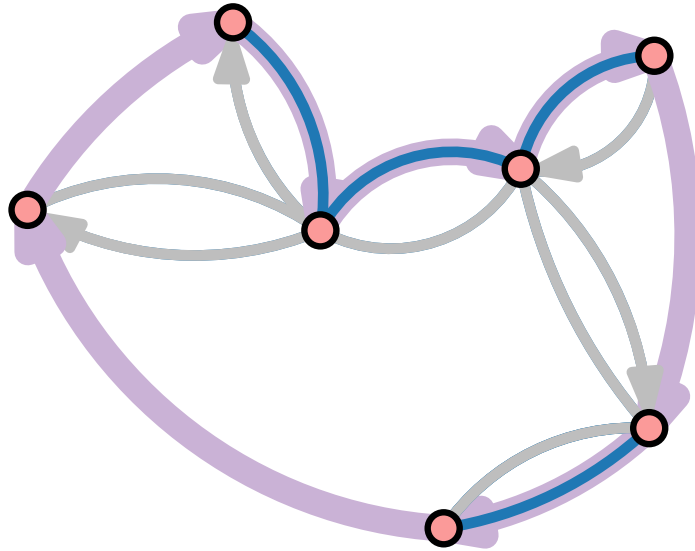
Analyse.

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.

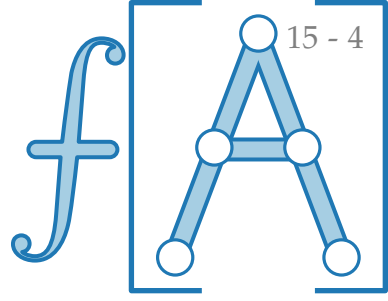


Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

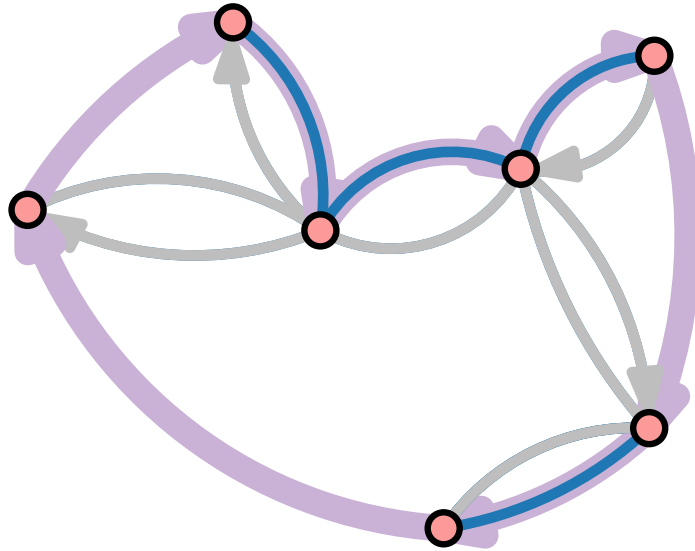
$$d(\text{ALG}) \leq$$

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



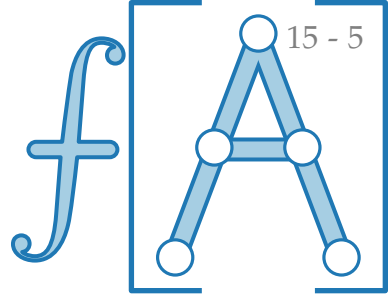
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq$$

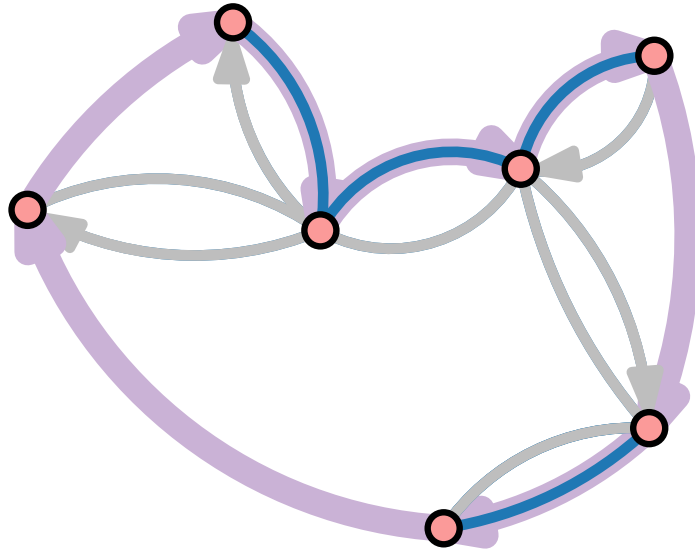
Dreiecksungleichung

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



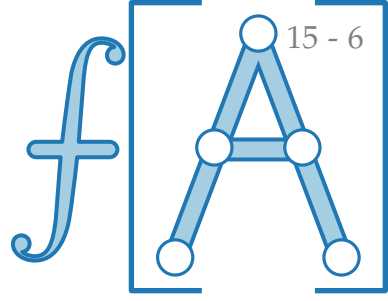
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) =$$

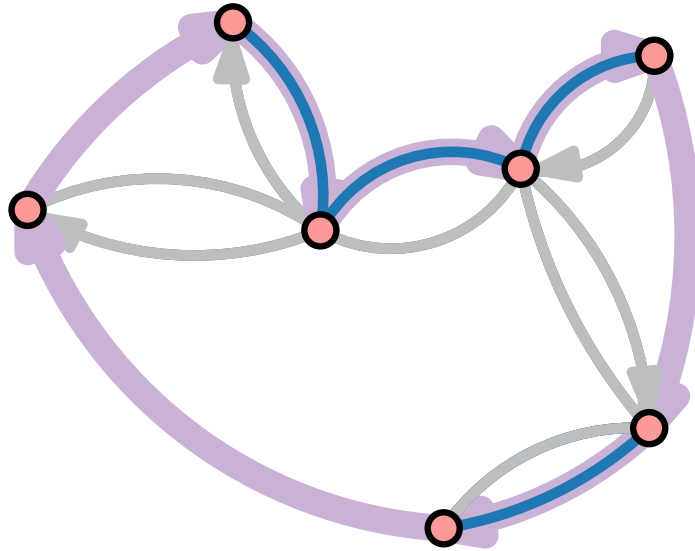
Dreiecksungleichung

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



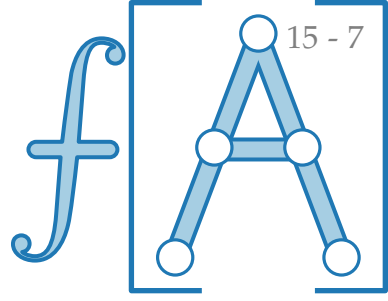
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) = 2 \cdot d(\text{MSB}) \leq$$

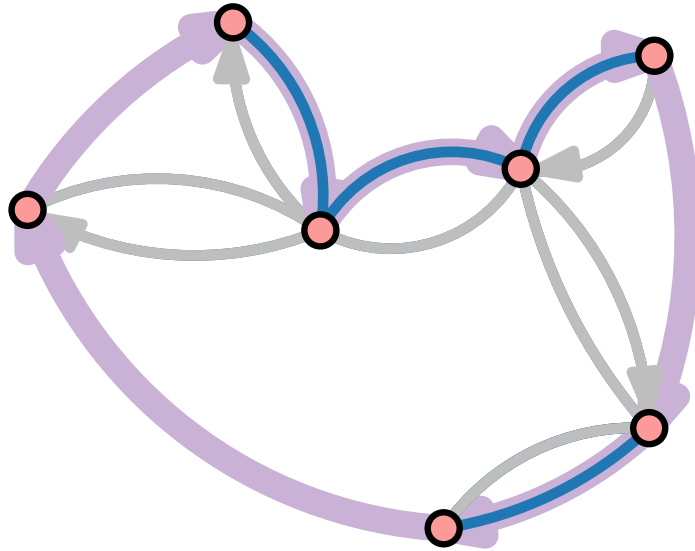
Dreiecksungleichung

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



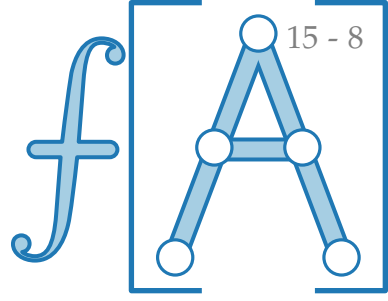
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) = 2 \cdot d(\text{MSB}) \leq 2 \cdot \boxed{}$$

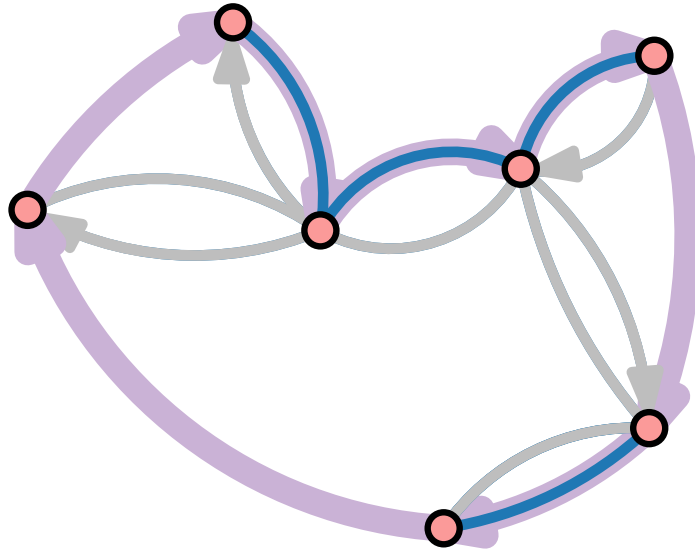
Dreiecksungleichung

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



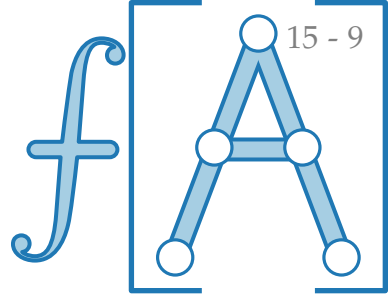
Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) = 2 \cdot d(\text{MSB}) \leq 2 \cdot \text{OPT}$$

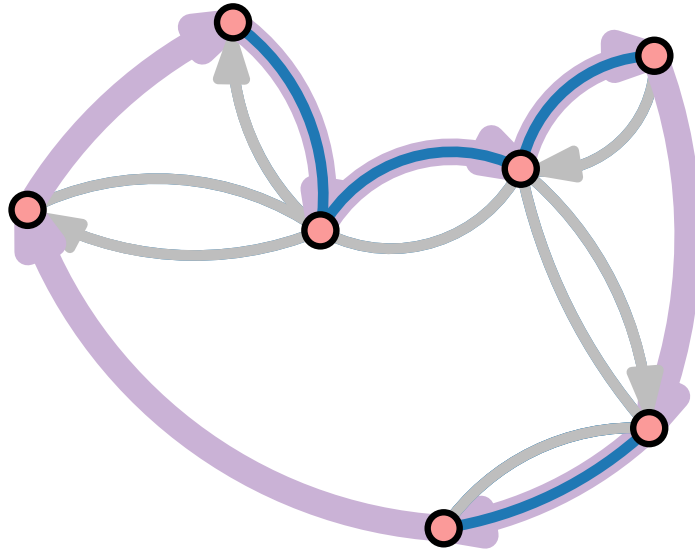
Dreiecksungleichung

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

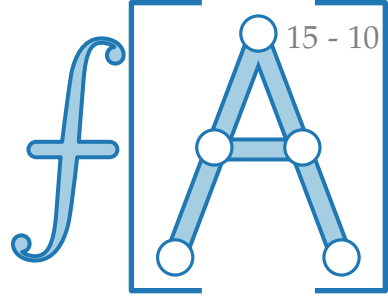
Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) = 2 \cdot d(\text{MSB}) \leq 2 \cdot \text{OPT}$$

Dreiecksungleichung

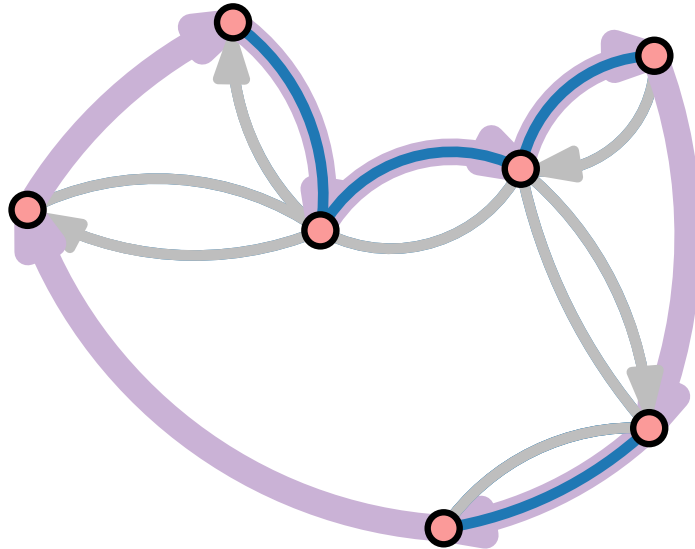
Optimale TSP-Tour minus eine Kante ist
(i.A. nicht minimaler) Spannbaum!

Analyse



Satz. Es gibt eine 2-Approximation für Δ -TSP.

Beweis.



Berechne min. Spannbaum **MSB**.
Verdopple **MSB** $\Rightarrow$ ergibt **Kreis**!
Durchlaufe den **Kreis**.
Überspringe besuchte Knoten.
Füge „Abkürzungen“ ein.

Analyse.

$$d(\text{ALG}) \leq d(\text{Kreis}) = 2 \cdot d(\text{MSB}) \leq 2 \cdot \text{OPT}$$

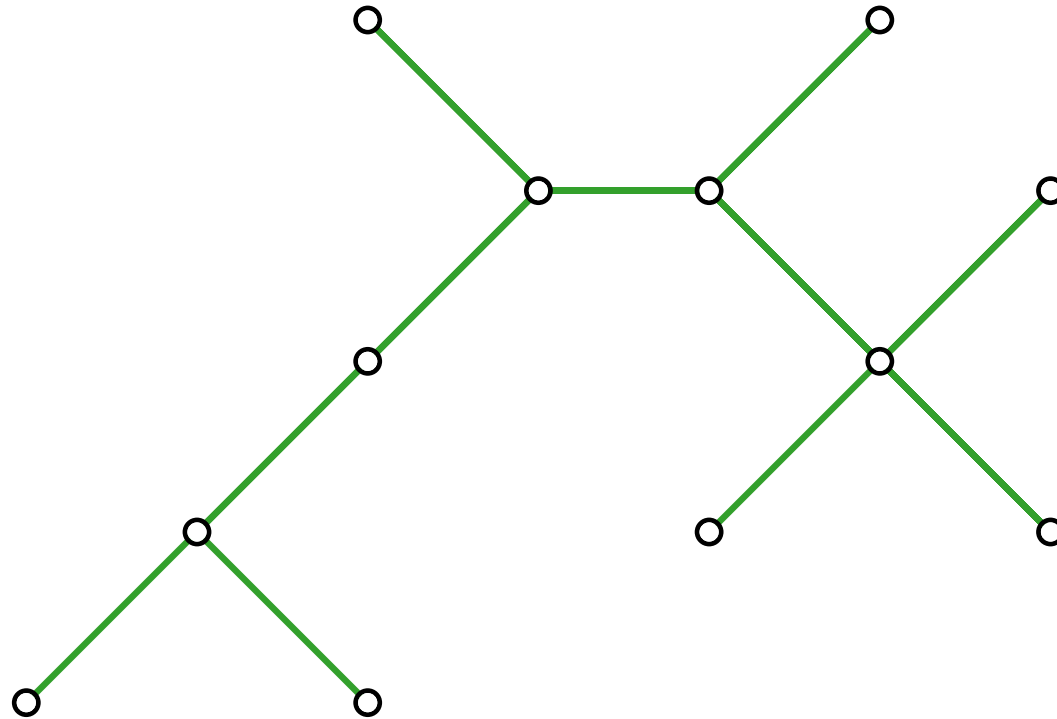
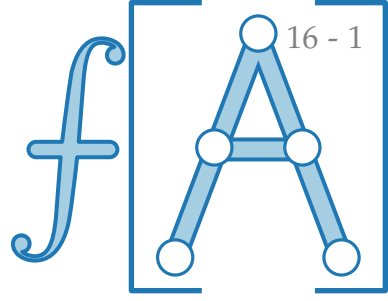
Dreiecksungleichung

Optimale TSP-Tour minus eine Kante ist
(i.A. nicht minimaler) Spannbaum!

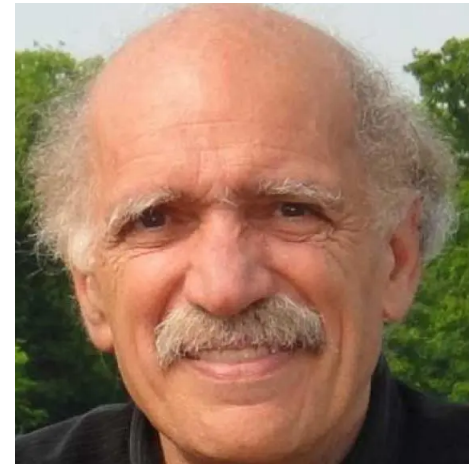
Die „Kunst“ der unteren Schranke: $d(\text{min. Spannbaum}) \leq d(\text{TSP-Tour})$

CHRISTOFIDES Algorithmus

Ermittle einen minimalen Spannbaum B für G .



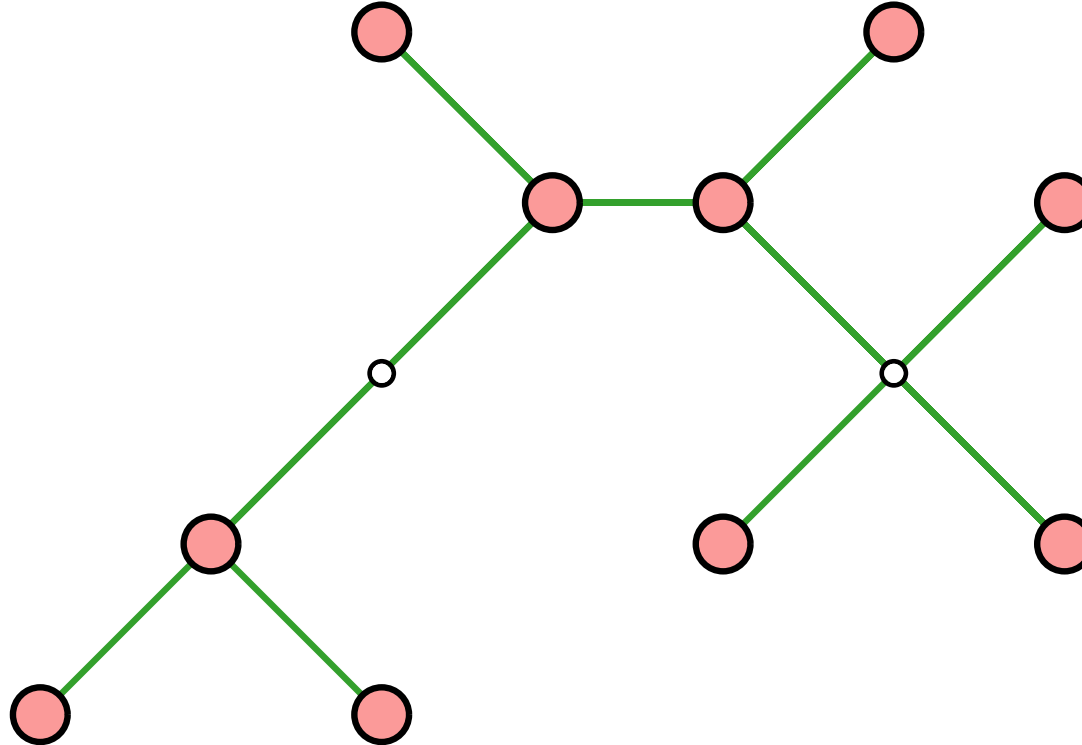
Nicos Christofides
*1942, Cyprus
†2019



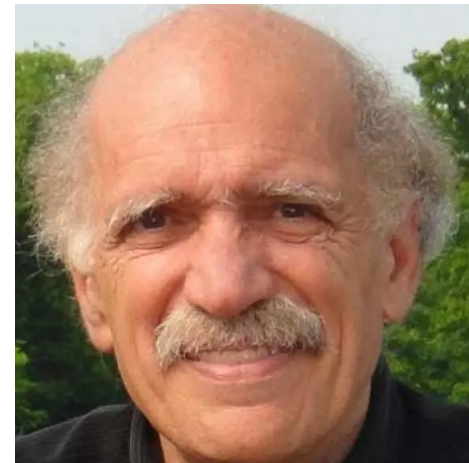
CHRISTOFIDES Algorithmus

Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .



Nicos Christofides
*1942, Cyprus
†2019

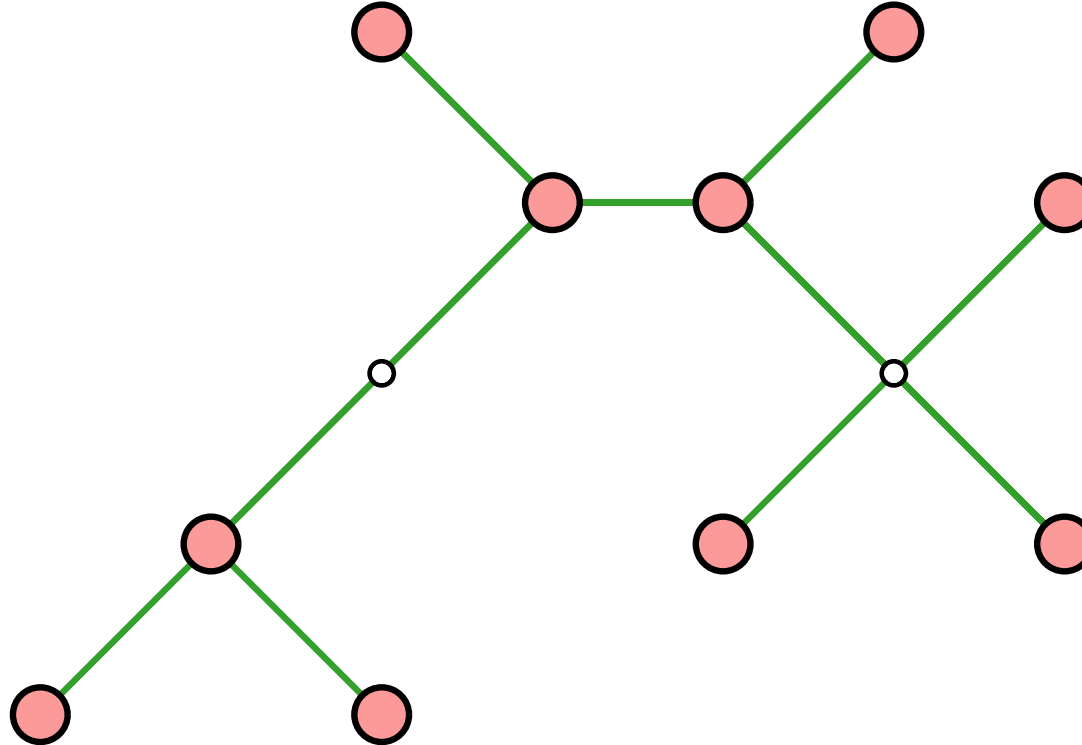


CHRISTOFIDES Algorithmus

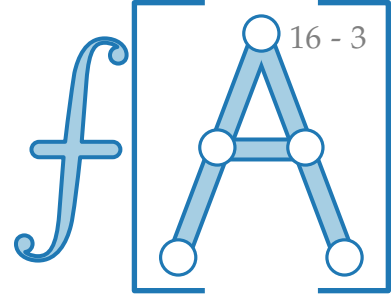
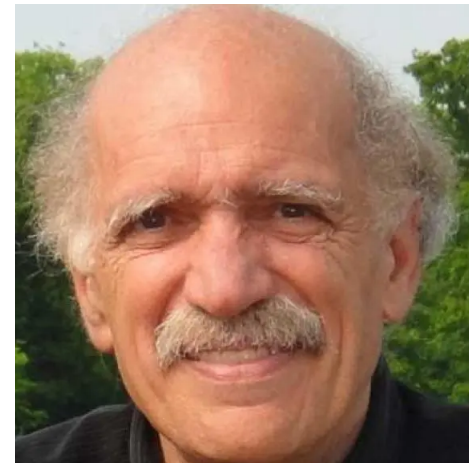
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

Ermittle **kostenminimales perfektes Matching** M für $G[U]$



Nicos Christofides
*1942, Cyprus
†2019

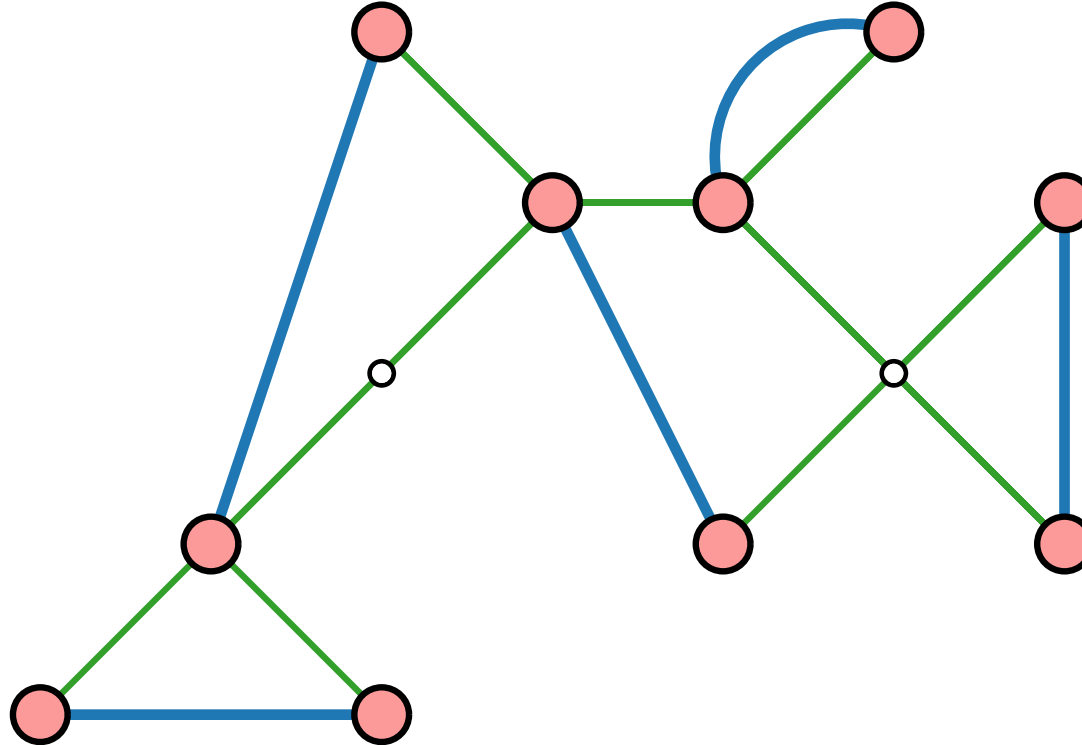


CHRISTOFIDES Algorithmus

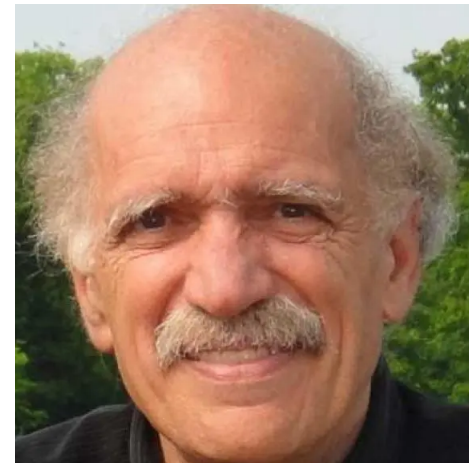
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

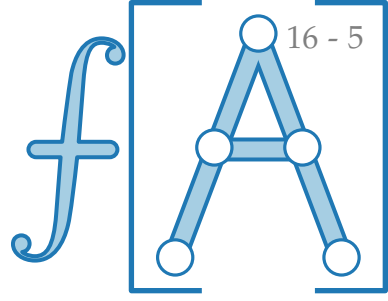
Ermittle **kostenminimales perfektes Matching** M für $G[U]$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus

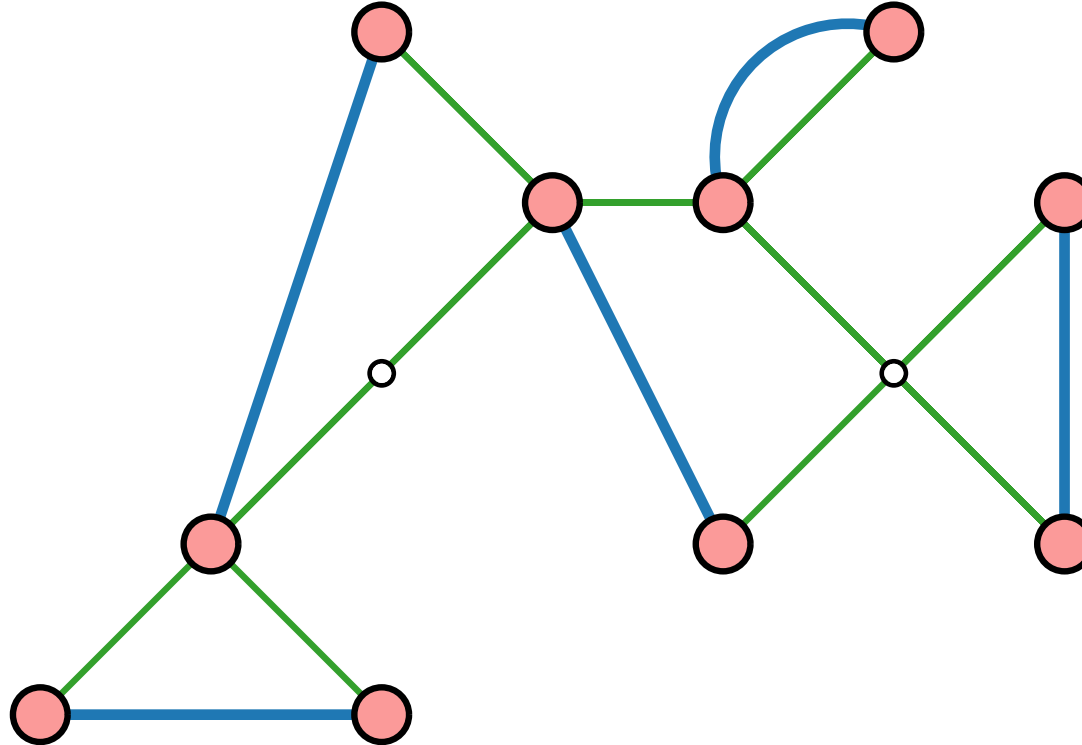


Ermittle einen minimalen Spannbaum B für G .

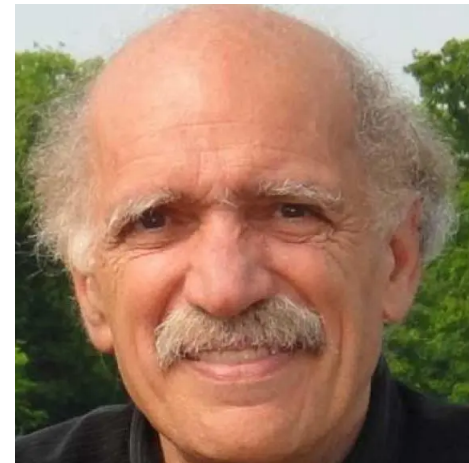
Sei U die Menge der Knoten ungeraden Grades in B .

Ermittle **kostenminimales perfektes Matching** M für $G[U]$

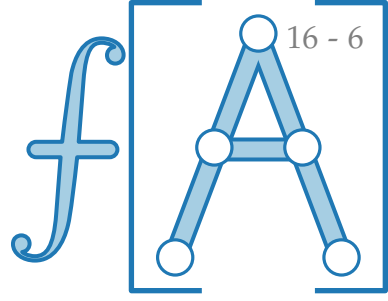
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus

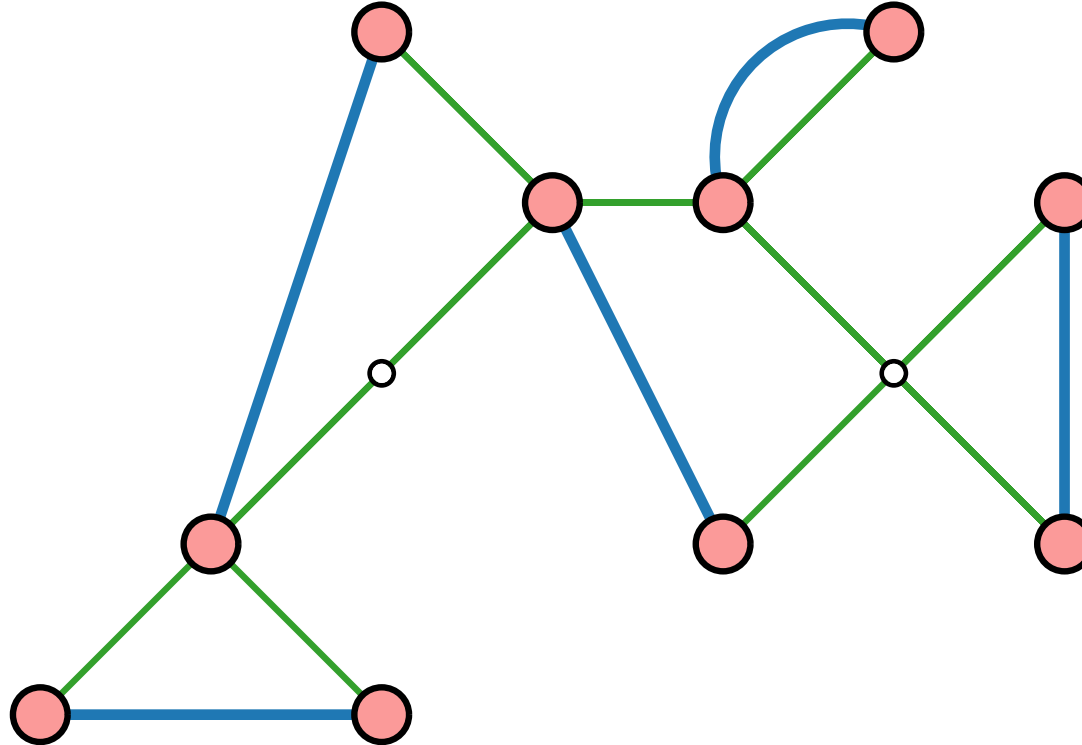


Ermittle einen minimalen Spannbaum B für G .

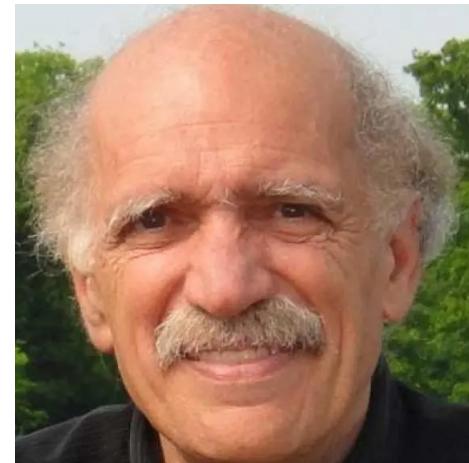
Sei U die Menge der Knoten ungeraden Grades in B .

Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

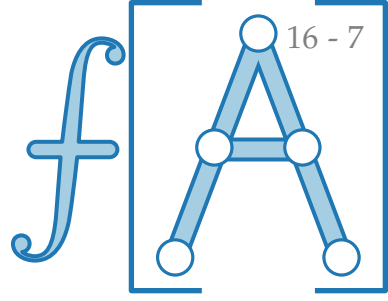
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



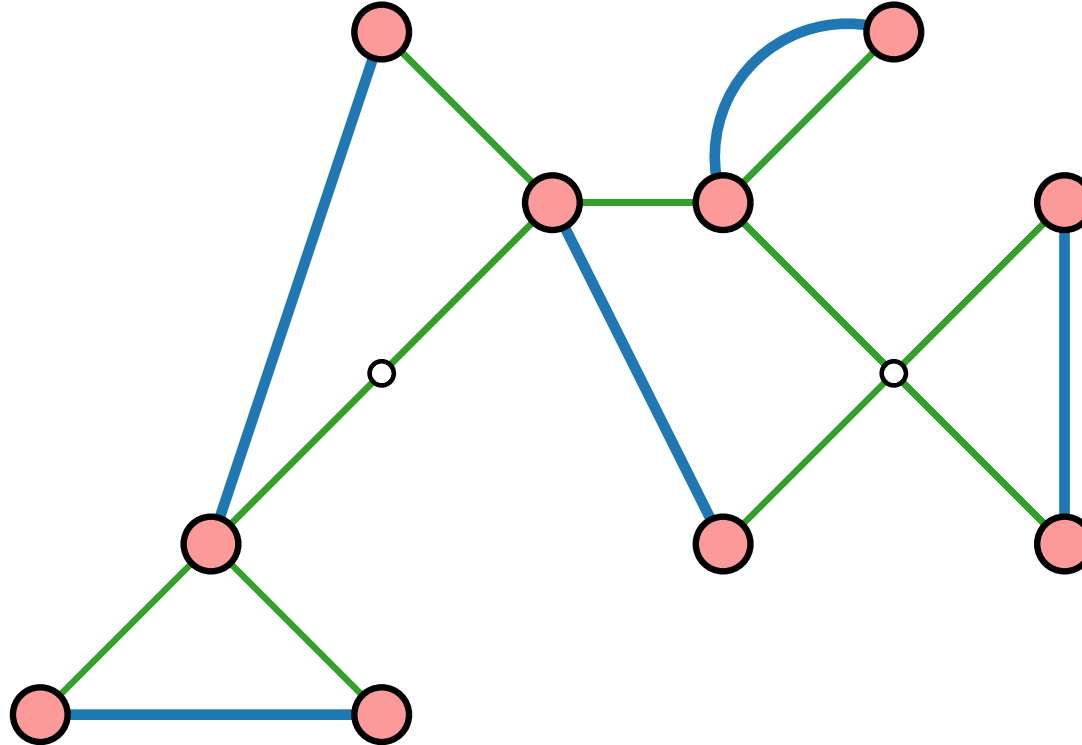
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

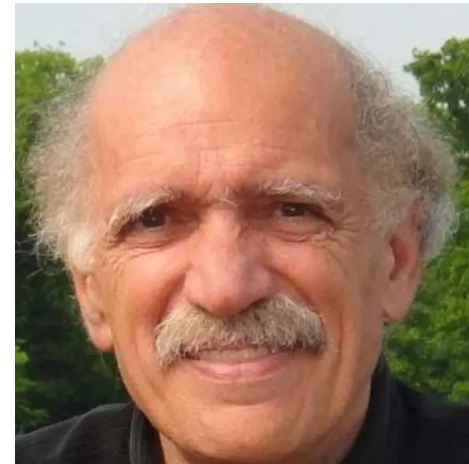
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Warum?

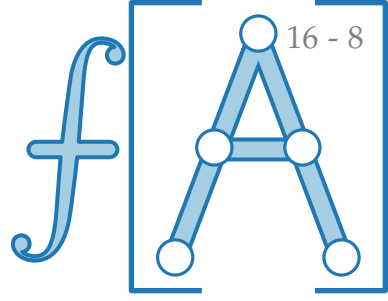
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



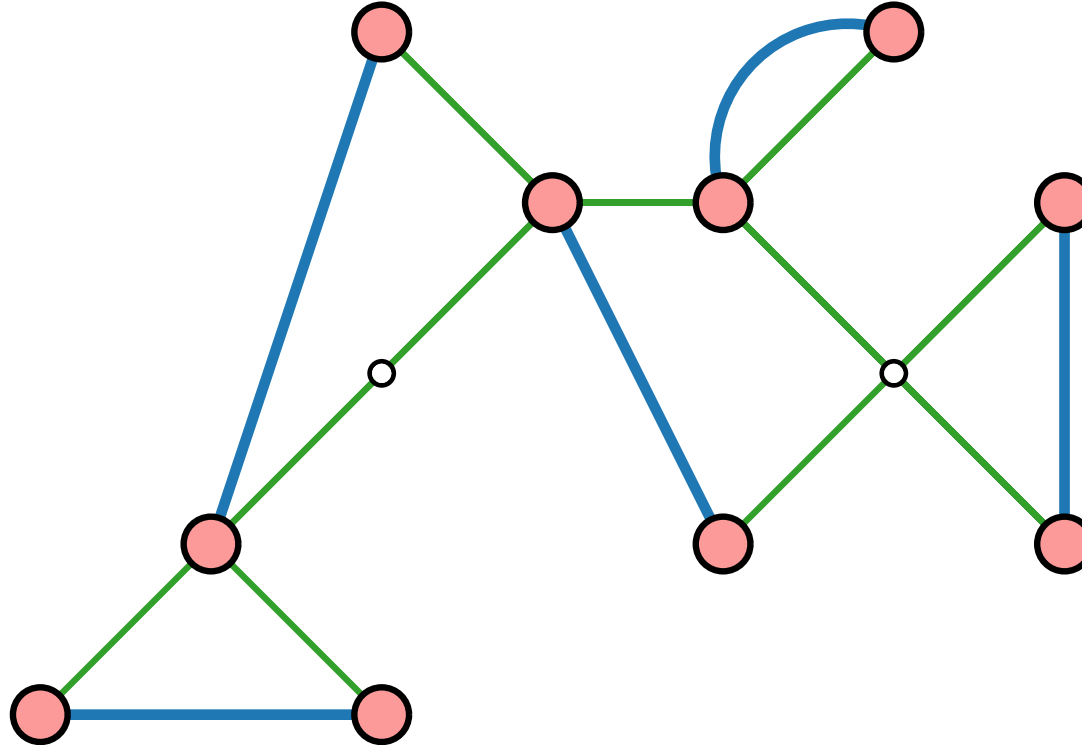
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

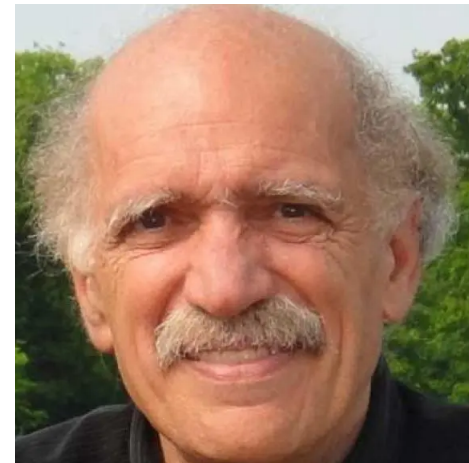
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

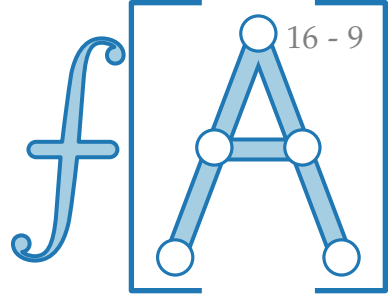
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



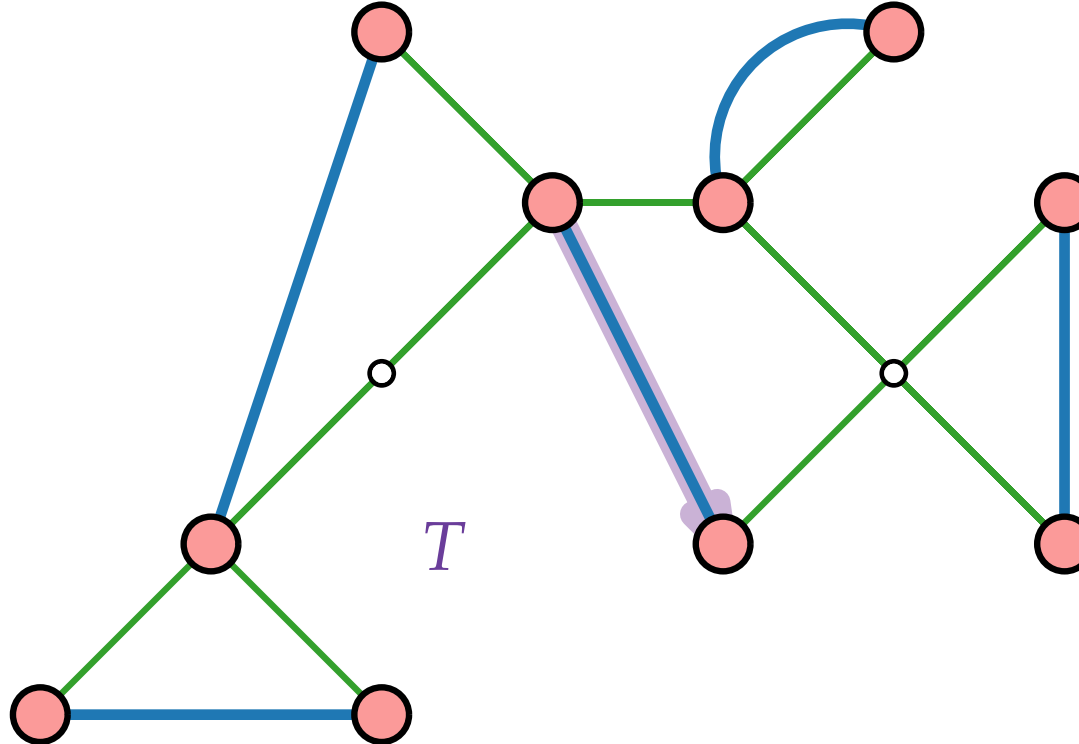
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

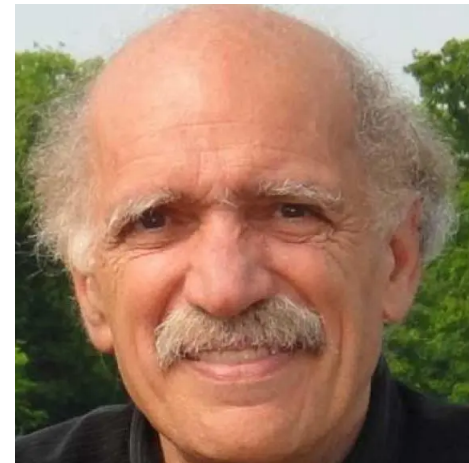
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

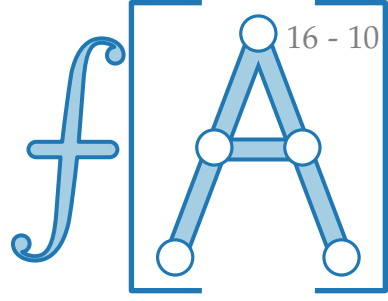
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



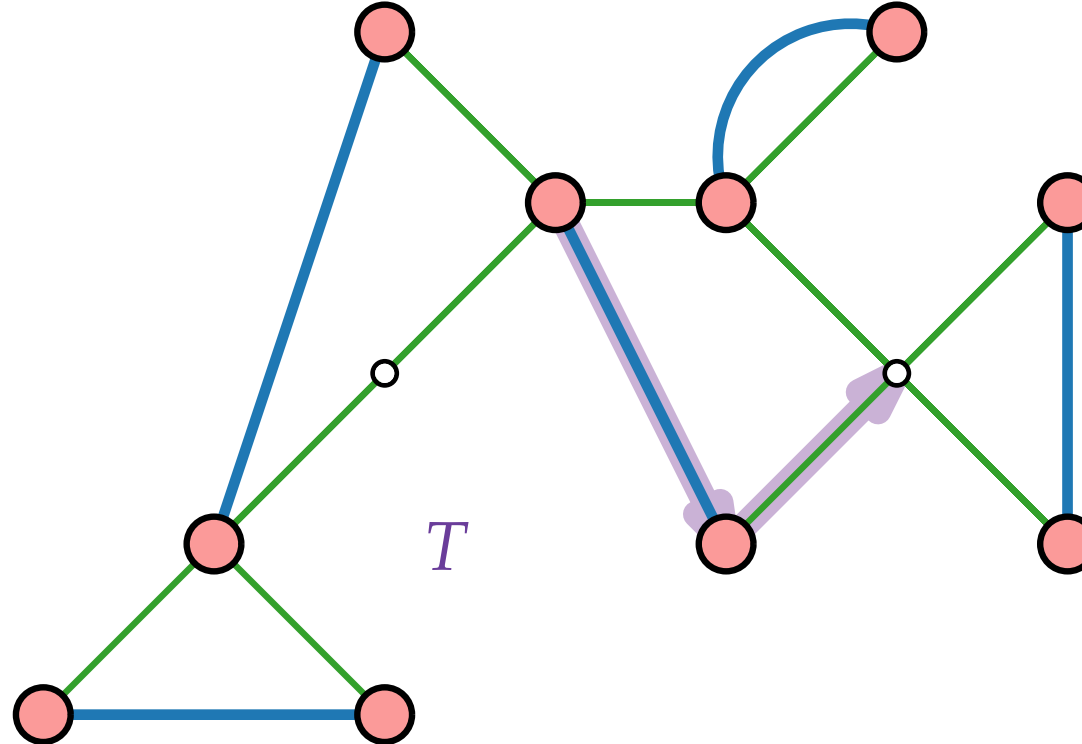
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

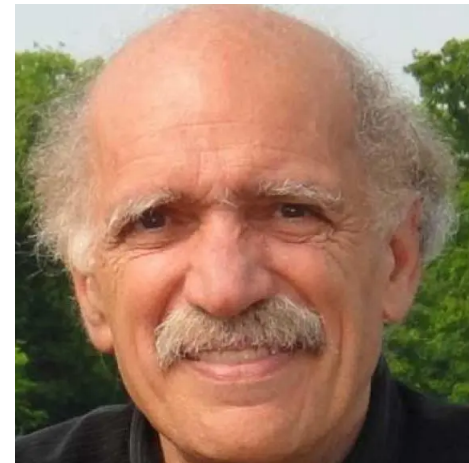
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

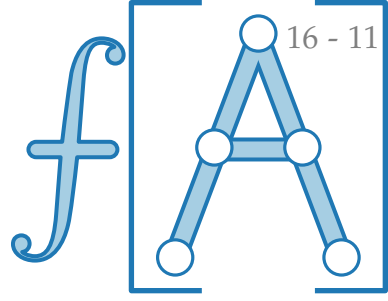
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



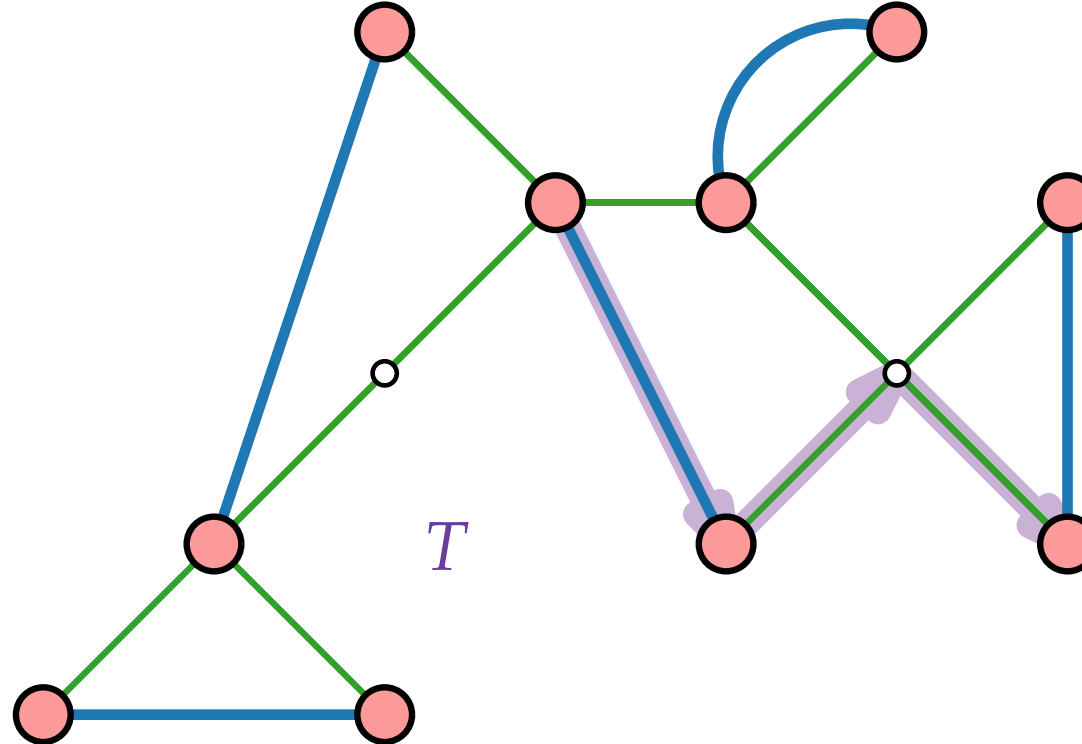
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

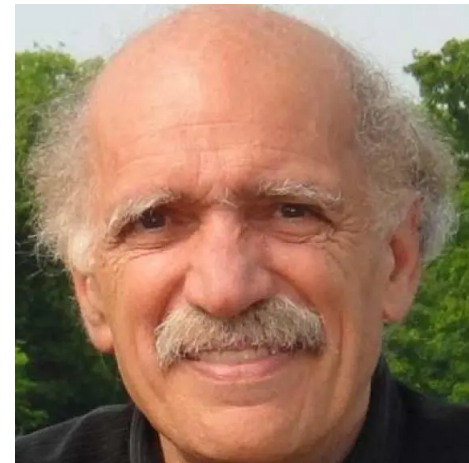
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

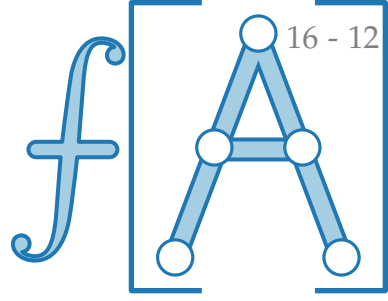
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



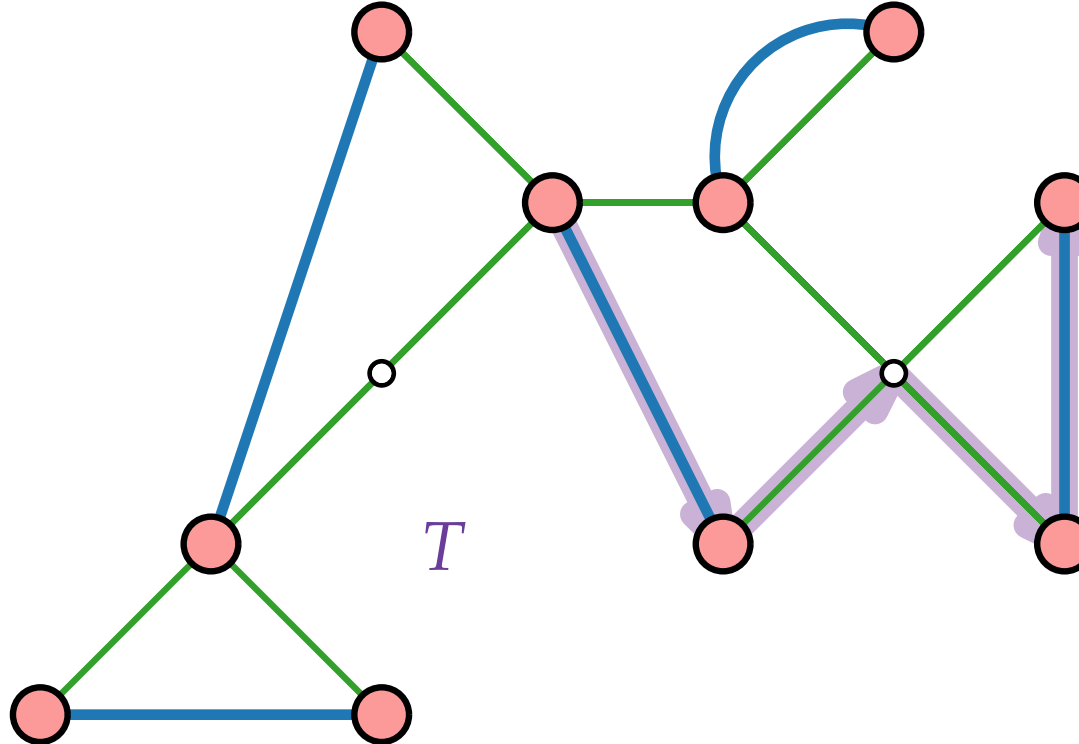
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

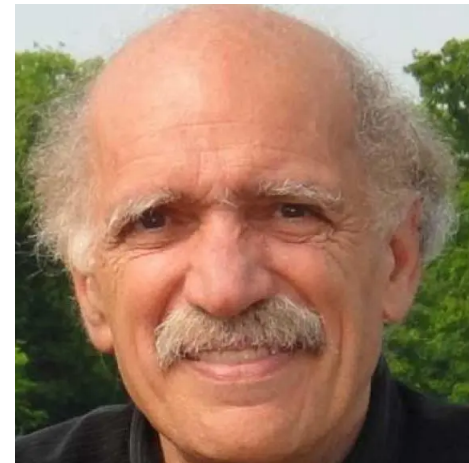
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

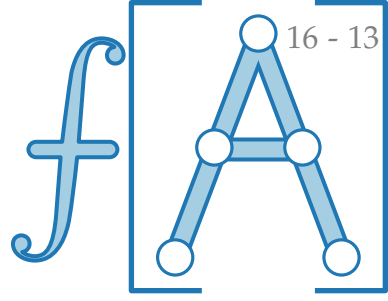
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



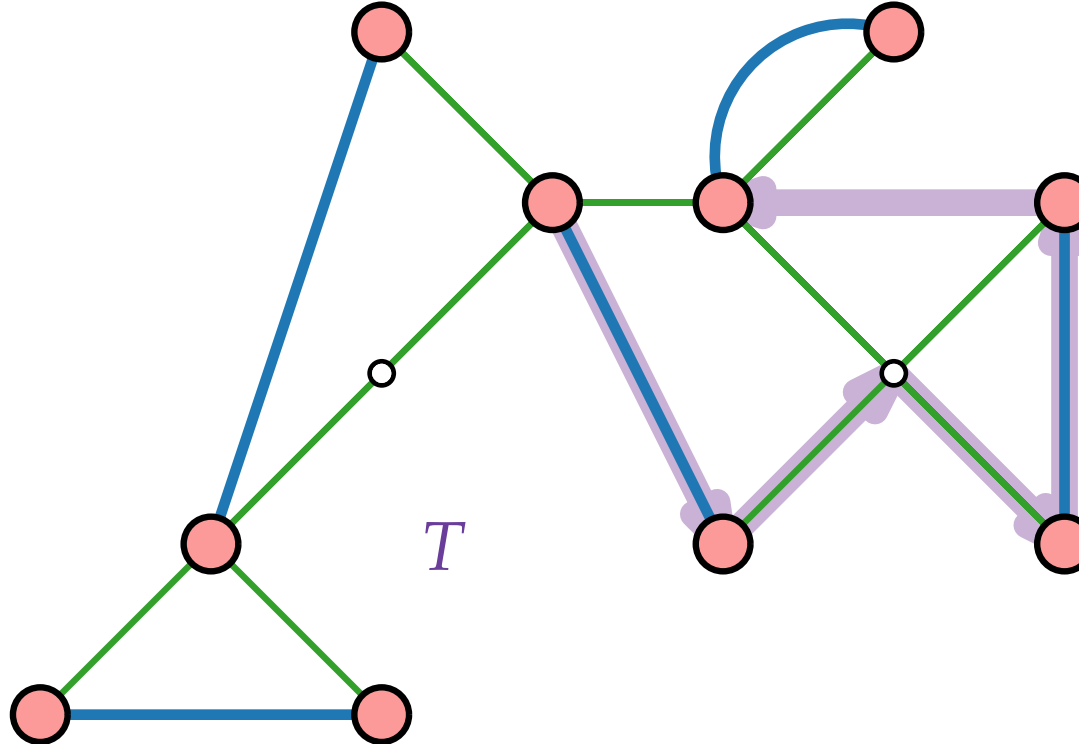
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

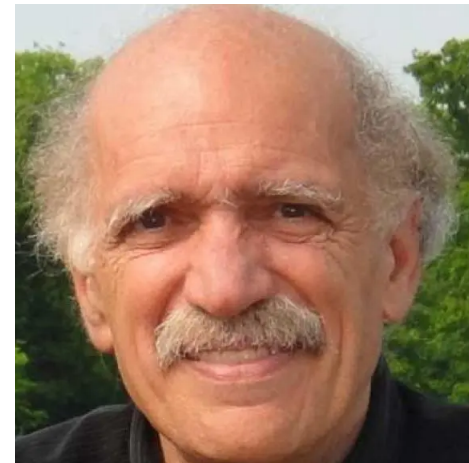
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

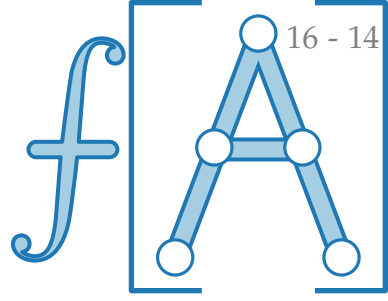
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



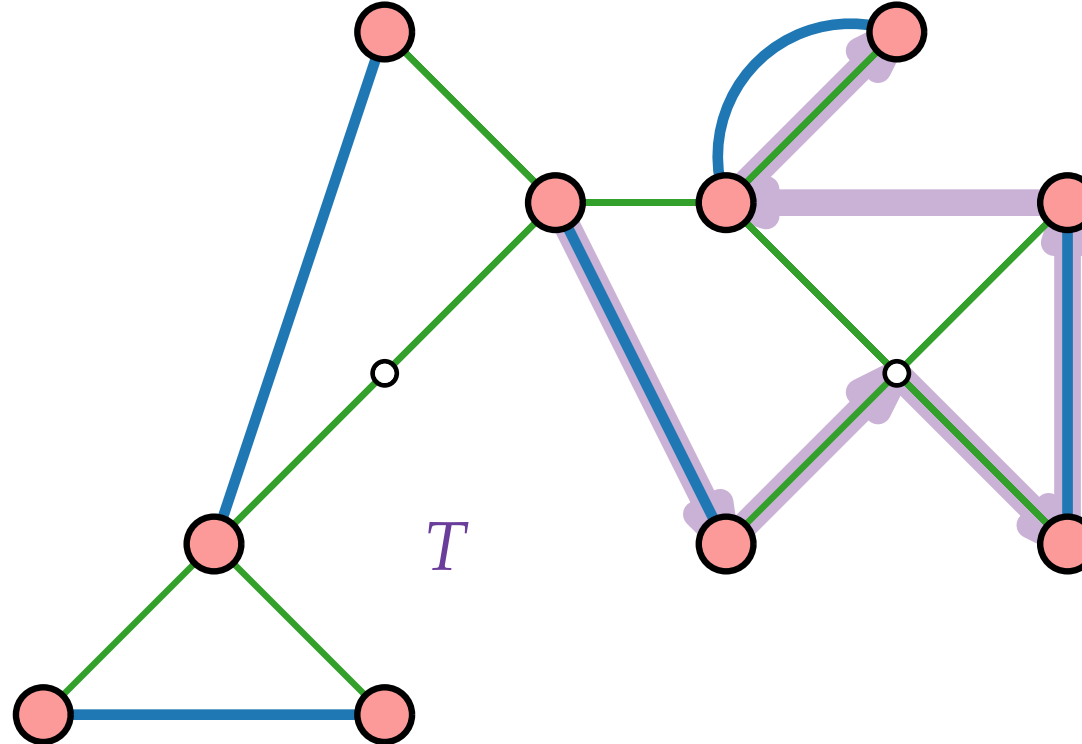
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

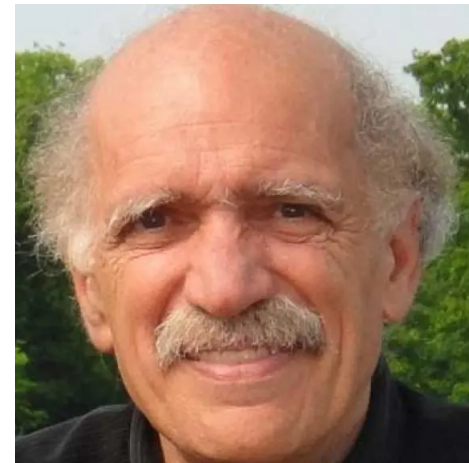
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

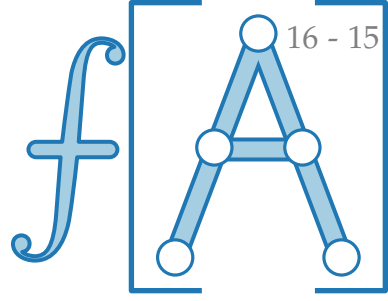
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



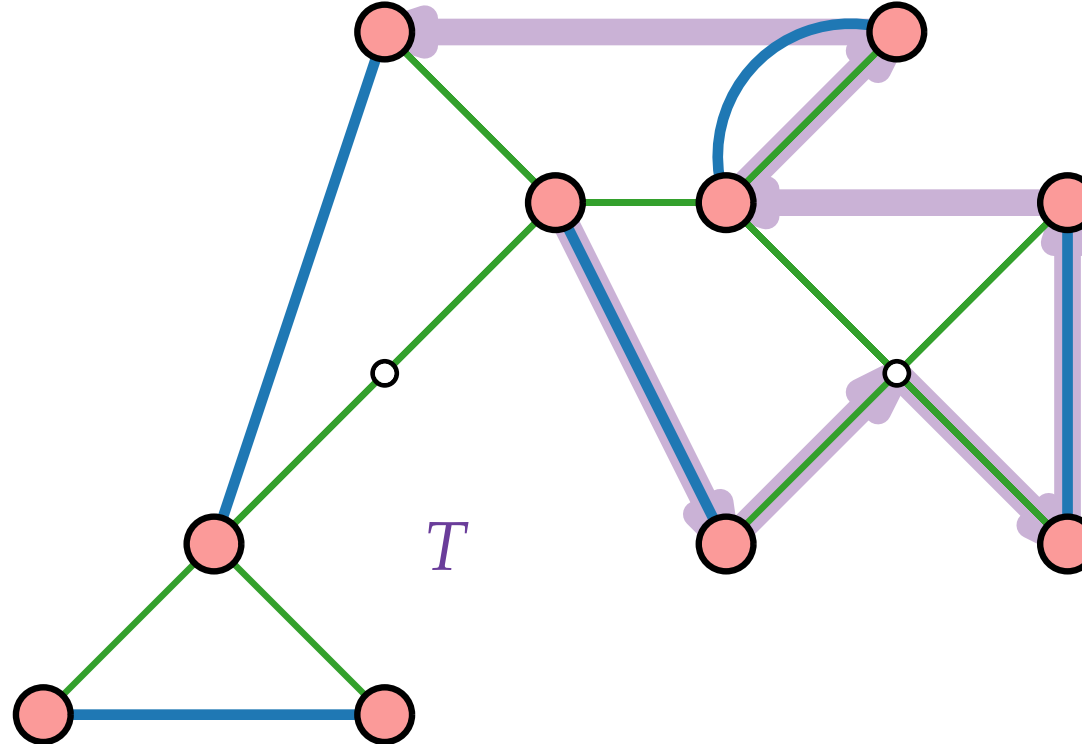
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

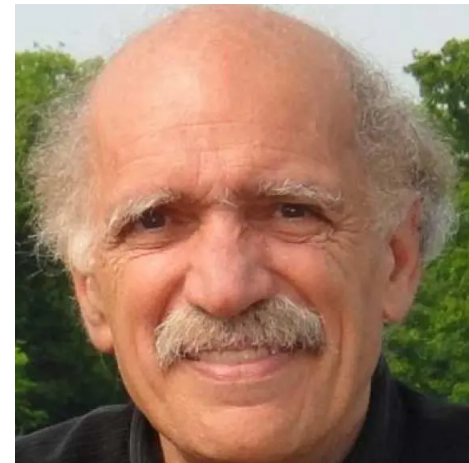
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

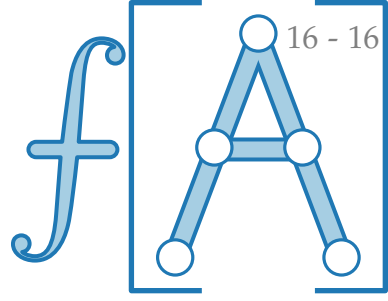
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



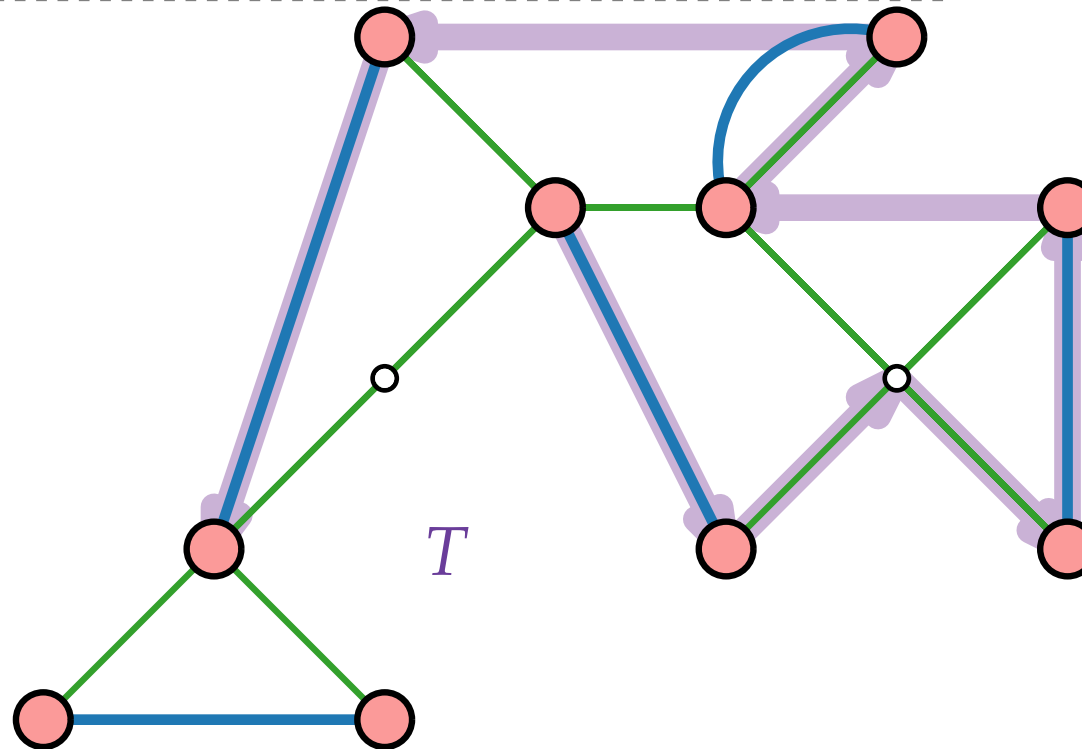
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

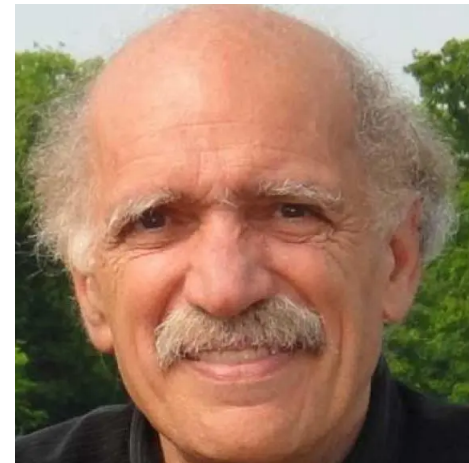
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

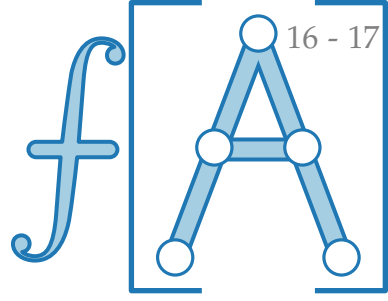
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



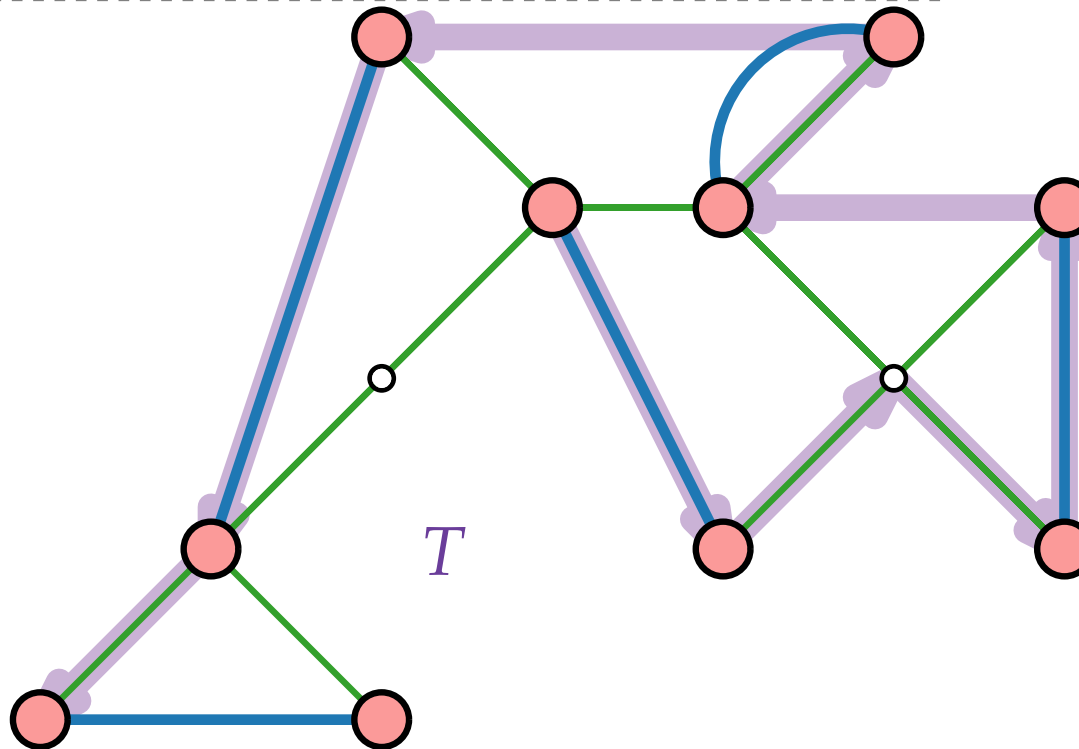
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

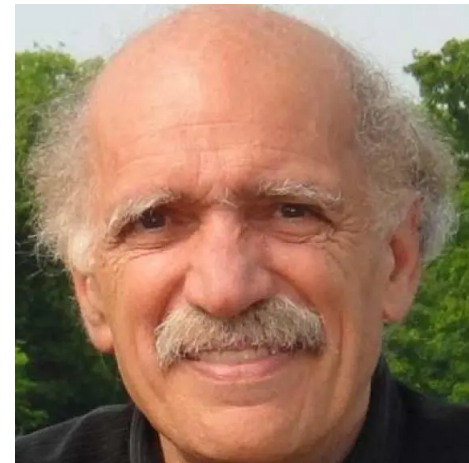
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

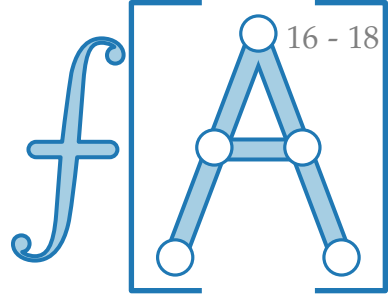
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



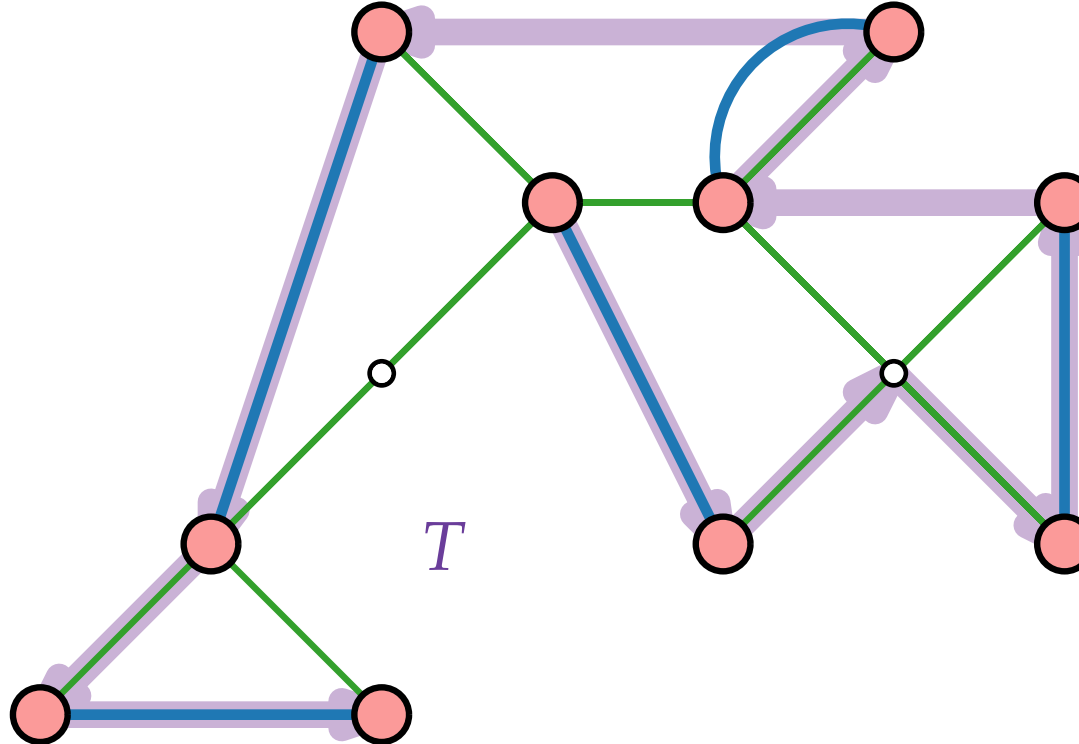
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

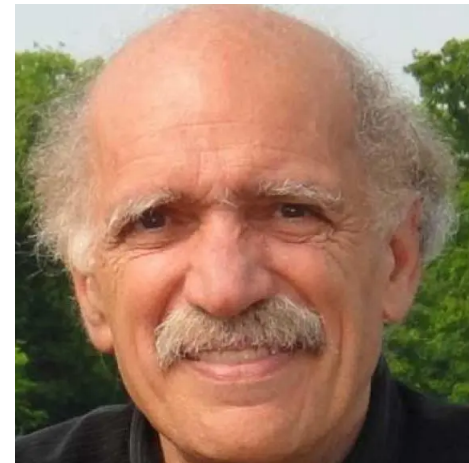
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

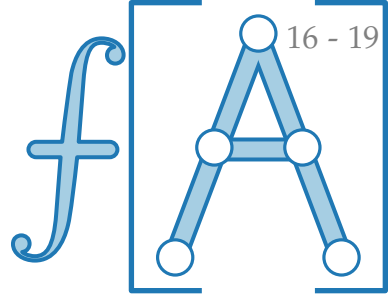
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



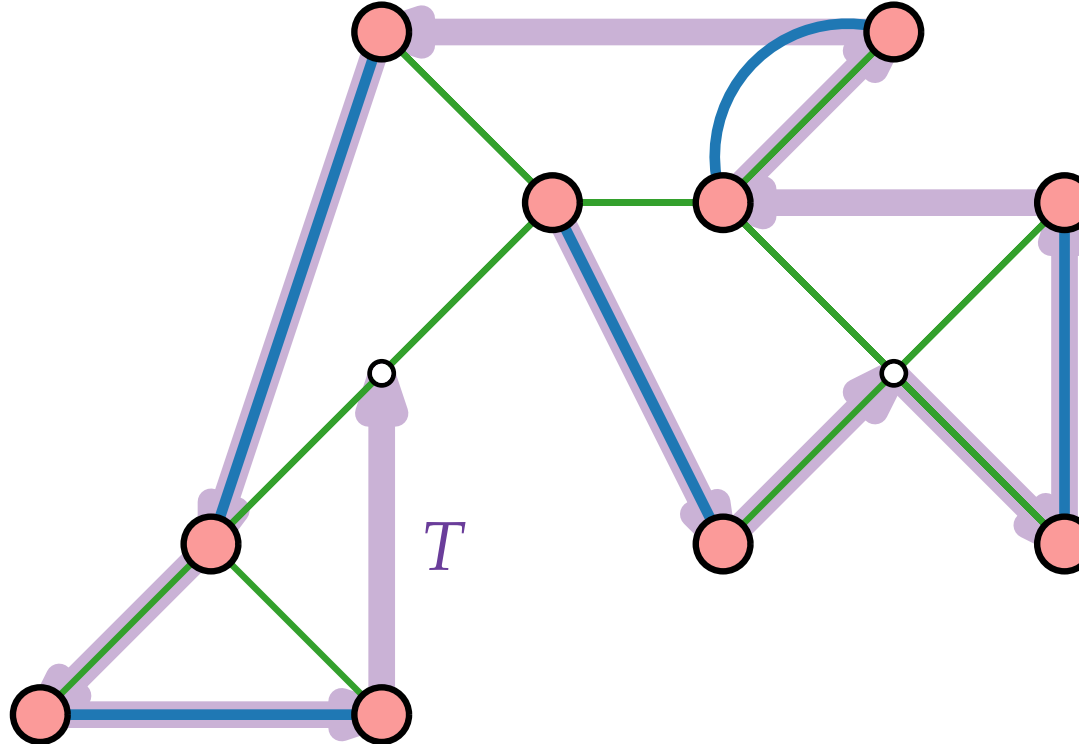
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

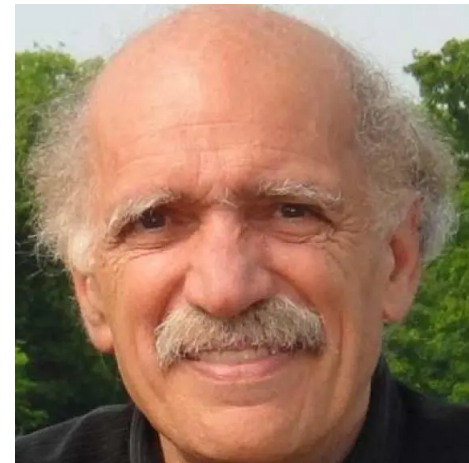
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

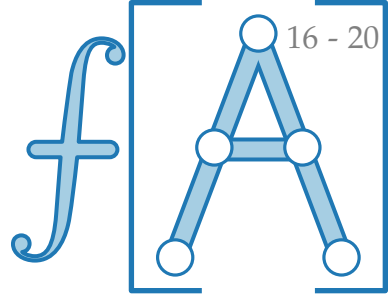
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



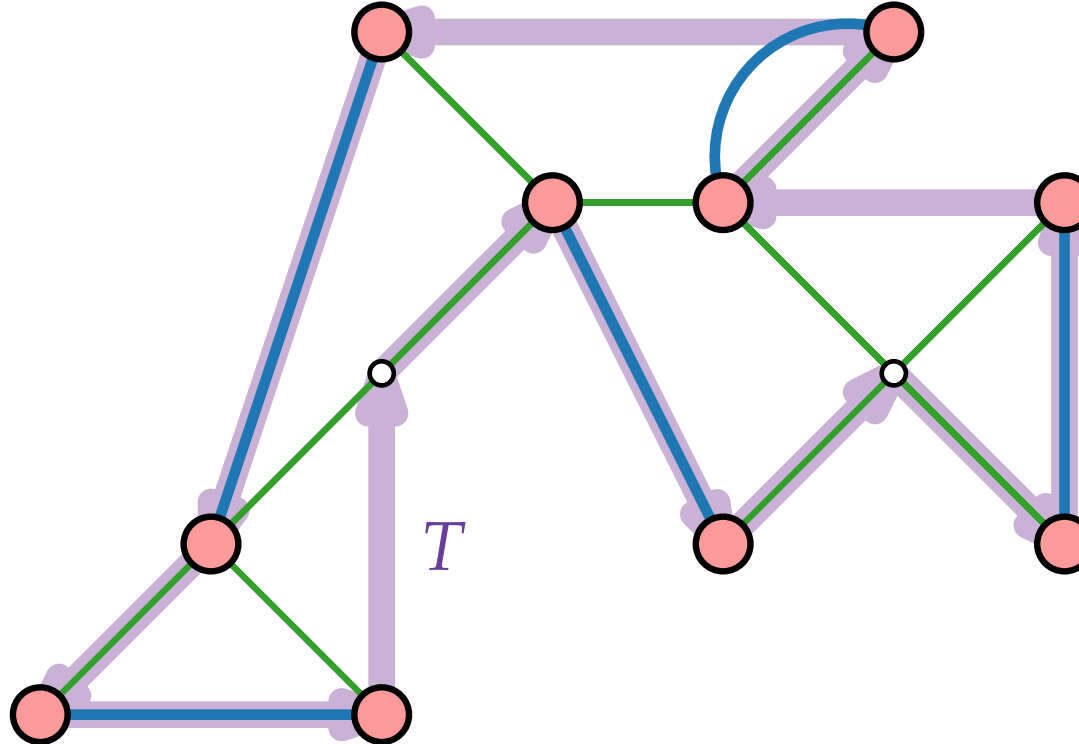
Ermittle einen minimalen Spannbaum B für G .

Sei U die Menge der Knoten ungeraden Grades in B .

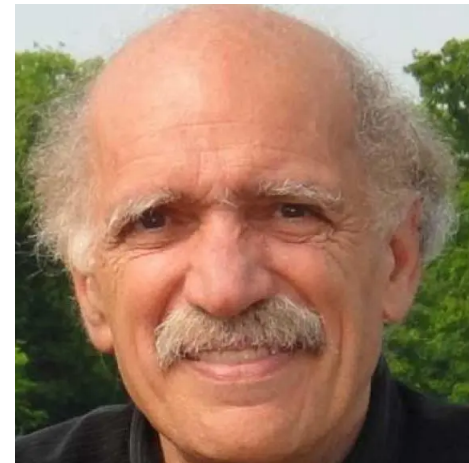
Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

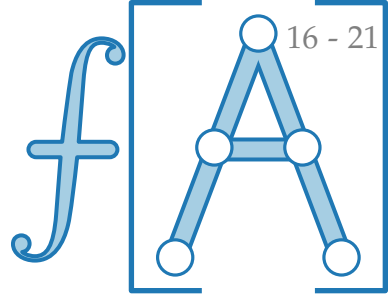
der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$



Nicos Christofides
*1942, Cyprus
†2019



CHRISTOFIDES Algorithmus



Ermittle einen minimalen Spannbaum B für G .

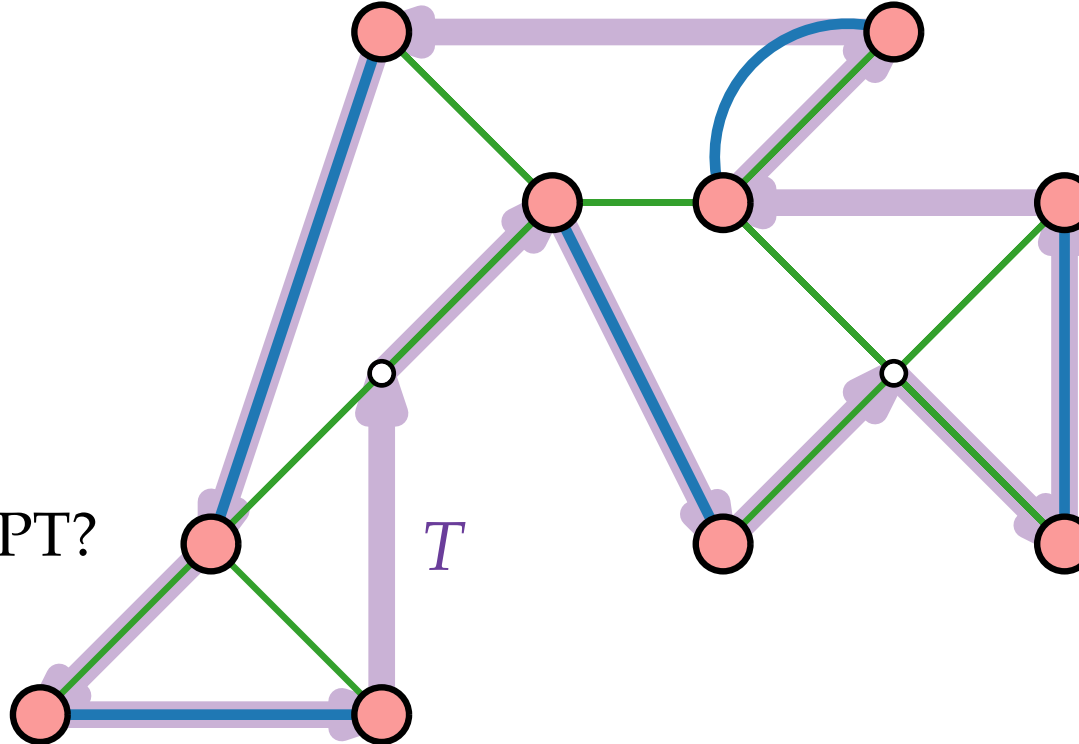
Sei U die Menge der Knoten ungeraden Grades in B .

Ermittle **kostenminimales perfektes Matching** M für $G[U]$
(existiert, da $|U|$ gerade und $G[U]$ vollständig).

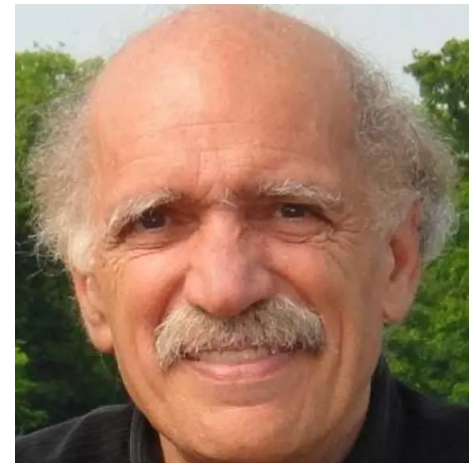
Berechne im eulerschen Graphen $B \cup M$ erst Eulertour
und dann Rundtour T wie bei Tree-Doubling.

der von U induzierte Graph
 $(U, \{(v, w) \in E(G) : v \in U, w \in U\})$

Wie gut ist T im Vergleich zu OPT?

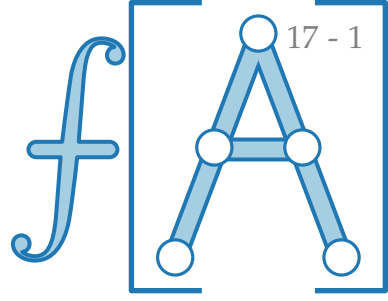


Nicos Christofides
*1942, Cyprus
†2019

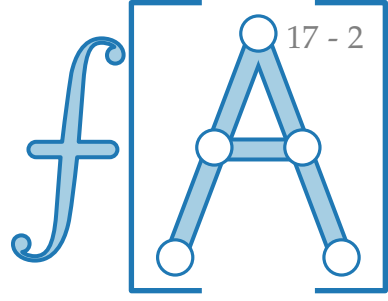


Analyse von CHRISTOFIDES

Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.



Analyse von CHRISTOFIDES

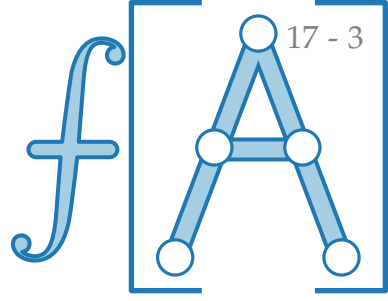


Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$

Analyse von CHRISTOFIDES

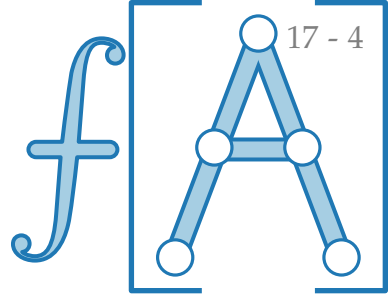


Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.

Analyse von CHRISTOFIDES

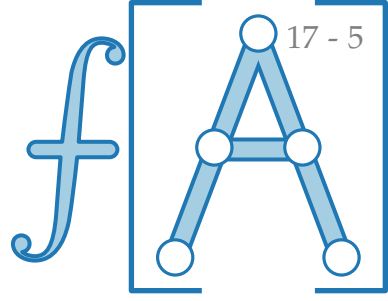


Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.

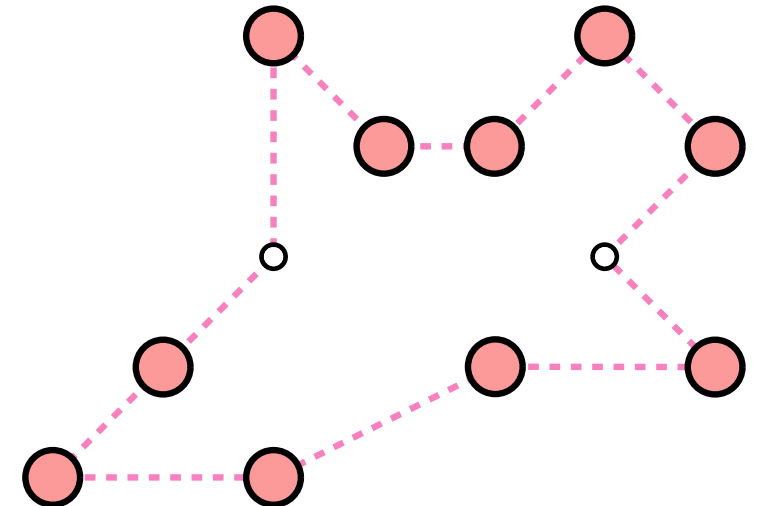
Analyse von CHRISTOFIDES



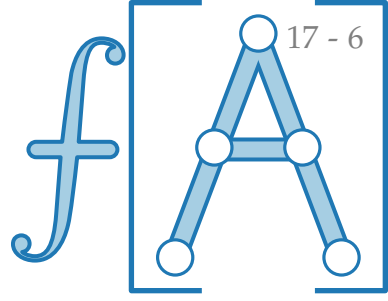
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .



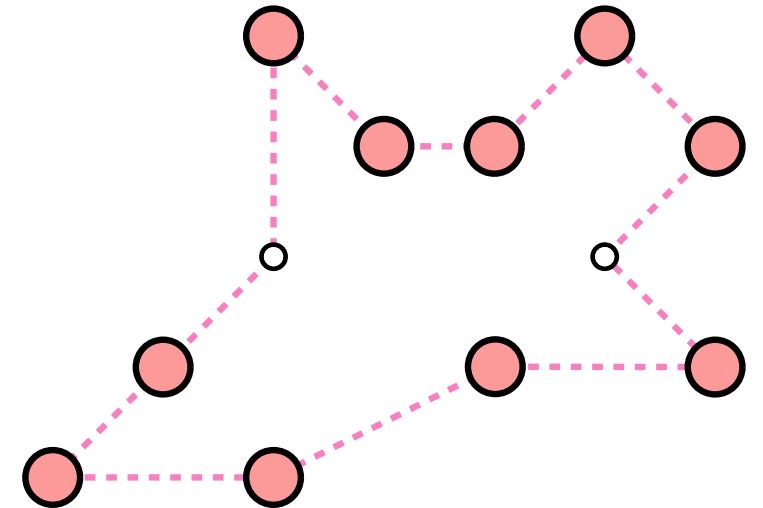
Analyse von CHRISTOFIDES



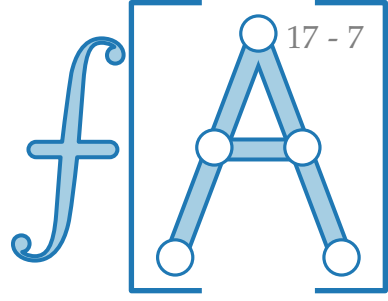
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.



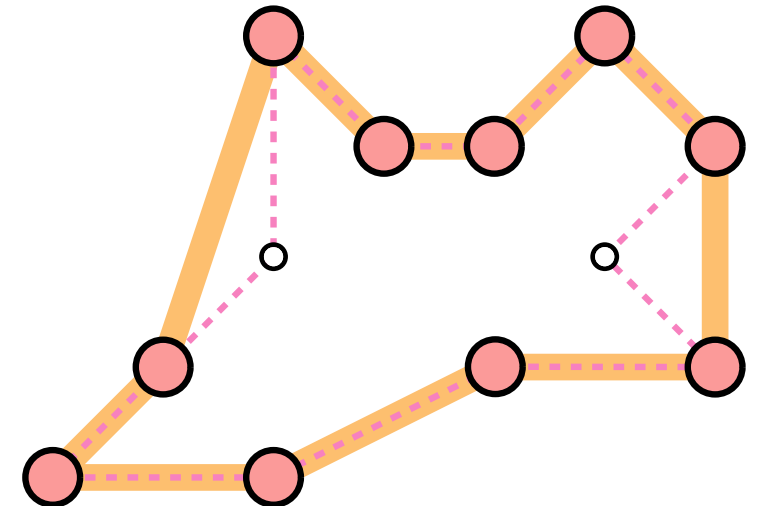
Analyse von CHRISTOFIDES



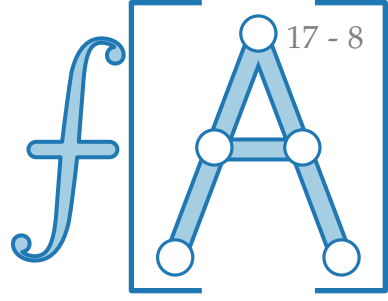
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.



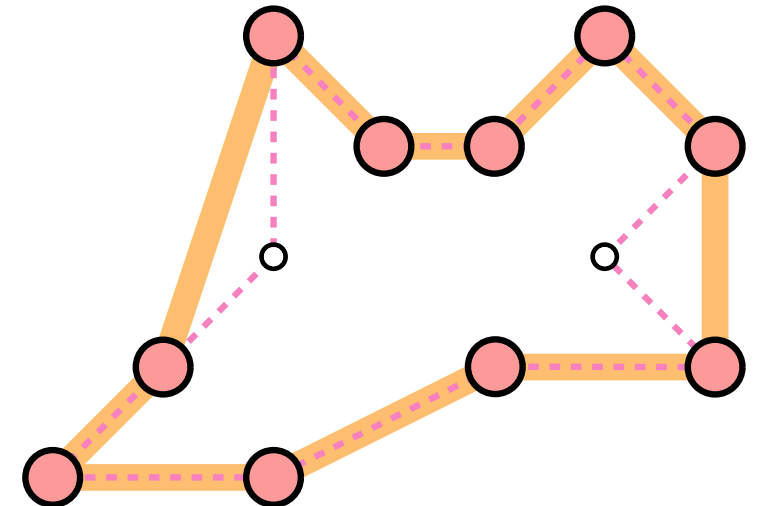
Analyse von CHRISTOFIDES



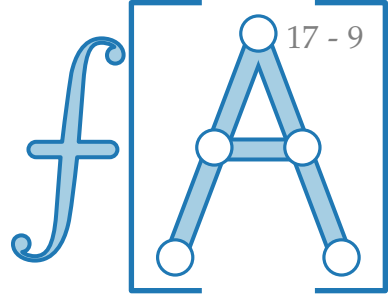
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.



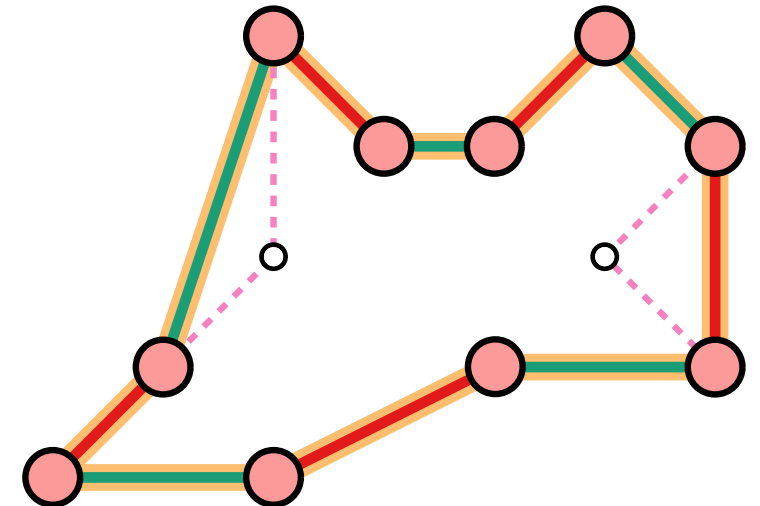
Analyse von CHRISTOFIDES



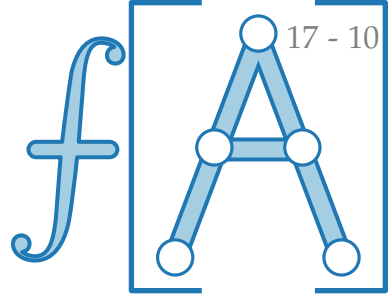
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.



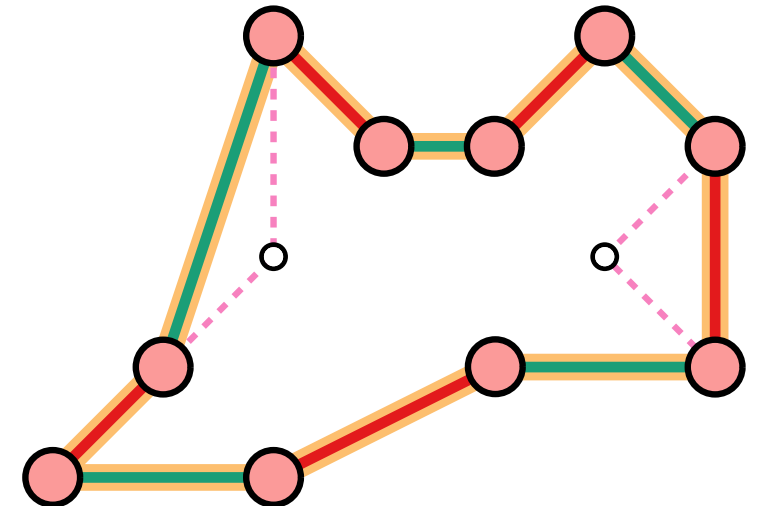
Analyse von CHRISTOFIDES



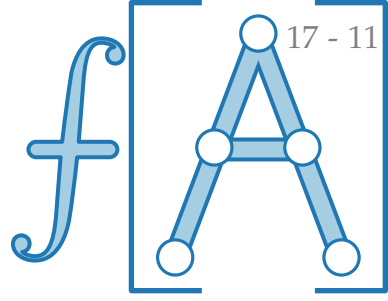
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.



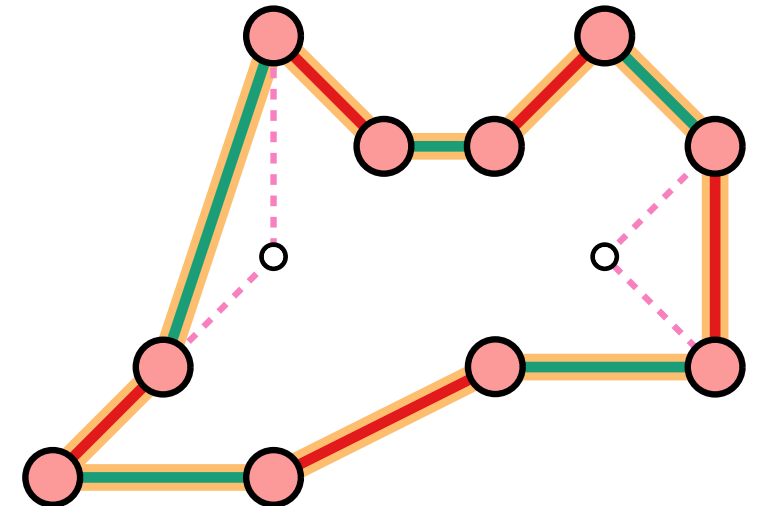
Analyse von CHRISTOFIDES



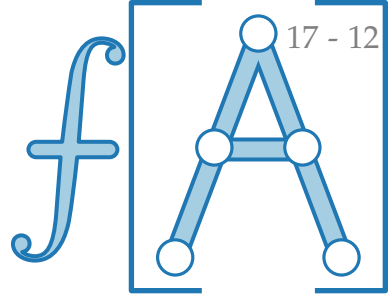
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M')$



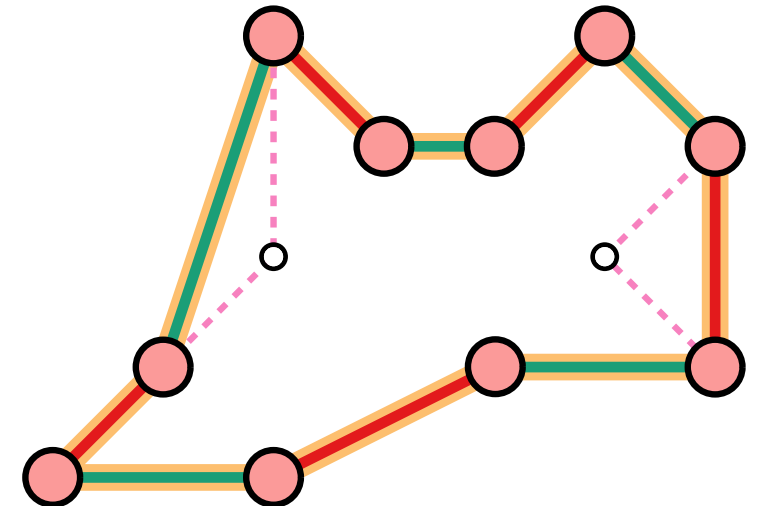
Analyse von CHRISTOFIDES



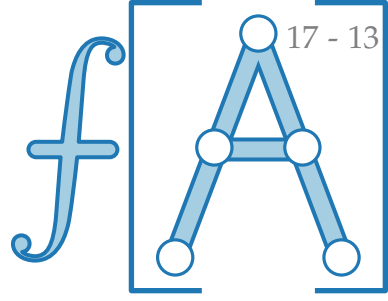
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2$



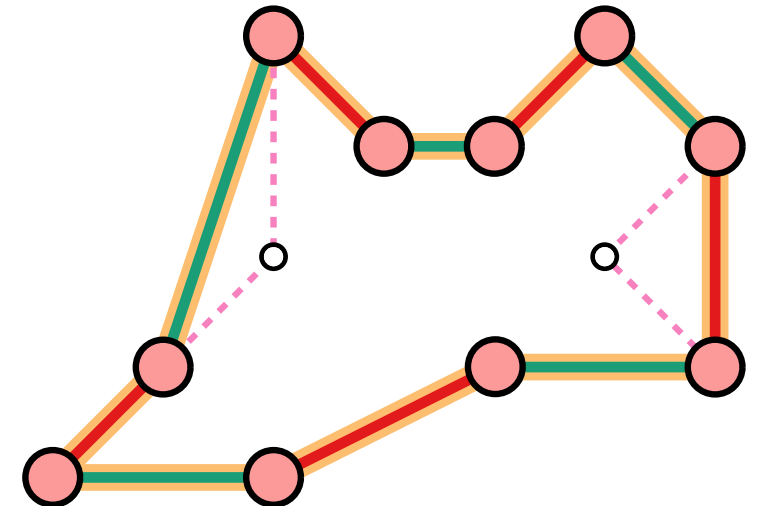
Analyse von CHRISTOFIDES



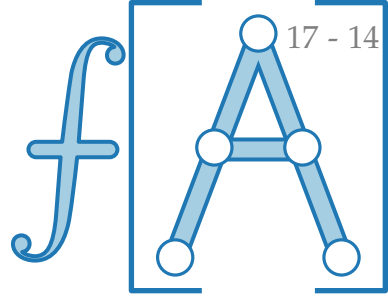
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2$



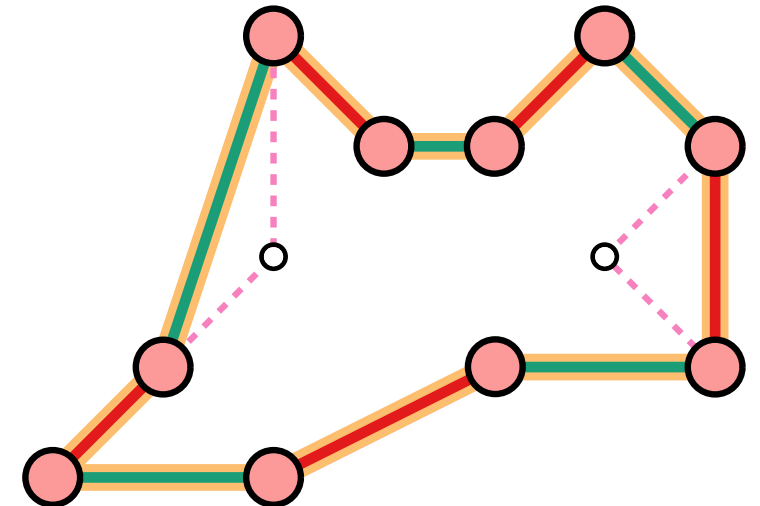
Analyse von CHRISTOFIDES



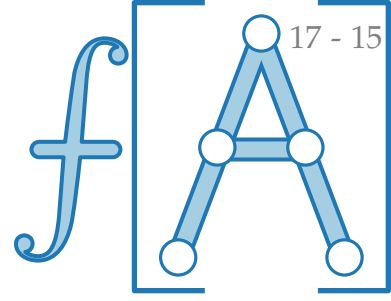
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2 \leq \text{OPT}/2$.



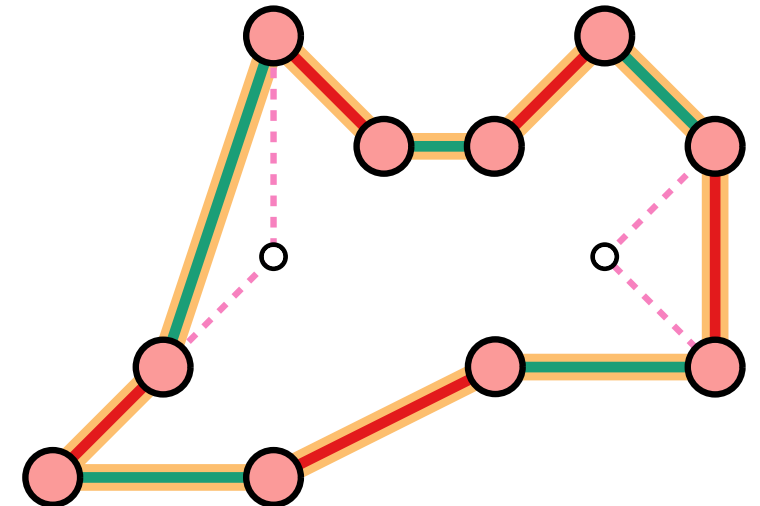
Analyse von CHRISTOFIDES



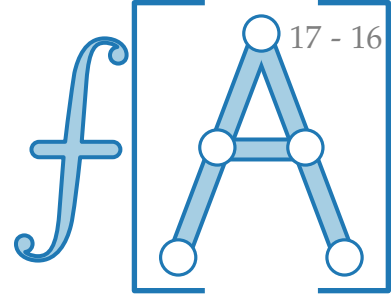
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2 \leq \text{OPT}/2$.
- Außerdem gilt $d(M) \leq d(M')$, da



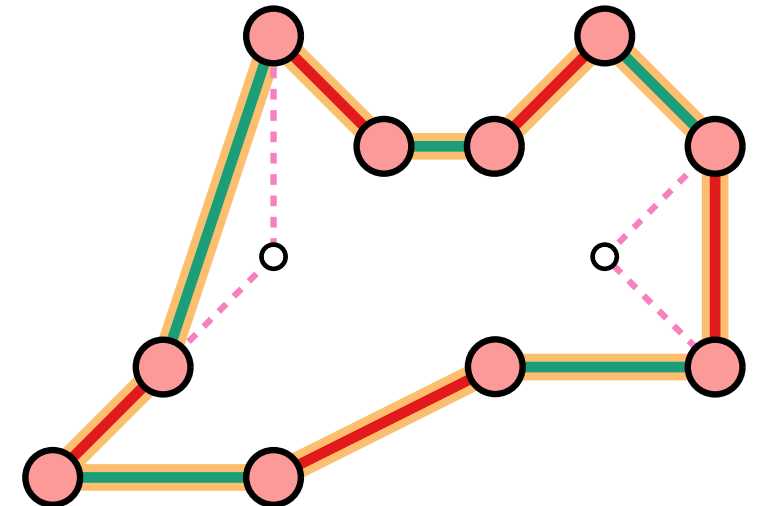
Analyse von CHRISTOFIDES



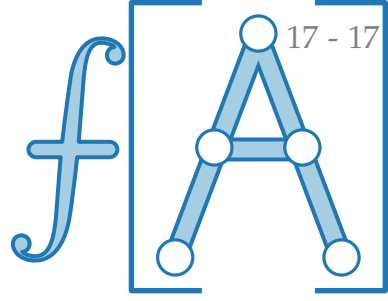
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2 \leq \text{OPT}/2$.
- Außerdem gilt $d(M) \leq d(M')$, da
 - M' perfektes Matching in $G[U]$



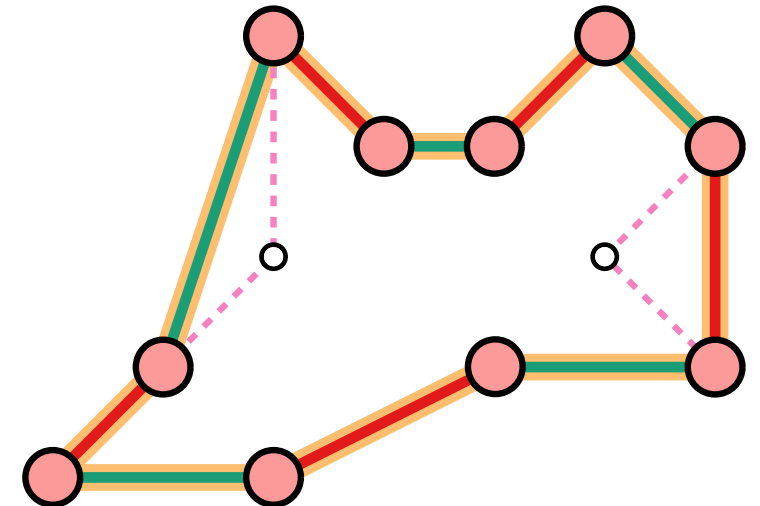
Analyse von CHRISTOFIDES



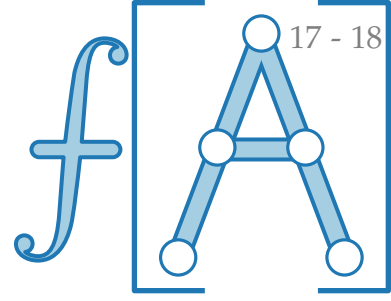
Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2 \leq \text{OPT}/2$.
- Außerdem gilt $d(M) \leq d(M')$, da
 - M' perfektes Matching in $G[U]$
 - M kostenminimales perfektes Matching in $G[U]$



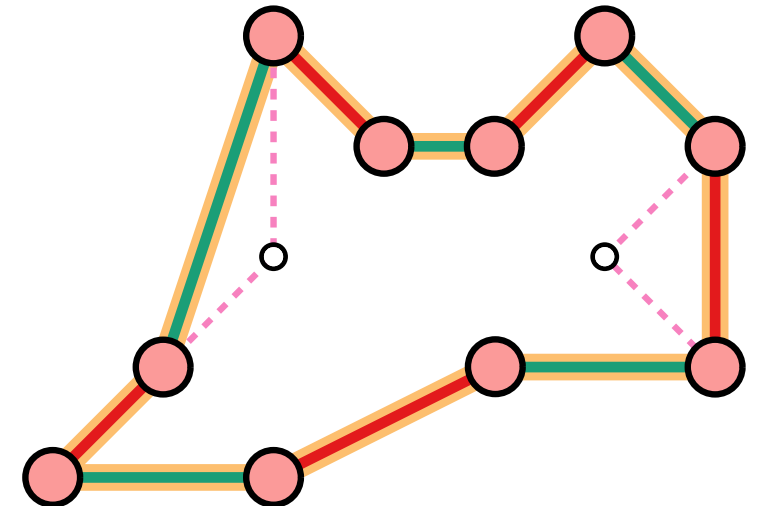
Analyse von CHRISTOFIDES



Satz. CHRISTOFIDES liefert eine $3/2$ -Approximation für Δ -TSP.

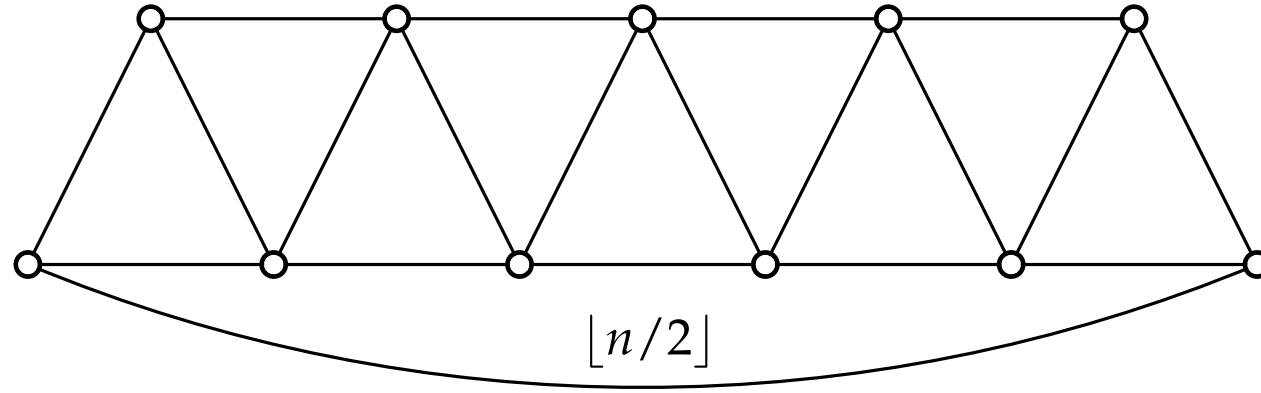
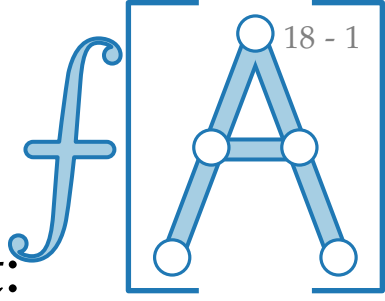
Beweis.

- Genügt zu zeigen, dass $d(B \cup M) \leq 3/2 \cdot \text{OPT}$, da $d(T) \leq d(B \cup M)$.
- Da $d(B) \leq \text{OPT}$, genügt es zu zeigen, dass $d(M) \leq \text{OPT}/2$.
- Betrachte optimale Tour T^* .
- Abkürzen von T^* liefert Tour T^{**} in $G[U]$ mit $d(T^{**}) \leq d(T^*) = \text{OPT}$.
- Zerlege in T^{**} in zwei disjunkte perfekte Matchings M', M'' für $G[U]$.
- O.B.d.A. sei $d(M') \leq d(M'')$.
- Also gilt $d(M') \leq d(T^{**})/2 \leq \text{OPT}/2$.
- Außerdem gilt $d(M) \leq d(M')$, da
 - M' perfektes Matching in $G[U]$
 - M kostenminimales perfektes Matching in $G[U]$ $\square$



Der Approximationsfaktor ist scharf

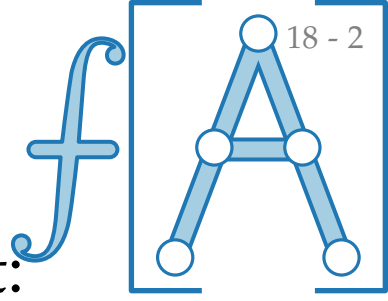
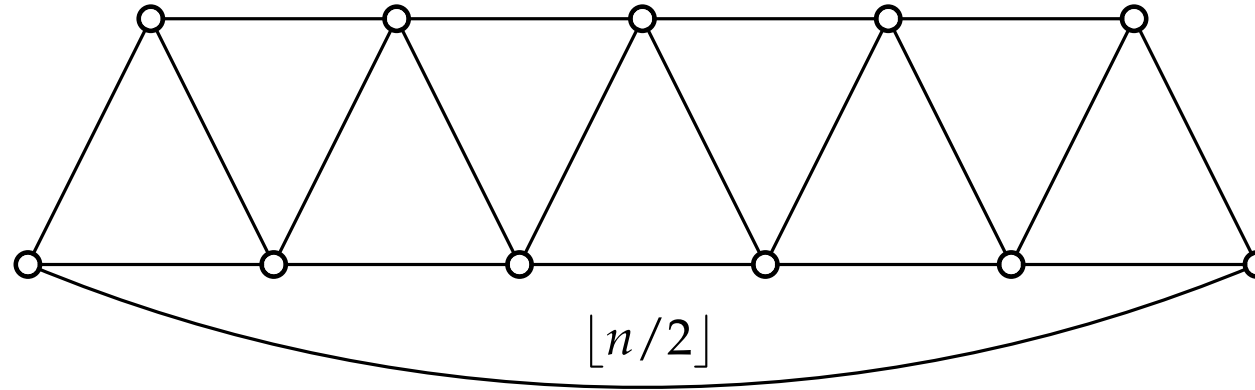
Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:



Der Approximationsfaktor ist scharf

Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

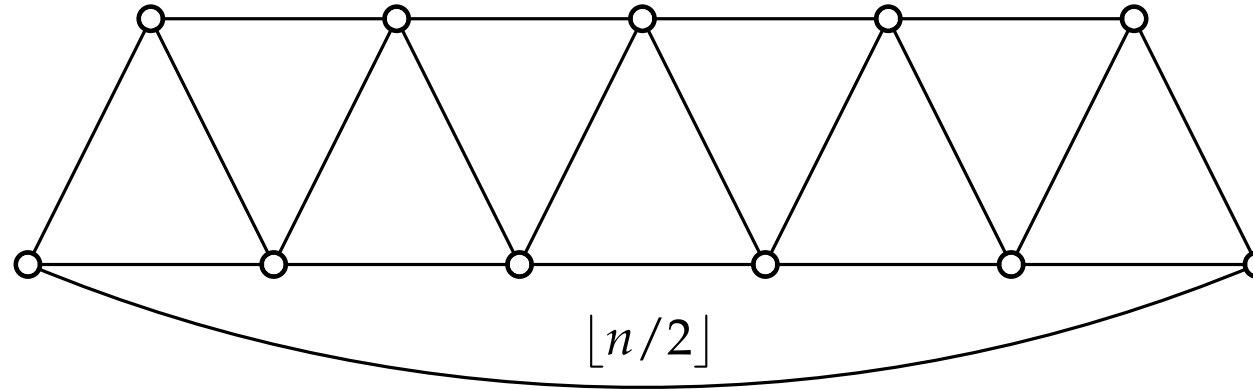
- Unbeschriftete Kanten haben Kosten 1.



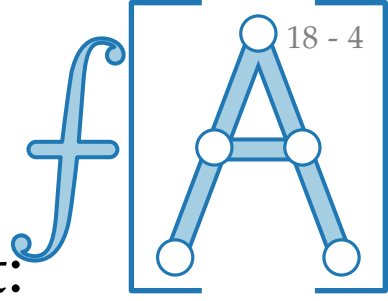
Der Approximationsfaktor ist scharf

Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$

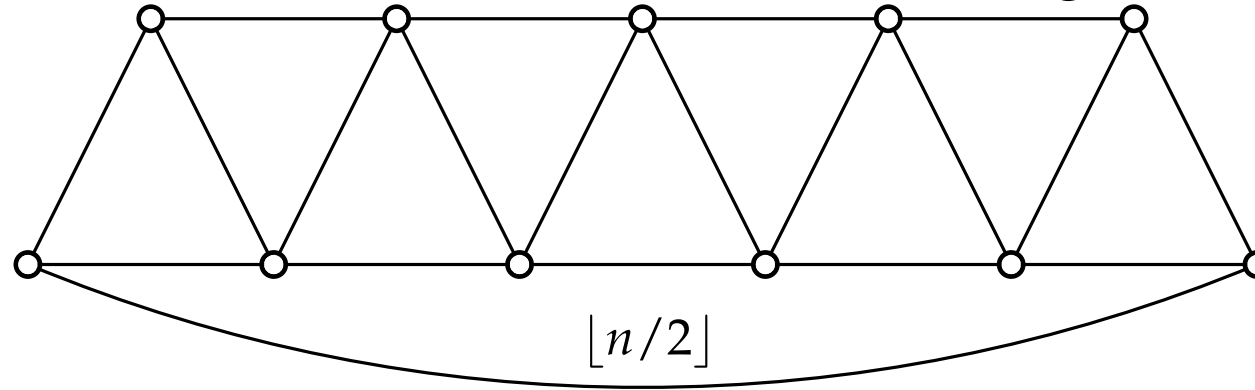


Der Approximationsfaktor ist scharf

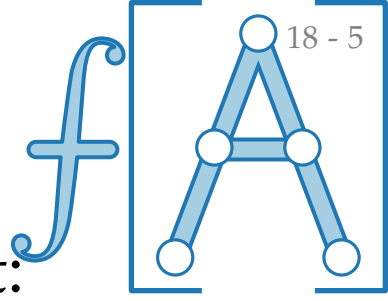


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges



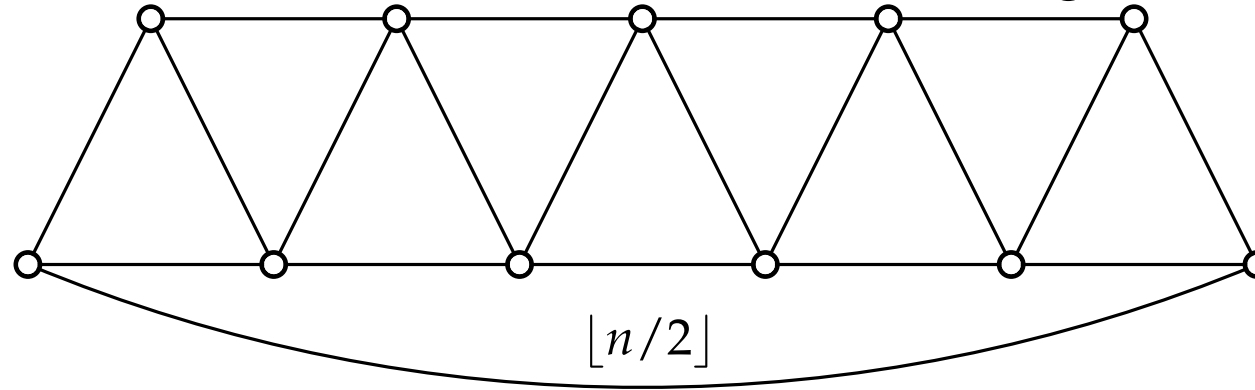
Der Approximationsfaktor ist scharf



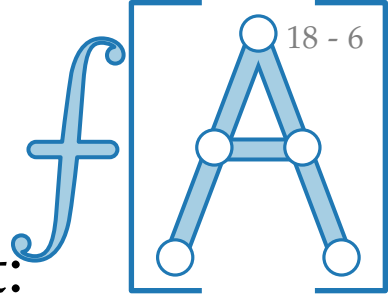
Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges

($\Rightarrow G$ metrisch!)



Der Approximationsfaktor ist scharf

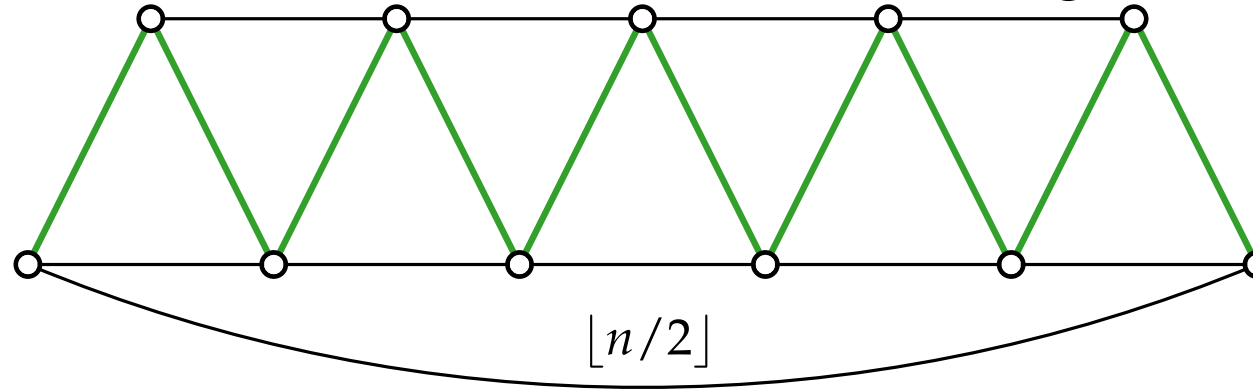


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

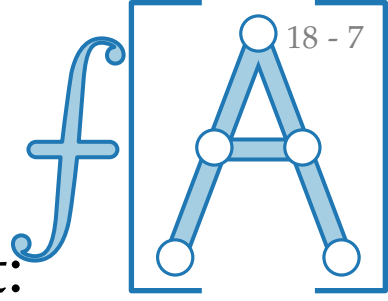
- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges

($\Rightarrow G$ metrisch!)

minimaler Spannbaum B



Der Approximationsfaktor ist scharf

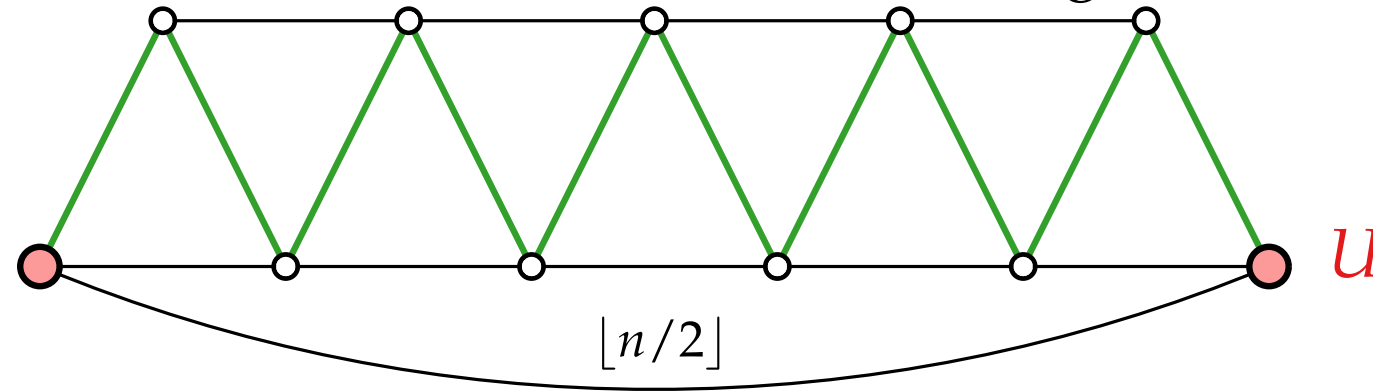


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

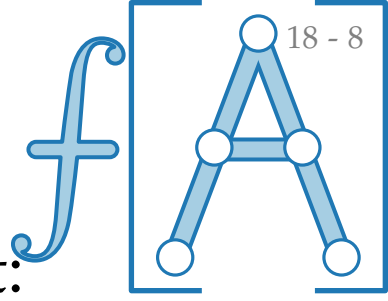
- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges

($\Rightarrow G$ metrisch!)

minimaler Spannbaum B



Der Approximationsfaktor ist scharf

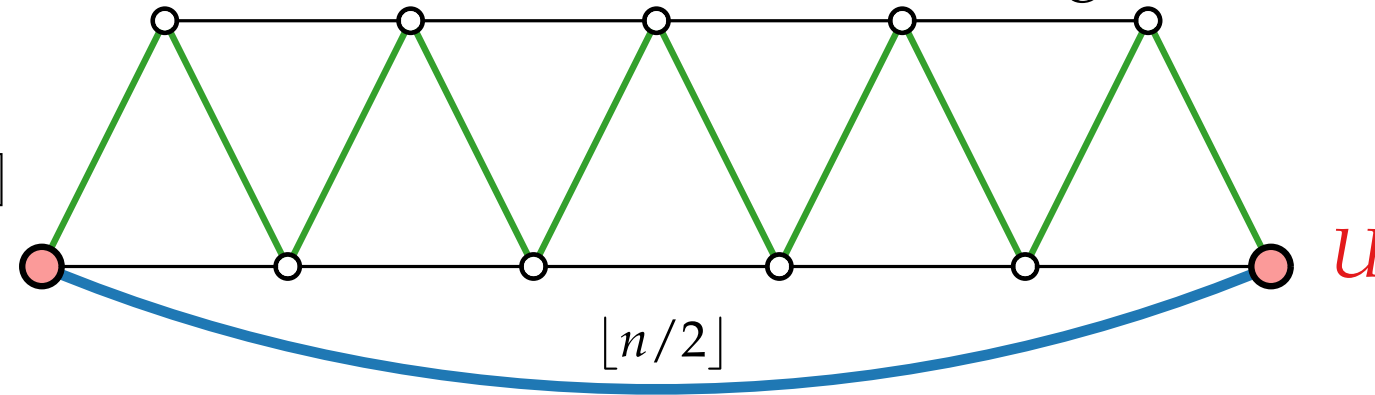


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

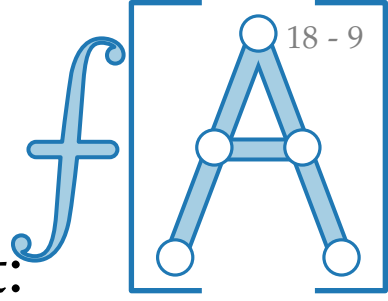
- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[u]$



Der Approximationsfaktor ist scharf

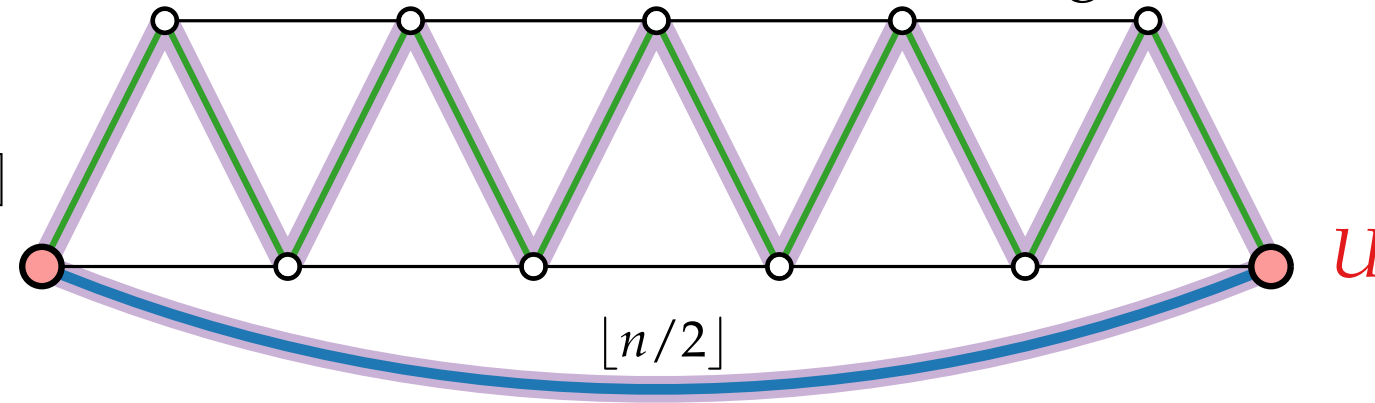


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

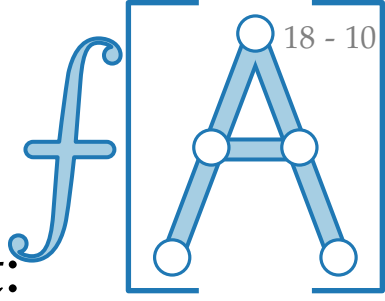
minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG =$

Der Approximationsfaktor ist scharf

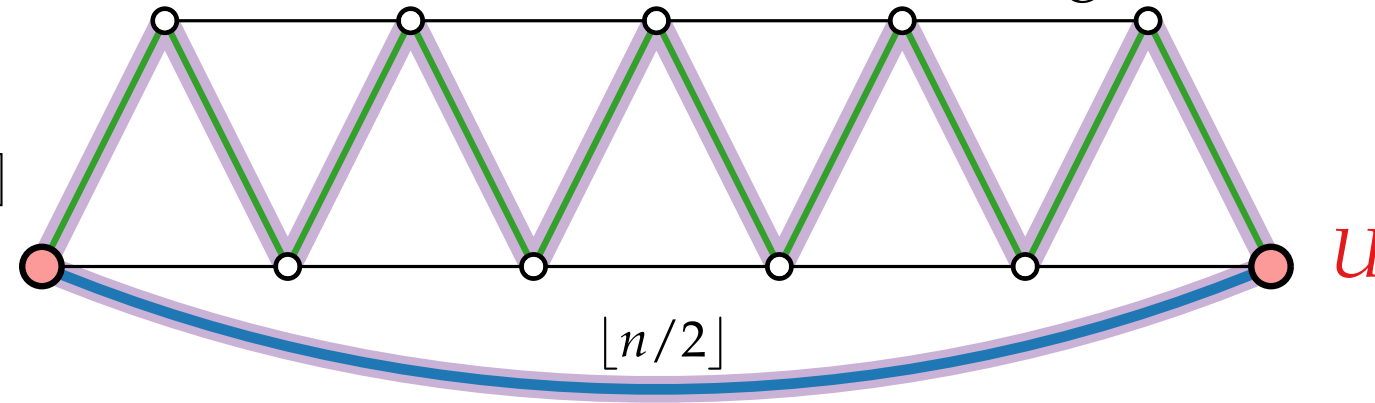


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

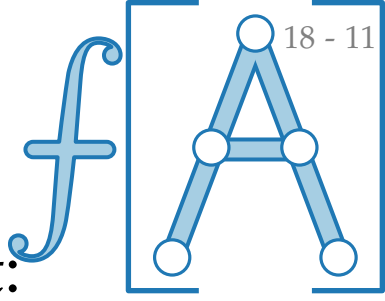
minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$

Der Approximationsfaktor ist scharf

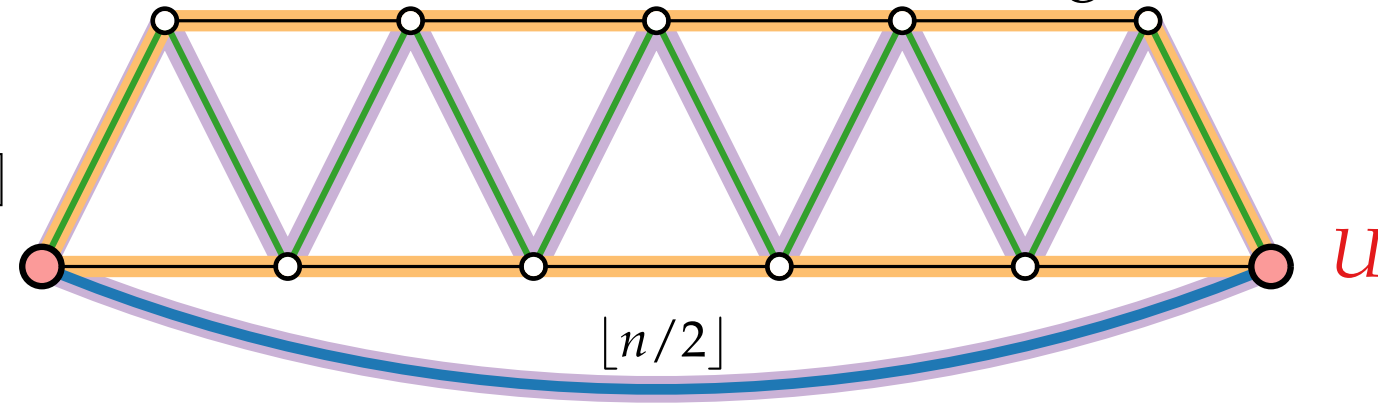


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

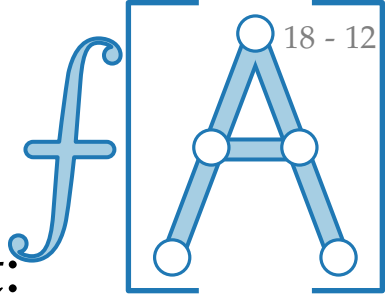
minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT =$

Der Approximationsfaktor ist scharf

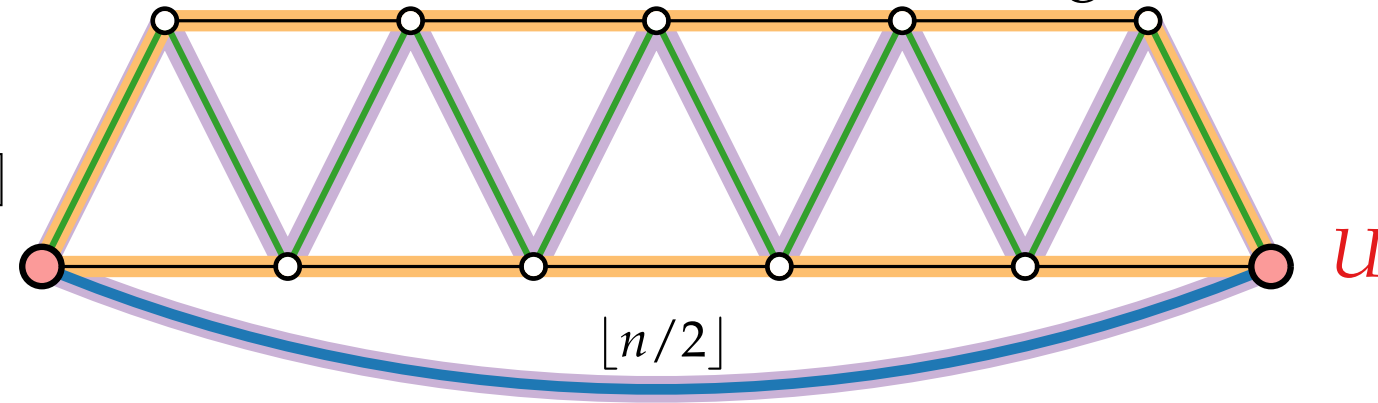


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

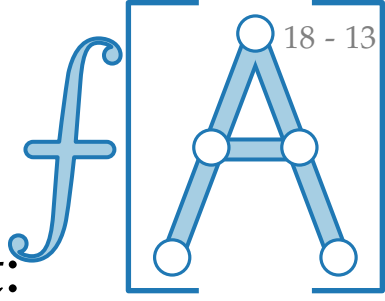
minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT = n$

Der Approximationsfaktor ist scharf

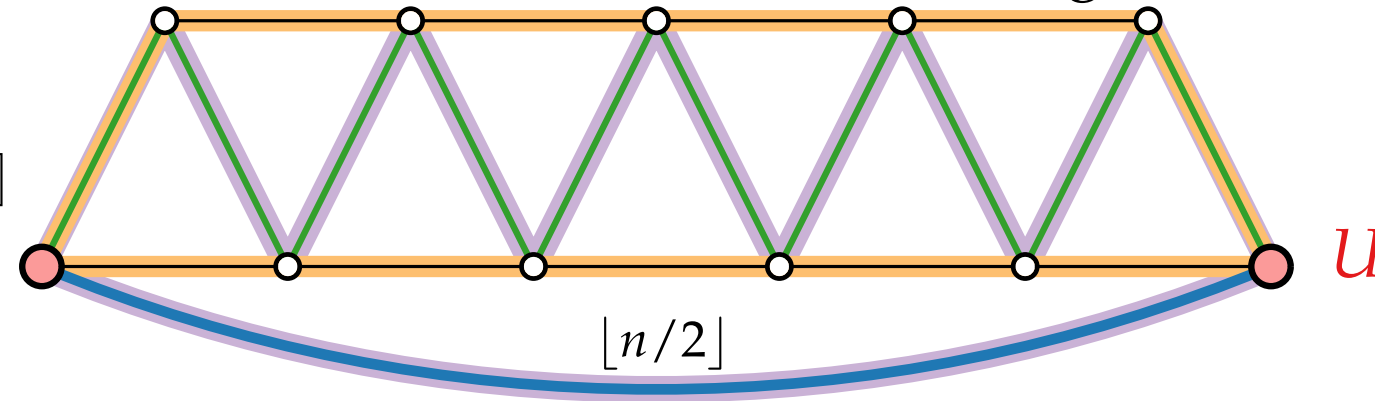


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT = n$

$ALG / OPT = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow$

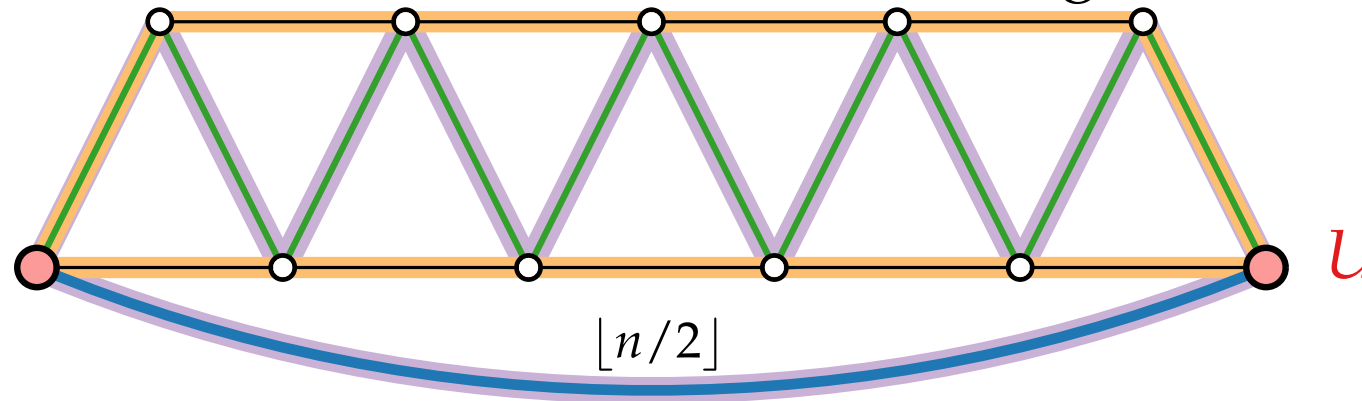
Der Approximationsfaktor ist scharf

Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges

minimaler Spannbaum B

perfektes Matching M in $G[U]$

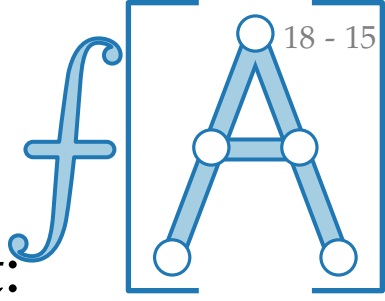


Christofides-Tour T mit Kosten $\text{ALG} = n + \lfloor n/2 \rfloor - 1$

$$\text{OPT} = n$$

$$\text{ALG} / \text{OPT} = (n + \lfloor n/2 \rfloor - 1)/n \rightarrow 3/2$$

Der Approximationsfaktor ist scharf

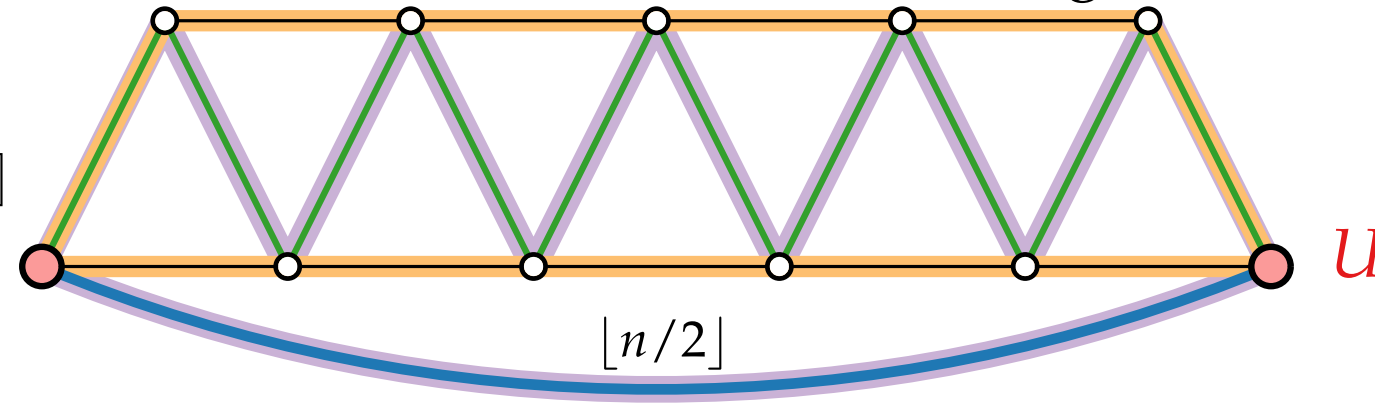


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

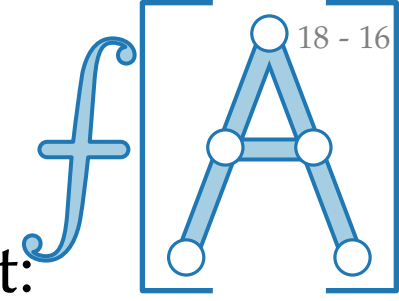
perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT = n$

$ALG / OPT = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow 3/2$ Also: Algorithmus nicht besser als $3/2$.

Der Approximationsfaktor ist scharf

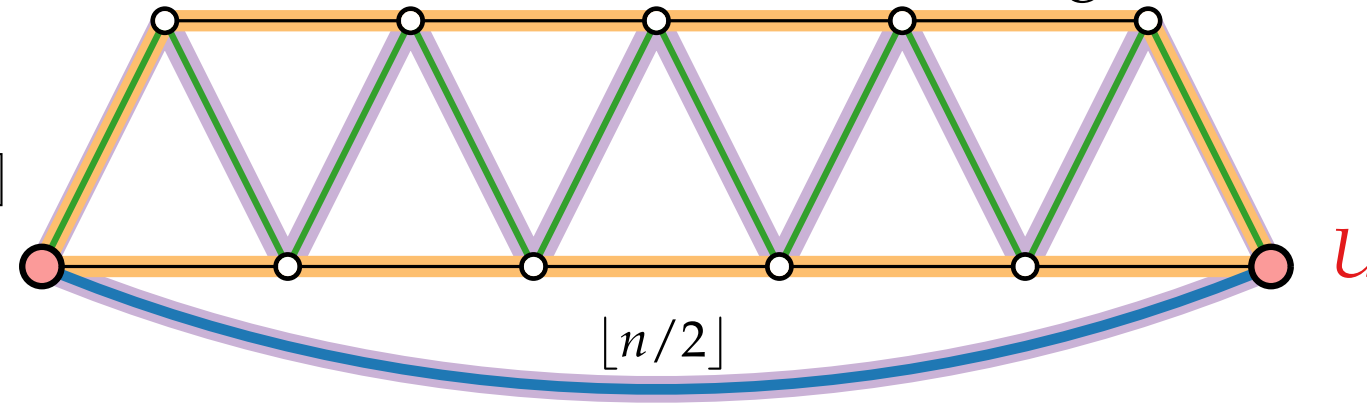


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[U]$

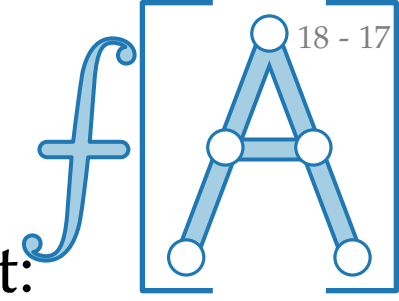


Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT = n$

$ALG / OPT = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow 3/2$ Also: Algorithmus nicht besser als $3/2$.

CHRISTOFIDES war lange Zeit der beste Approximationsalgorithmus für Δ -TSP.

Der Approximationsfaktor ist scharf

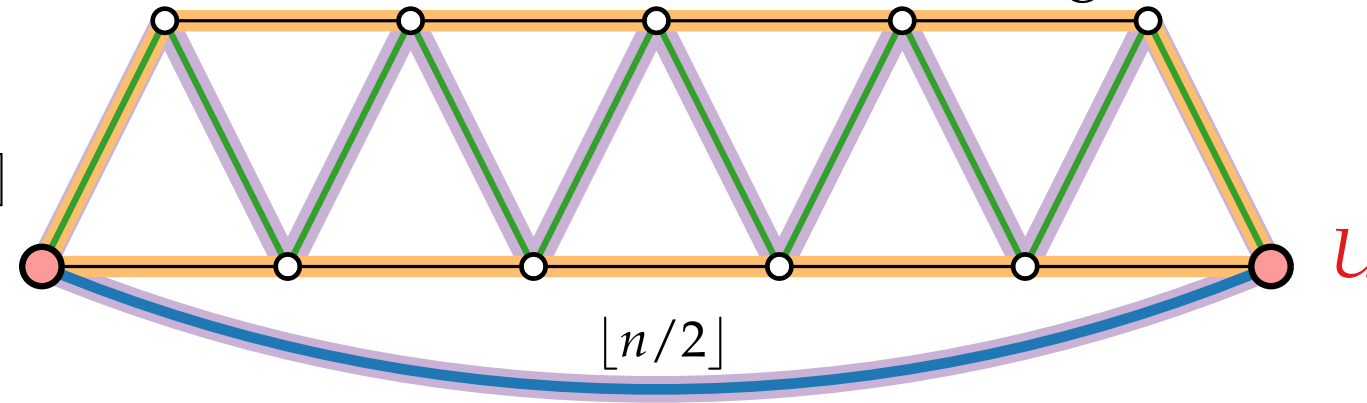


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[U]$

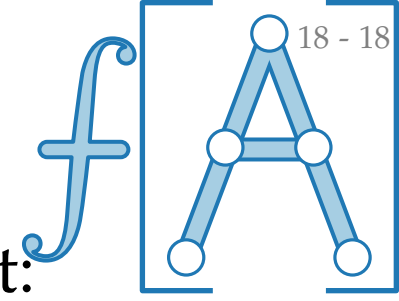


Christofides-Tour T mit Kosten $ALG = n + \lfloor n/2 \rfloor - 1$ $OPT = n$

$ALG / OPT = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow 3/2$ Also: Algorithmus nicht besser als $3/2$.

CHRISTOFIDES war lange Zeit der beste Approximationsalgorithmus für Δ -TSP. Bis...

Der Approximationsfaktor ist scharf

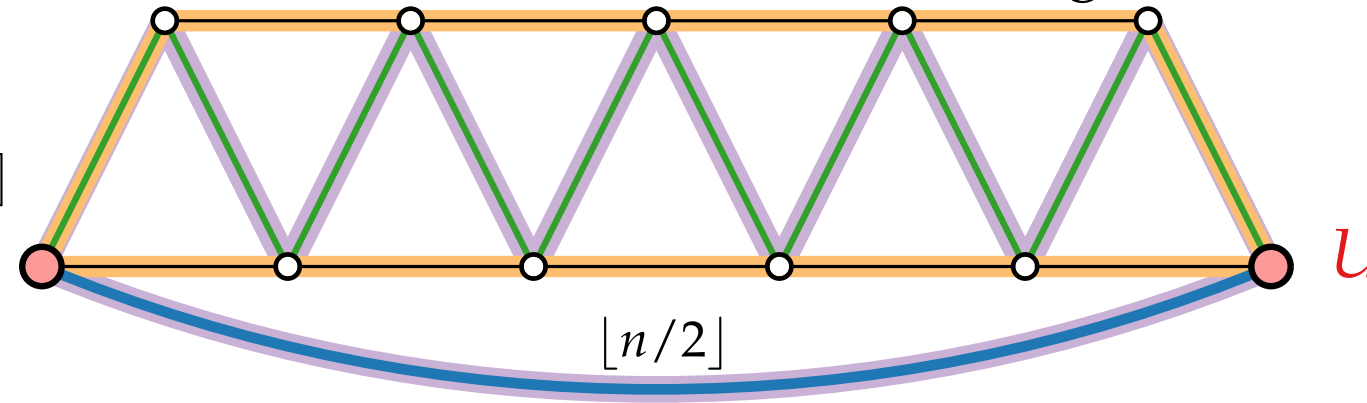


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $\text{ALG} = n + \lfloor n/2 \rfloor - 1$ $\text{OPT} = n$

$\text{ALG} / \text{OPT} = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow 3/2$ Also: Algorithmus nicht besser als $3/2$.

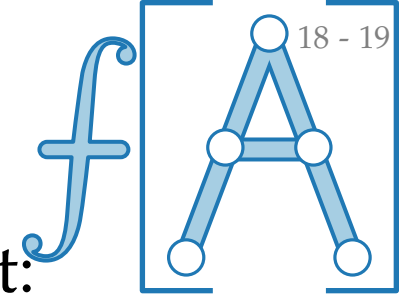
CHRISTOFIDES war lange Zeit der beste Approximationsalgorithmus für Δ -TSP. Bis...

Satz. Es gibt einen $(1.5 - 10^{-36})$ -Approximationsalgorithmus für metrisches TSP.

Der galaktische Algorithmus

[Karlin, Klein & Gharan, STOC 2021 (best paper)]

Der Approximationsfaktor ist scharf

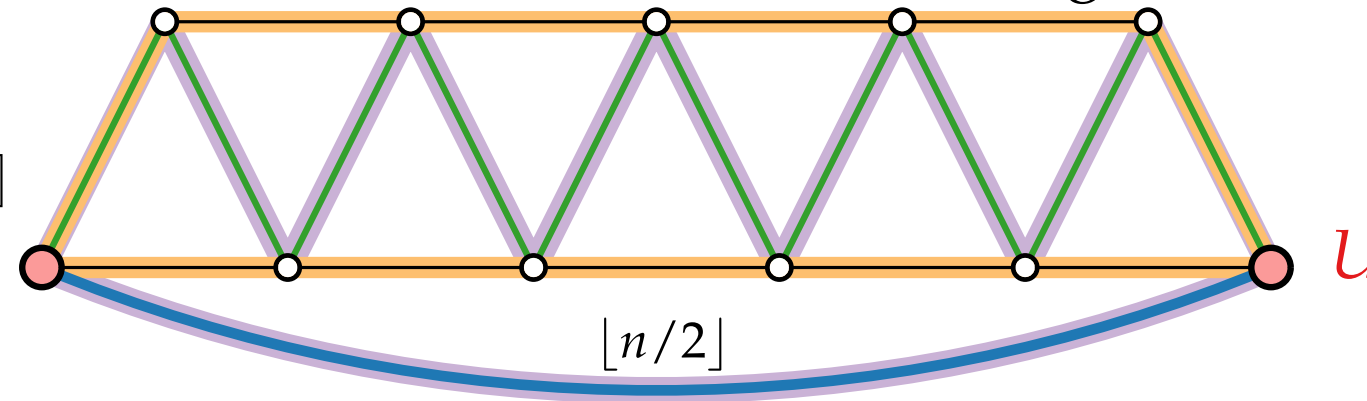


Konstruiere metrische Instanz G , für die CHRISTOFIDES möglichst **schlecht** ist:

- Unbeschriftete Kanten haben Kosten 1.
- Jede nicht gezeichnete Kante (u, v) hat Kosten $\equiv$ Länge eines kürzesten u - v -Weges ($\Rightarrow G$ metrisch!)

minimaler Spannbaum B

perfektes Matching M in $G[U]$



Christofides-Tour T mit Kosten $\text{ALG} = n + \lfloor n/2 \rfloor - 1$ $\text{OPT} = n$

$\text{ALG} / \text{OPT} = (n + \lfloor n/2 \rfloor - 1) / n \rightarrow 3/2$ Also: Algorithmus nicht besser als $3/2$.

CHRISTOFIDES war lange Zeit der beste Approximationsalgorithmus für Δ -TSP. Bis...

Satz. Es gibt einen $(1.5 - 10^{-36})$ -Approximationsalgorithmus für metrisches TSP.

Der galaktische Algorithmus

[Karlin, Klein & Gharan, STOC 2021 (best paper)]

Satz. Es gibt keinen $\frac{123}{122} \approx 1.0082$ -Approximationsalgorithmus für metrisches TSP (es sei denn $P = NP$). [Karpinski, Lampis & Schmied, JCSS 2015]